FPT UNIVERSITY

FPT UNIVERSITY
SoftwareDesignDocument

Frevia

GroupBar	
Group Members	Pham Kha Vy - CE182254 Nguyen Tien Thuan - CE181024 Tran Phat Tai - CE181632 Vo Nhut Tin - CE180619 Nguyen Lu Nhat Huy - CE181196
Supervisor	Mrs. Cao Hong Nga
Ext Supervisor	
Capstone Project code	FRE

- Can Tho, October 2026 -

Table of Contents

1 Overall Description **4**

1.1 Assumptions 4

1.2 Design Constraints 5

1.2.1 Platform Compatibility 5

1.2.2 Integration Complexity 5

1.2.3 Performance Requirements 5

1.3 Technology Suggestion 6

1.3.1 NestJS Framework 7

1.3.2 Next.js and React 19 Frontend 7

1.3.3 Prisma ORM and PostgreSQL 7

1.3.4 Supporting Services 7

2 System Architecture Design **8**

2.1 Overall Architecture 8

2.2 System Architecture 9

2.2.1 API Module Structure 9

2.3 Package Diagram 11

3 System Detailed Design **12**

3.1 Verify jwt 12

3.2 Authentication 13

3.2.1 UC-01 – Login with Google 15

3.2.2 UC-02 – Login 16

3.2.3 UC-03 – Logout 17

3.2.4 UC-04 – Register 18

3.2.5 UC-05 – Forgot Password 19

3.3 Role Management 20

3.3.1 UC-06.01 – View Role List 21

3.3.2 UC-06.02 – View Role Detail 22

3.3.3 UC-06.03 – Create Role 23

3.3.4 UC-06.04 – Update Role 24

3.3.5 UC-06.05 – Delete Role 25

3.3.6 UC-06.06 – Update Role Permissions 26

3.4 Permission Management 27

3.4.1 UC-15.01 – View Permission List 28

3.4.2 UC-15.02 – View Permission Detail 29

3.5 Session Management 29

3.5.1 UC-16.01 – View Session List . . . 31
3.5.2 UC-16.02 – View Session Detail . . . 32
3.5.3 UC-16.03 – Revoke Session . . . 33
3.6 Forum Category Management . . . 33
3.6.1 UC-17.01 – Create Forum Category . . . 35
3.6.2 UC-17.02 – Update Forum Category . . . 36
3.6.3 UC-17.03 – Delete Forum Category . . . 37
3.6.4 UC-17.06 – Restore Forum Category . . . 38
3.6.5 UC-14.01 – View Forum Category List . . . 39
3.6.6 UC-14.02 – View Forum Category Detail . . . 40
3.7 User Management . . . 40
3.7.1 UC-28.01 – View User List . . . 42
3.7.2 UC-28.02 – View User Detail . . . 43
3.7.3 UC-28.03 – Create User . . . 44
3.7.4 UC-28.04 – Update User . . . 45
3.7.5 UC-28.05 – Update Client Profile . . . 46
3.7.6 UC-28.06 – Update Freelancer Basic Info . . . 47
3.7.7 UC-28.07 – Update Freelancer Skills . . . 48
3.7.8 UC-28.08 – Update Freelancer Portfolio . . . 49
3.8 Banner Management . . . 49
3.8.1 UC-37.01 – View Banner List . . . 51
3.8.2 UC-37.02 – View Banner Detail . . . 52
3.8.3 UC-37.03 – Create Banner . . . 53
3.8.4 UC-37.04 – Update Banner . . . 54
3.8.5 UC-37.05 – Delete Banner . . . 55
3.9 Skill Management . . . 55
3.9.1 UC-38.01 – View Skill List . . . 57
3.9.2 UC-38.02 – View Skill Detail . . . 58
3.9.3 UC-38.03 – Create Skill . . . 59
3.9.4 UC-38.04 – Update Skill . . . 60
3.9.5 UC-38.05 – Delete Skill . . . 61
3.9.6 UC-38.06 – Restore Skill . . . 62
3.10 Expert Profile Management . . . 62
3.10.1 UC-28.09 – Update Expert Profile . . . 64
3.11 Job Category Management . . . 64
3.11.1 UC-46.01 – View Job Category List . . . 66
3.11.2 UC-46.02 – View Job Category Detail . . . 67
3.11.3 UC-46.03 – Create Job Category . . . 68

3.11.4 UC-46.04 – Update Job Category 69
3.11.5 UC-46.05 – Delete Job Category 70

4 Class Specification 71

4.1 AuthGuard Class 71
4.2 AuthController Class 72
4.3 AuthService Class 75
4.4 AuthRepository Class 79
4.5 RolesController Class 82
4.6 RolesService Class 83
4.7 RolesRepository Class 85
4.8 PermissionsController Class 87
4.9 PermissionsService Class 87
4.10 PermissionsRepository Class 88
4.11 SessionsController Class 89
4.12 SessionsService Class 90
4.13 SessionsRepository Class 92
4.14 Forum Category Administration (Shared Classes) 93
4.15 UsersController Class 93
4.16 UsersService Class 96
4.17 UsersRepository Class 99
4.18 BannersAdminController Class 102
4.19 BannersAdminService Class 104
4.20 BannersAdminRepository Class 106
4.21 SkillsAdminController Class 107
4.22 SkillsAdminService Class 109
4.23 SkillsAdminRepository Class 110
4.24 AdminExpertProfileController Class 112
4.25 ExpertProfileService Class 113
4.26 ExpertProfileRepository Class 115
4.27 JobCategoriesAdminController Class 116
4.28 JobCategoriesAdminService Class 118
4.29 JobCategoriesAdminRepository Class 119

1 Overall Description

1.1 Assumptions

This system is designed based on the following assumptions:

- The Node.js runtime and the frameworks chosen for the project (NestJS 11 for the back-end and Next.js 16 for the frontend) will continue to be actively supported and maintained, ensuring long-term stability and compatibility with evolving technologies.
- The RESTful API architecture will remain a scalable and efficient choice for separating the Next.js presentation layer from the NestJS business layer.
- PostgreSQL, accessed through Prisma ORM 7, will continue to provide reliable, scalable, and secure relational data storage suitable for handling user identity, jobs, contracts, escrow payments, and forum data.
- Third-party services used by the platform — Stripe (payments and escrow), Cloudinary (file storage), Google (OAuth login), the AI moderation/analysis service, and mail delivery via Nodemailer — will stay available and API-compatible during the project lifetime.
- Modern web browsers (e.g., Chrome, Edge, Firefox, Safari) will continue to support the front-end standards required by Next.js/React 19, including HTML5, CSS3 (TailwindCSS), JavaScript ESM, and responsive rendering with React Server Components.
- The application will be deployed in environments with stable internet access to support real-time messaging (Socket.io) and online payment processing.
- Users (Guests, Clients, Freelancers, Experts, and Admins) are familiar with basic web application navigation: browsing job boards, filling forms, managing profiles, and following payment flows.

This system design is also based on the following risks:

- Any breaking change in the major versions of Next.js, NestJS, or Prisma could require refactoring of the user interface components or of the data access layer.
- Cross-browser inconsistencies or outdated device software may affect rendering or the real-time messaging experience.
- Cybersecurity threats such as credential stuffing, token hijacking, or database breaches could compromise sensitive personal and financial information if not proactively managed (the design therefore includes bcrypt-hashed passwords, JWT access/refresh tokens, session revocation, and role-based access control).

- Outages or policy changes of the integrated third-party services (Stripe, Cloudinary, Google OAuth, AI moderation) could temporarily disrupt payments, file uploads, login, or content moderation.
- Network connectivity issues could disrupt access to job proposals, contract collaboration, or checkout flows that rely on server interaction.

1.2 Design Constraints

This system should comply with the following constraints:

1.2.1 Platform Compatibility

The Frevia application must be compatible with modern web browsers (e.g., Chrome, Edge, Firefox, Safari) for all user roles including Guest, Client, Freelancer, Expert, and Admin. The frontend is implemented with Next.js 16 (App Router, React Server Components) styled with TailwindCSS 4 and shadcn/ui components; the backend is a NestJS 11 REST API. Responsive web design principles must be applied so that a consistent user experience is provided across desktop, tablet, and mobile browsers. Internationalization is achieved with next-intl so the interface can be served in multiple languages from the same code base.

1.2.2 Integration Complexity

The Frevia platform integrates a variety of components: the Next.js web application, the NestJS REST API, the PostgreSQL database (via Prisma 7 with the pg adapter), Stripe Checkout and Stripe Connect for escrow payments and payouts, Cloudinary for image and document storage, Socket.io for real-time chat and notifications, Google OAuth for federated login, e-mail for OTP delivery, and AI services for content moderation, CV parsing, and identity document verification. Communication between the client and the server is carried out through versioned REST endpoints secured with JWT access tokens and refresh tokens stored in dedicated session records; role-based access control restricts each operation to the allowed roles. Design decisions must keep these integrations loosely coupled behind services and repositories so that a provider can be replaced without rewriting business logic.

1.2.3 Performance Requirements

The application must remain responsive under concurrent usage by many freelancers and clients. Job browsing, proposal listing, forum feeds, and conversation histories must load promptly; therefore the design mandates server-side pagination for every list operation, database

indexes on frequently filtered columns, optimized Prisma queries executed inside transactions for counted lists, and streaming React Server Components to reduce time-to-first-byte. Real-time features (messaging, notifications) must propagate events with minimal delay via Socket.io. Payment operations must reconcile eventual states through Stripe webhooks and explicit synchronization endpoints.

1.3 Technology Suggestion

Frevia adopts a RESTful API architecture to support secure, stateless, and scalable communication between the Next.js frontend and the NestJS backend. This approach provides:

- Modular, reusable API endpoints for account management, job posting, proposals, contracting, milestone escrow, dispute resolution, community forum, and administration.
- Easy extension for mobile applications or third-party integrations in the future.
- Secure access using JWT tokens with role-based authorization for every endpoint.
- Uniform use of standard HTTP methods (GET, POST, PATCH, PUT, DELETE), making the system intuitive to maintain.
- A clear separation between the presentation layer and the business logic layer.

All request/response payloads are validated and typed with shared Zod schemas packaged in `@shared/types`, which is consumed by both the frontend and the backend so that a single source of truth prevents contract drift between the two applications.

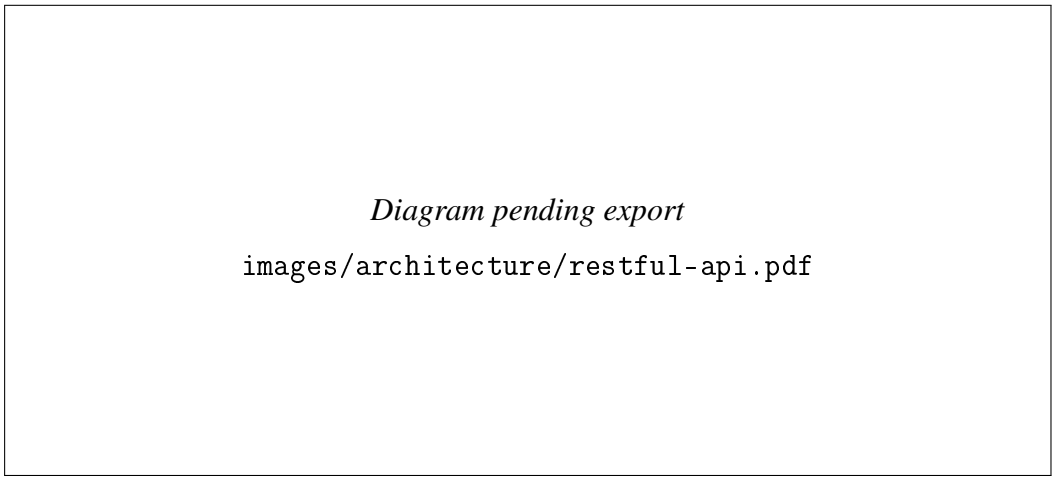


Figure 1: RESTful API communication between the Next.js frontend and the NestJS backend

1.3.1 NestJS Framework

The backend of Frevia is built with NestJS 11, a TypeScript-first, modular web framework running on Node.js. It is selected for its:

- Rich dependency-injection and module ecosystem, matching the Controller–Service–Repository layering.
- First-class decorator-based REST routing with guards and interceptors for authentication, roles, permissions, and response serialization.
- Excellent support for Zod schema validation (nestjs-zod) and typed DTOs.
- Component structure for WebSockets (Socket.io gateways) used by the messaging module.
- Straightforward integration with Prisma ORM and PostgreSQL for persistence.

1.3.2 Next.js and React 19 Frontend

The web client uses Next.js 16 with the App Router. React Server Components render public pages (job lists, job details, forum) directly from the server, while TanStack Query drives interactive islands (forms, likes, real-time chat). react-hook-form together with Zod resolvers validate user input with the same schemas the API uses.

1.3.3 Prisma ORM and PostgreSQL

Persistence relies on PostgreSQL accessed through Prisma 7. The schema defines 62 models covering identity and access, profiles, jobs, proposals, contracts and milestones, payments and disputes, reviews, messaging, the community forum, and platform configuration. Prisma's type-safe client keeps data access code verifiable at compile time.

1.3.4 Supporting Services

Payments and payouts use Stripe Checkout and Stripe Connect with webhook-driven reconciliation; binaries are stored in Cloudinary; emails (OTP, notifications) are sent with Nodemailer; Google login uses OAuth 2.0; environment configuration is loaded through Infisical so no secret is committed to the repository.

2 System Architecture Design

2.1 Overall Architecture

Frevia follows a three-tier, deployment-separated architecture (Figure 2):

- **Presentation tier — Next.js 16 web application:** server components render public pages (job board, job detail, community forum, expert directory) and private dashboards; client components handle interactivity such as forms, likes, and chat. All server access goes through a single typed HTTP client that attaches the JWT access token and refreshes it transparently when it expires.
- **Application tier — NestJS REST API:** a modular monolith organized into 31 route modules. Every module is internally layered as Controller → Service → Repository. Controllers validate and serialize with shared Zod DTOs; services hold business rules and authorization checks; repositories encapsulate all Prisma queries.
- **Data tier — PostgreSQL via Prisma 7:** the relational store for 62 domain models. Binary files are never stored in the database but in Cloudinary; real-time transport for chat and notifications is handled by a Socket.io gateway beside the REST API.

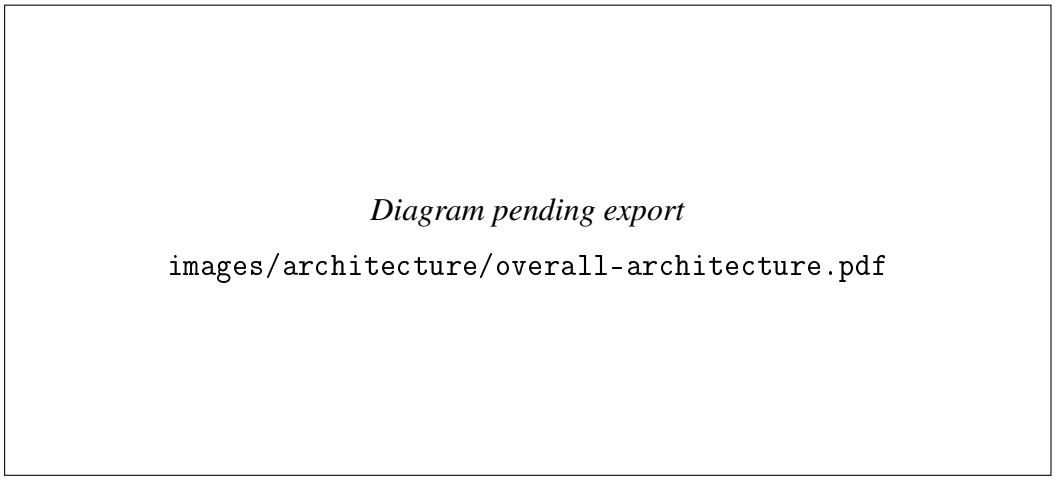


Figure 2: Overall architecture of the Frevia system

The two applications are deployed independently (the web app on Vercel, the API on a Node.js host) and communicate only through the documented REST contract validated by the shared Zod schemas package.

2.2 System Architecture

The internal architecture of the API follows a strict layered pattern, enforced identically in every module (Figure 3):

- **Controller layer** — thin Routing Layer. Exposes REST endpoints, applies guards (authentication, roles, permissions), binds validated request DTOs, and delegates to a service. Controllers never query the database directly.
- **Service layer** — business logic layer. Implements use cases, ownership/authorization checks, state-machine transitions (proposal, contract, dispute), and cross-aggregation logic; it throws typed HTTP exceptions (`NotFoundException 404`, `ForbiddenException 403`, `ConflictException 409`) that a global exception filter converts to a uniform error response.
- **Repository layer** — data access layer. The only component allowed to touch Prisma; complex list queries (pagination + counting) are wrapped in `$transaction` for consistency.
- **Shared kernel** — the `@shared/types` workspace package contains Zod models, constants, permission names, and error codes shared with the web app.

Security flow: every request carries a JWT access token; the auth guard validates it and resolves the active role; role and permission guards authorize module-specific operations; refresh tokens are stored server-side in a `Session` row and can be revoked from the session management feature.

AI-assisted pipeline: user-generated content (forum posts/comments, CVs, identity documents) is passed through AI moderation or AI analysis services; forum content can be auto-approved when clean, or queued in `ModerationQueue` for administrator approval when suspicious.

2.2.1 API Module Structure

Table 1 summarizes the 31 route modules of the API and their responsibilities.

Table 1: API route modules and responsibilities

Module	Responsibility
auth	registration, OTP, login, refresh token, logout, forgot password, Google OAuth, current account, role switching/joining
sessions	device session listing, detail, revocation

Module	Responsibility
account-profile	general account profile, avatar, identity verification upload/AI check, client profile, social links, favorite freelancers, following
users	administrator user management and skill catalog
roles, permissions	RBAC role and permission administration
profiles	professional profile, CV upload/AI analysis, skills
portfolios	portfolio item CRUD on profiles
expert-profile	expert profile management and public expert directory
profile-revisions	profile change request approval workflow
identity-verifications-admin	identity verification review
job-categories	public job category browsing
job-categories-admin	job category administration (soft delete + restore)
skills-admin	skill catalog administration (soft delete + restore)
browse-job	public job listing and job detail by slug
manage-job	client job management, bookmarks, skill search
proposals	proposal lifecycle: draft, submit, withdraw, interview, reject, accept
contract	contract lifecycle: create, update, sign, complete, cancel
milestone	milestone management inside a contract
milestone-file, milestone-submission	milestone attachments and work submission review
shared-file	contract shared workspace files
payments	Stripe Connect onboarding, platform fee, milestone funding, release, refund, dispute fee, transactions, webhook
disputes	dispute lifecycle (user side and admin arbitration side)
reviews	contract reviews and responses
conversations	direct messaging with real-time delivery
notifications	notification feed management
forums	forum categories and posts (public side)
forums-comment, forums-like, forums-reports	comments, likes, content reports
forums-admin	AI moderation queue, pending posts, trash restore, forum statistics
saved-search	saved job search filters
banners, banners-admin	advertisement banner delivery and administration

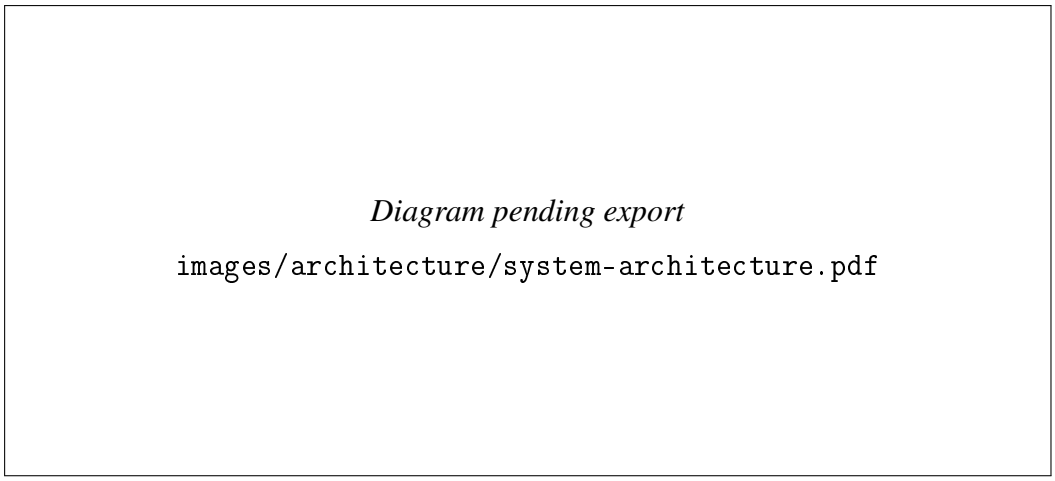


Figure 3: Layered system architecture with security and AI moderation flows

2.3 Package Diagram

The repository is a pnpm monorepo managed by Turborepo with three workspaces (Figure 4):

- `apps/web` — the Next.js application. Sub-packages group screens by actor (guest browsing, client, freelancer, admin), shared UI components (`@repo/ui`), server actions, and the i18n message catalogs used by next-intl.
- `apps/api` — the NestJS application. `src/routes/*` contains the 31 domain modules described above; `src/shared` contains cross-cutting infrastructure: authentication guards and decorators, permission constants, global HTTP exception filter, Zod serializer interceptor, Cloudinary storage service, and Prisma client setup. Each domain module folder is organized as `<module>.controller.ts`, `<module>.service.ts`, `<module>.repository.ts`, DTO classes, and module-specific error factories (e.g., `forums.error.ts`).
- `packages/shared-type` — the single source of truth for API contracts: Zod request/response models (`model/*`), enumeration/constant definitions, and permission name lists, imported by both `apps/web` and `apps/api`.

Dependencies only point downwards: `web` and `api` both depend on `shared-type`; no route module imports another route module — cross-module data needs flow through services at the Prisma schema level.

Diagram pending export

`images/architecture/package-diagram.pdf`

Figure 4: Package diagram of the Frevia monorepo

3 System Detailed Design

3.1 Verify jwt

Verify JWT is the shared security guard protecting API endpoints across all modules. Every incoming HTTP request with a Bearer token is intercepted, validated for cryptographic signature and expiration time, and verified against the database to guarantee the associated session remains active and has not been revoked.

a. Class Diagram

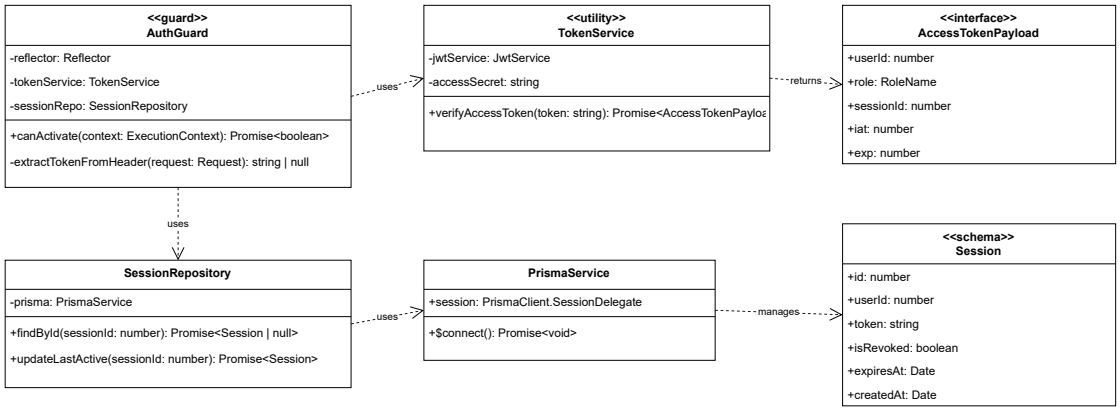


Figure 5: Class diagram of Verify JWT

b. Sequence Diagram(s)

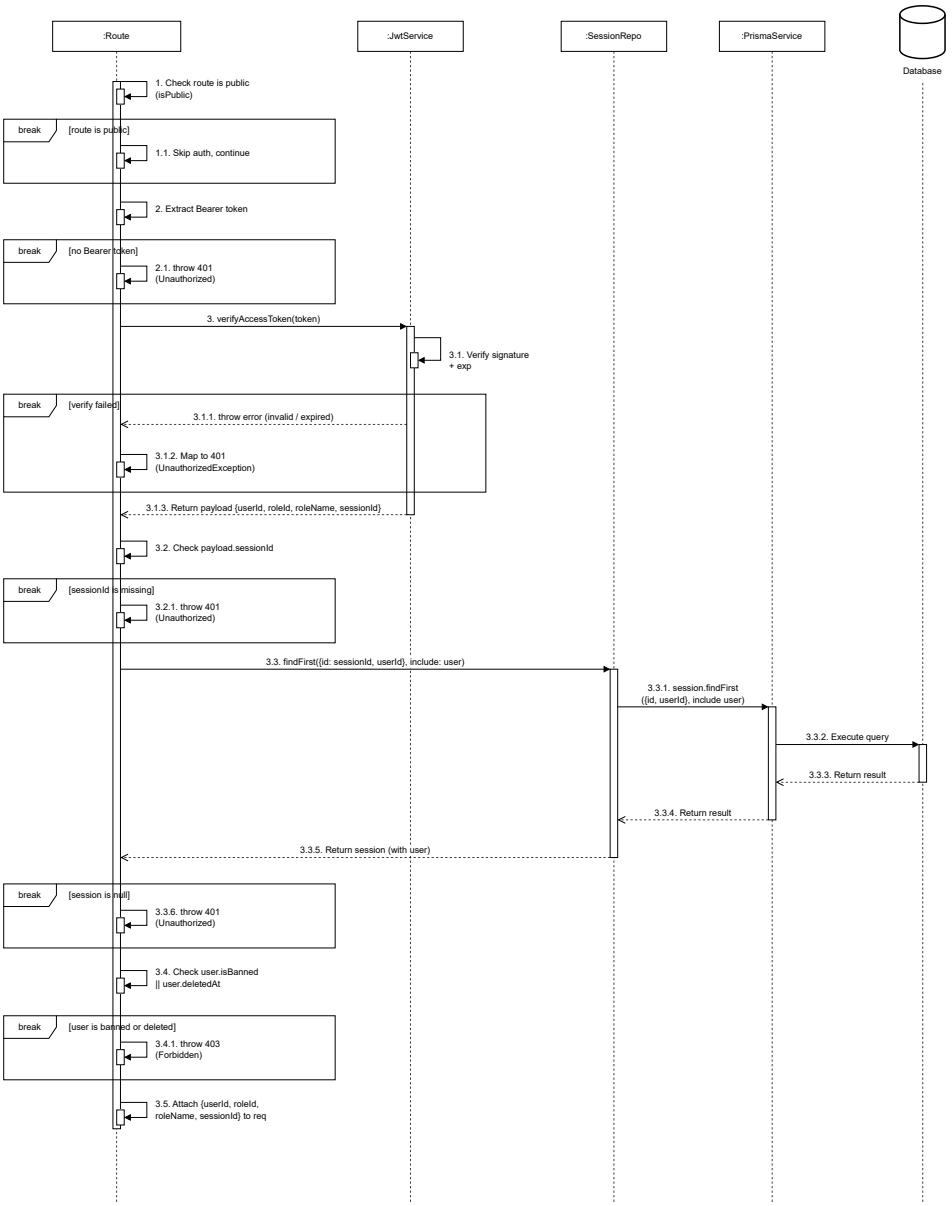


Figure 6: Sequence diagram of Verify JWT

3.2 Authentication

The authentication module covers account registration, email/password and Google OAuth login, logout, password recovery, and the JWT/session flow shared by all protected endpoints.

a. Class Diagram

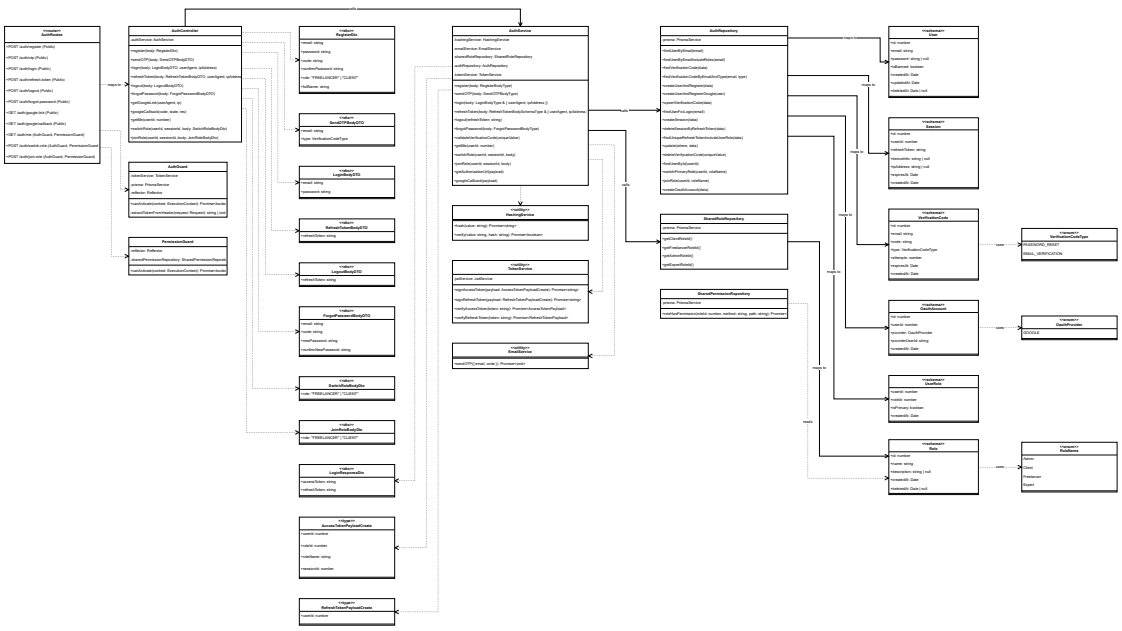


Figure 7: Class diagram of Authentication

b. Sequence Diagram(s)

3.2.1 UC-01 – Login with Google

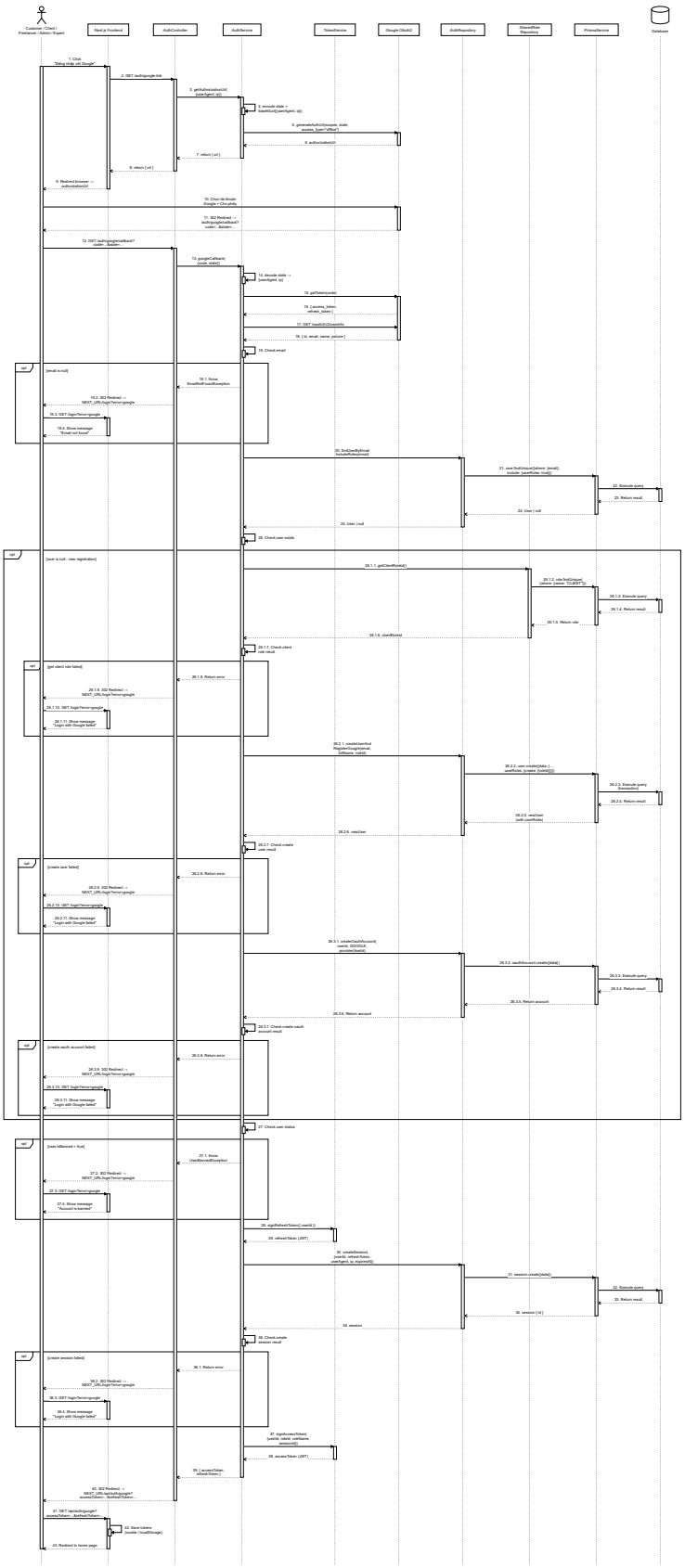


Figure 8: Sequence diagram of UC-01 Login with Google

3.2.2 UC-02 – Login

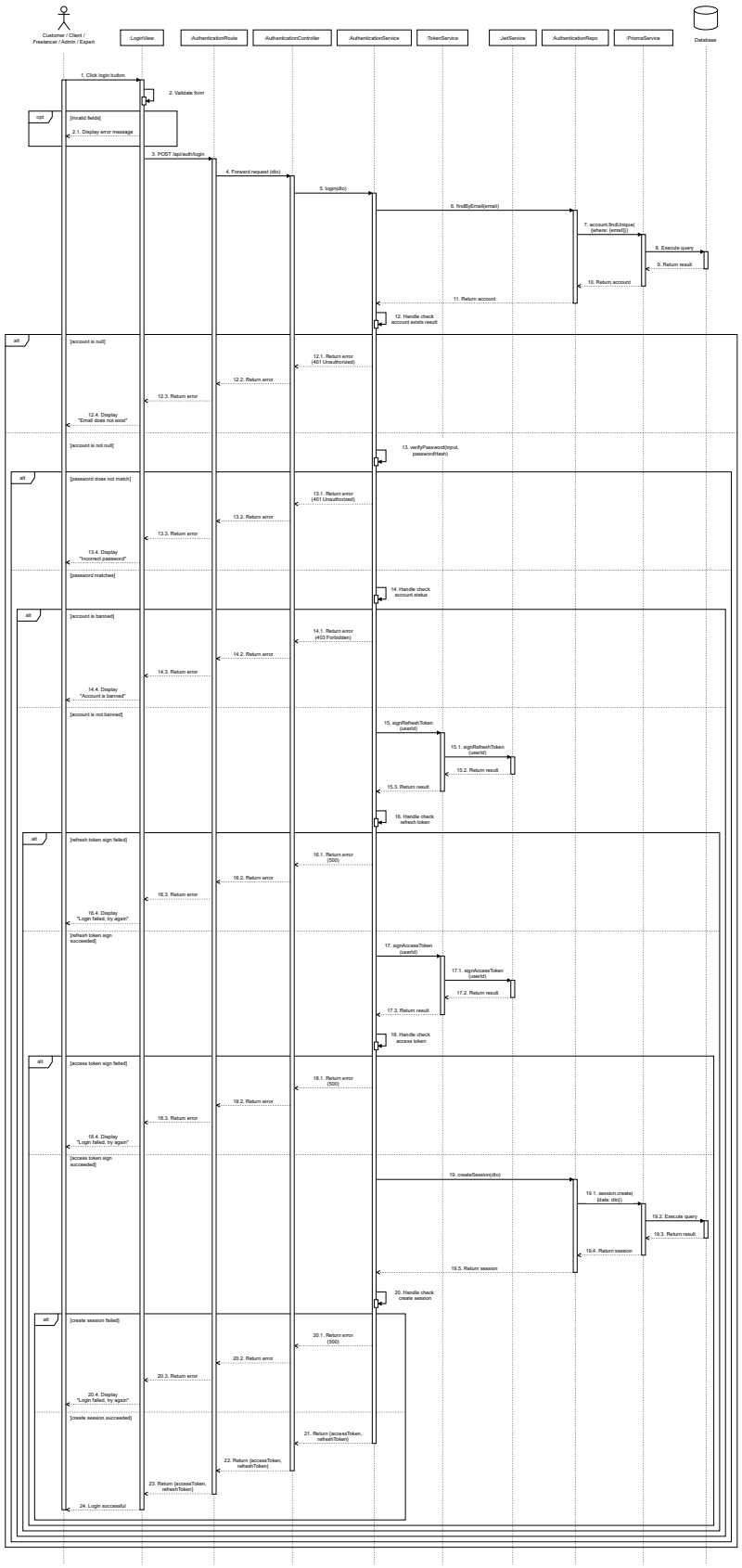


Figure 9: Sequence diagram of UC-02 Login

3.2.3 UC-03 – Logout

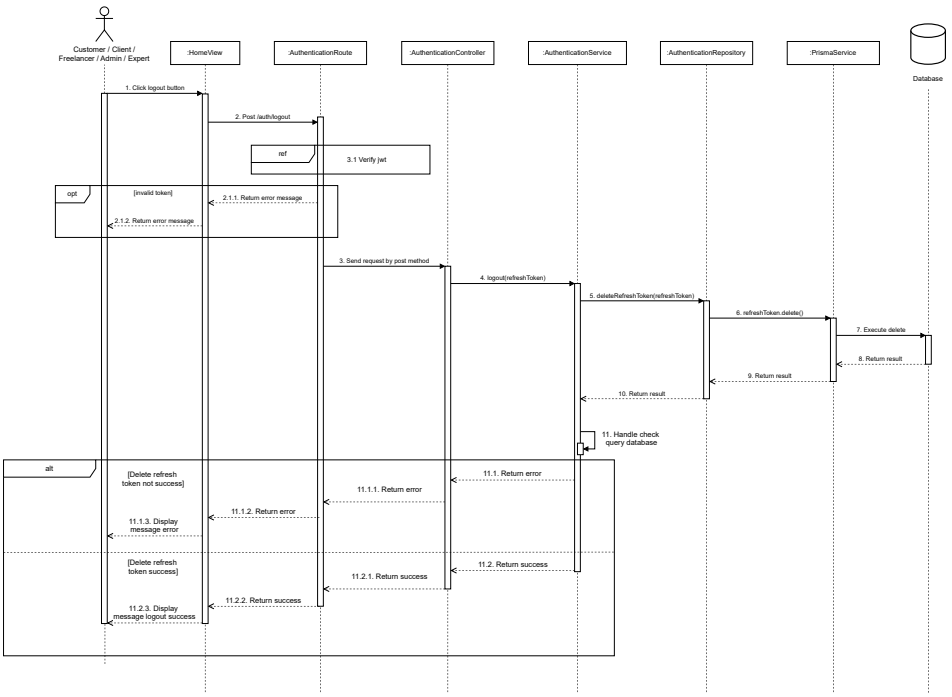


Figure 10: Sequence diagram of UC-03 Logout

3.2.4 UC-04 – Register

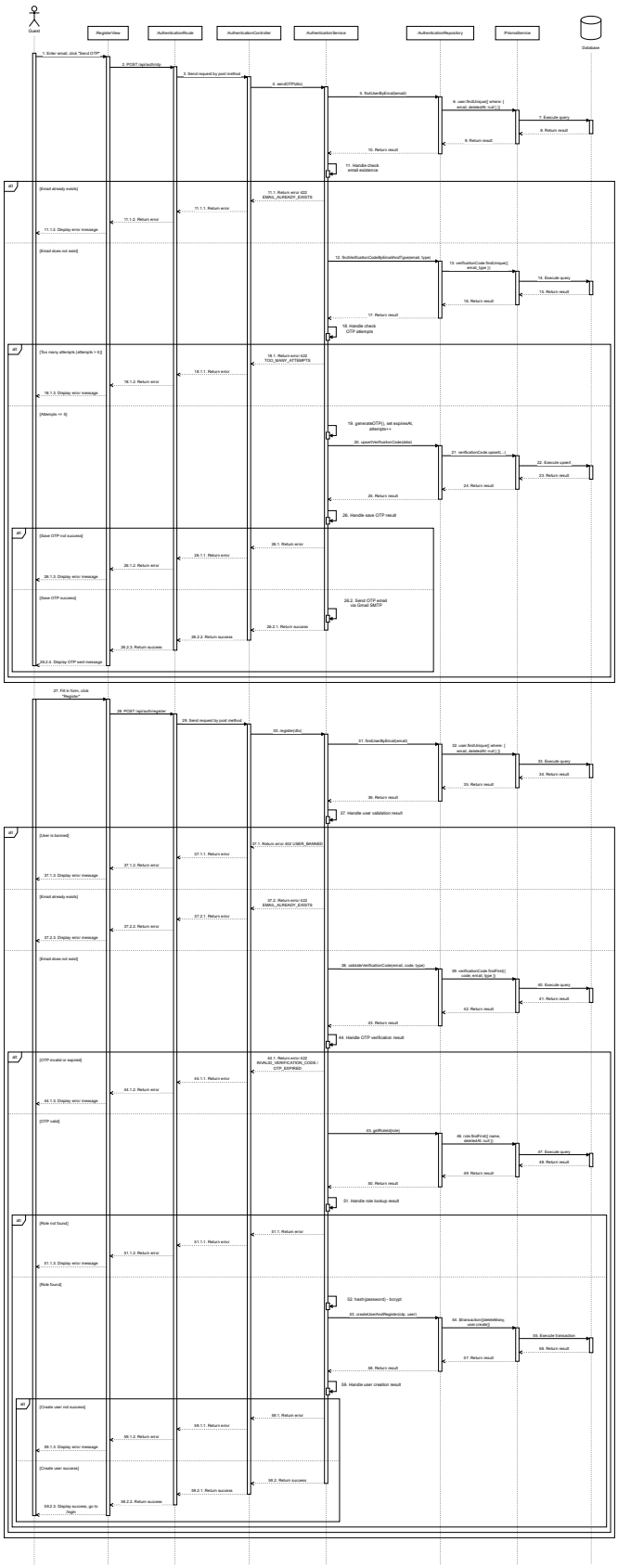


Figure 11: Sequence diagram of UC-04 Register

3.2.5 UC-05 – Forgot Password

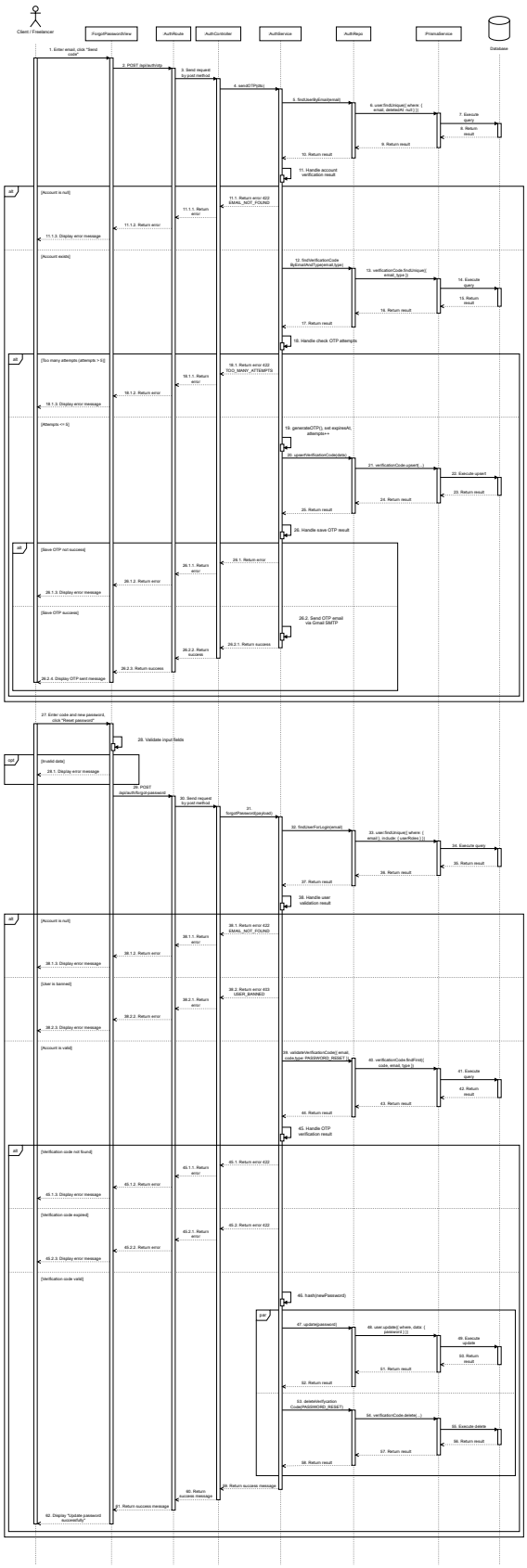


Figure 12: Sequence diagram of UC-05 Forgot Password

3.3 Role Management

Role Management lets an administrator inspect the role catalogue, maintain role metadata, and replace the permission set assigned to a role.

a. Class Diagram

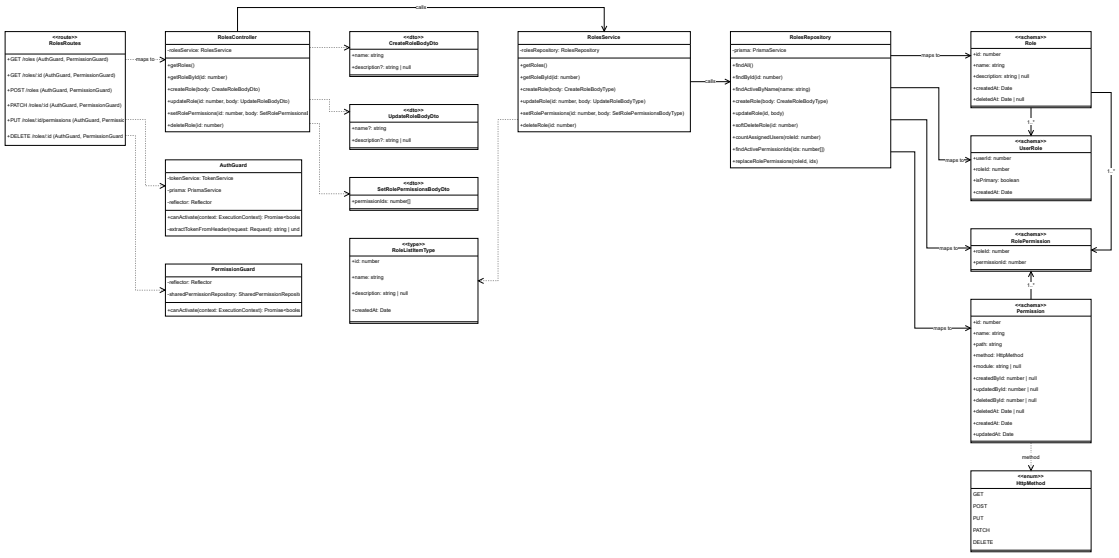


Figure 13: Class diagram of Role Management

b. Sequence Diagram(s)

3.3.1 UC-06.01 – View Role List

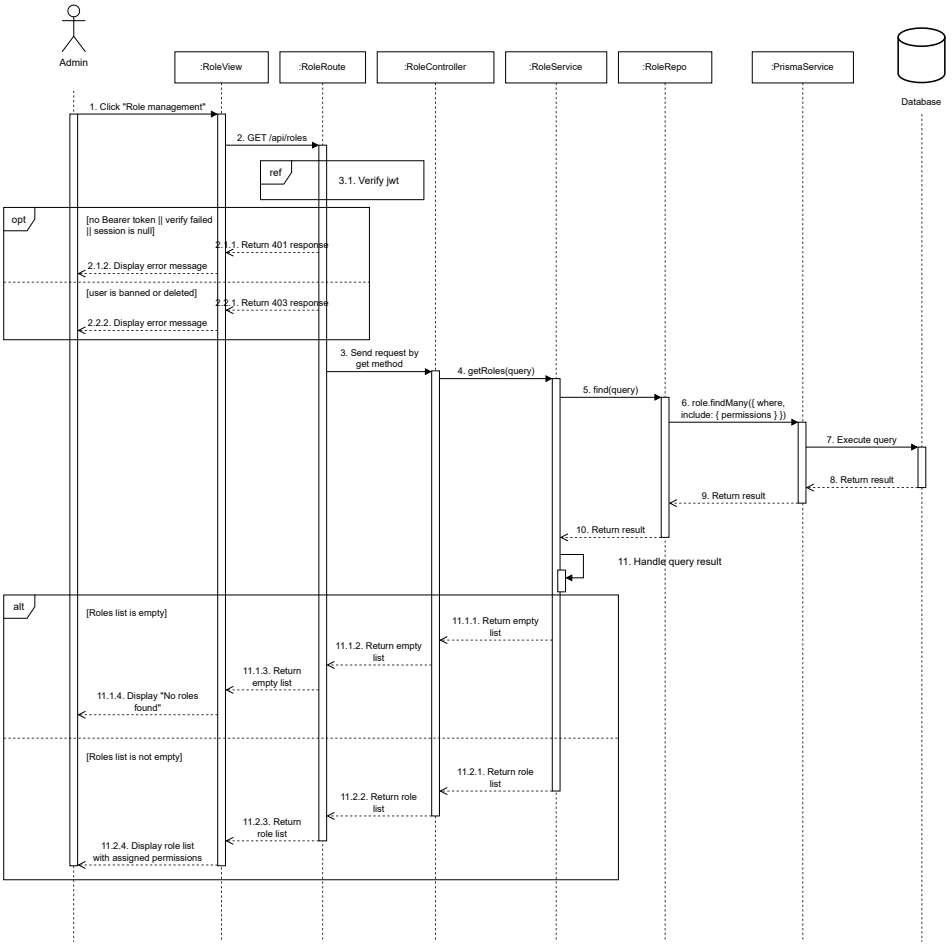


Figure 14: Sequence diagram of UC-06.01 View Role List

3.3.2 UC-06.02 – View Role Detail

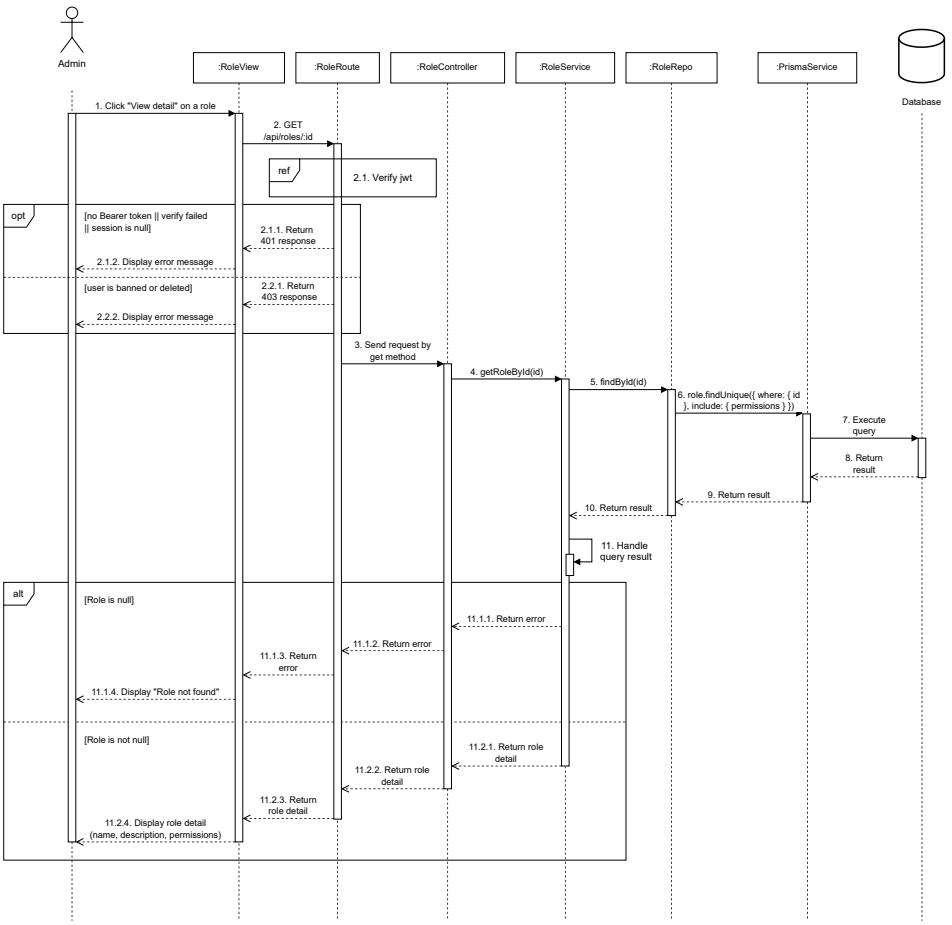


Figure 15: Sequence diagram of UC-06.02 View Role Detail

3.3.3 UC-06.03 – Create Role

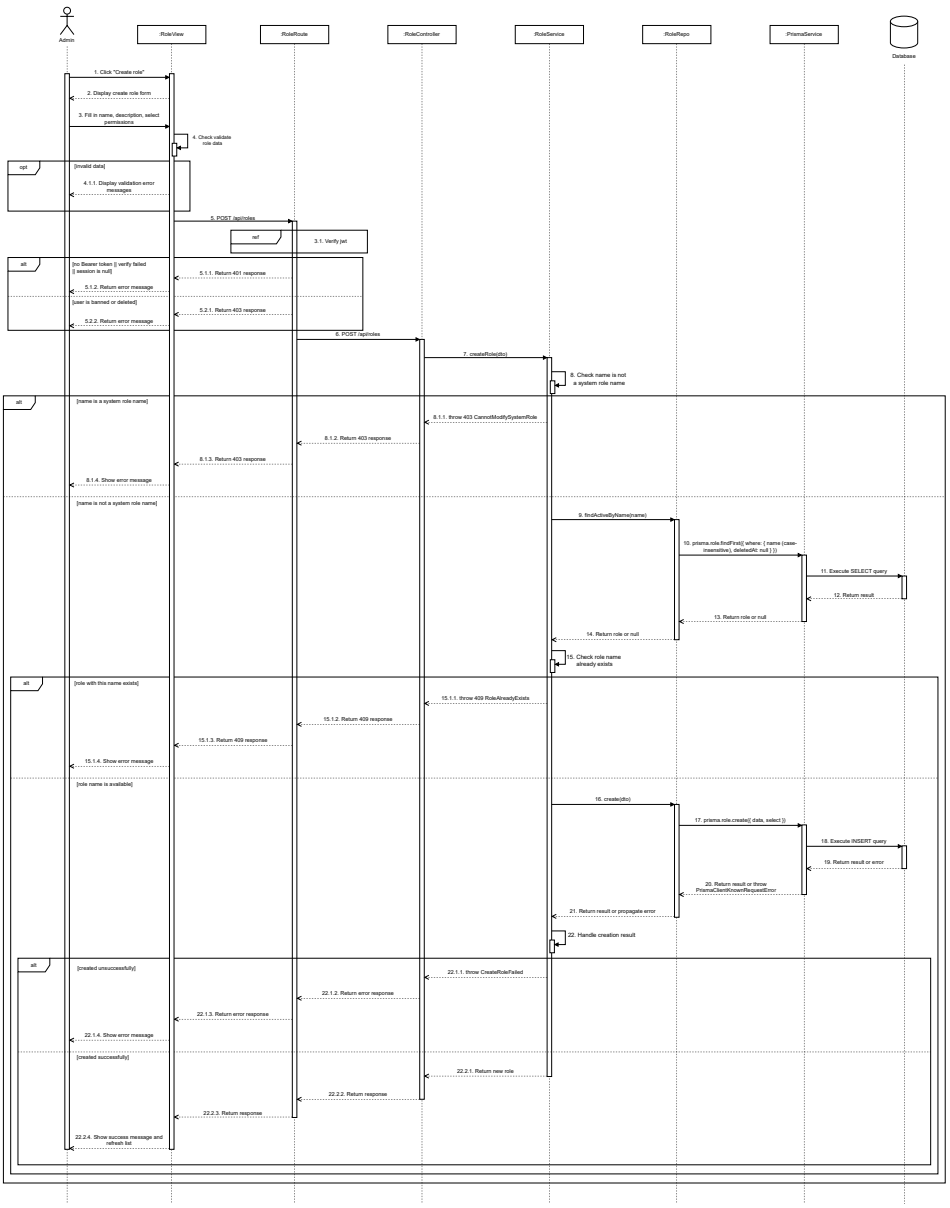


Figure 16: Sequence diagram of UC-06.03 Create Role

3.3.4 UC-06.04 – Update Role

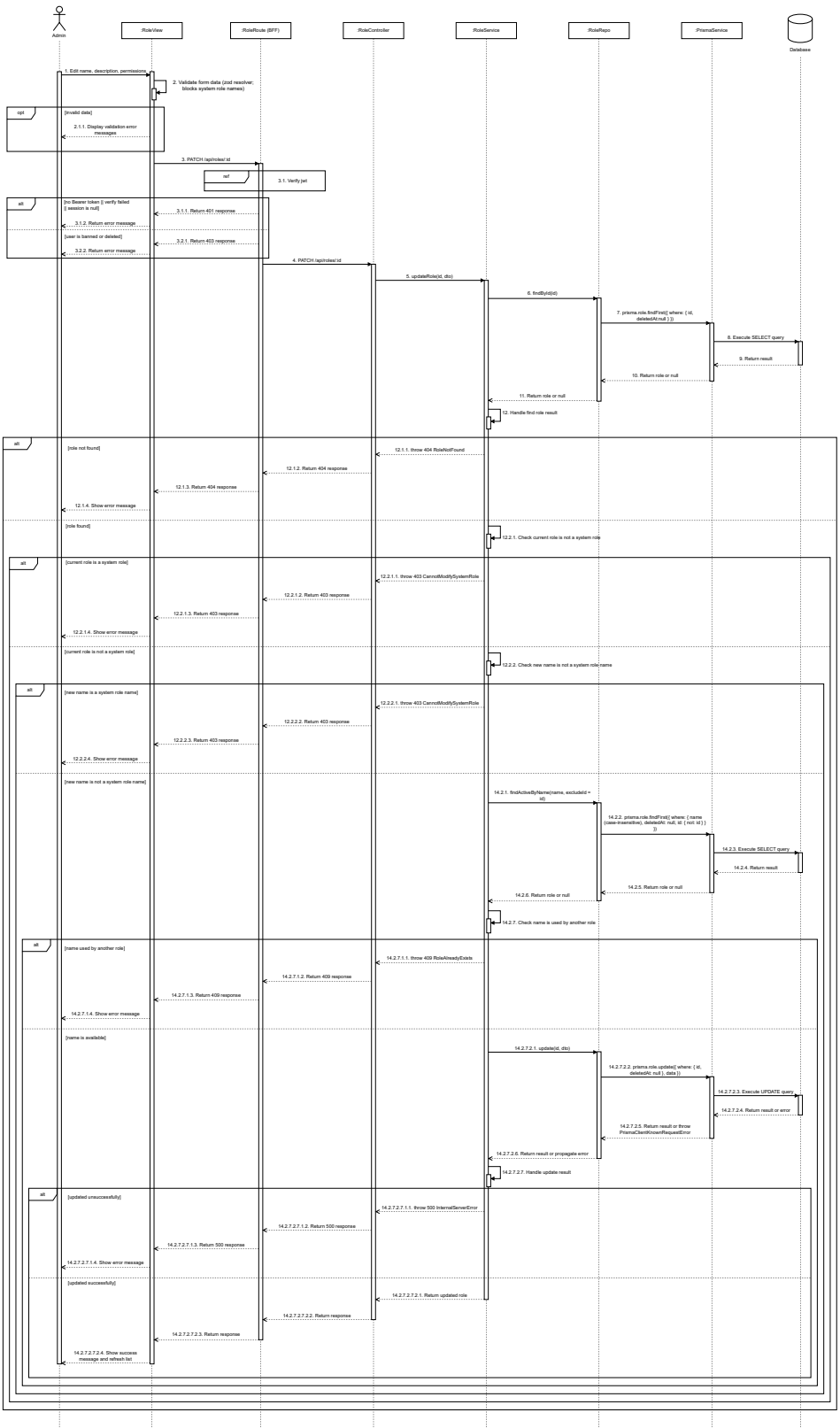


Figure 17: Sequence diagram of UC-06.04 Update Role

3.3.5 UC-06.05 – Delete Role

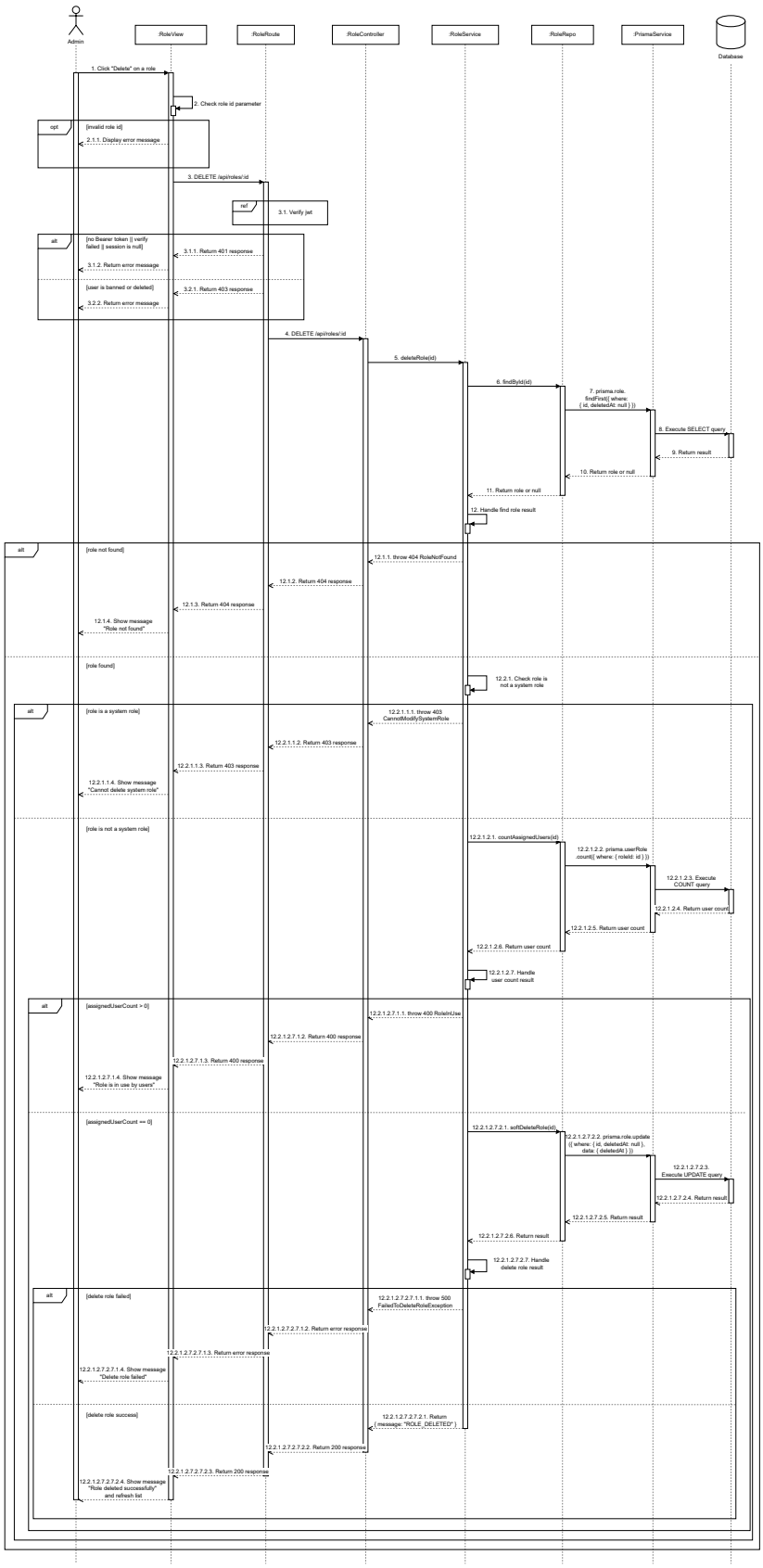


Figure 18: Sequence diagram of UC-06.05 Delete Role

3.3.6 UC-06.06 – Update Role Permissions

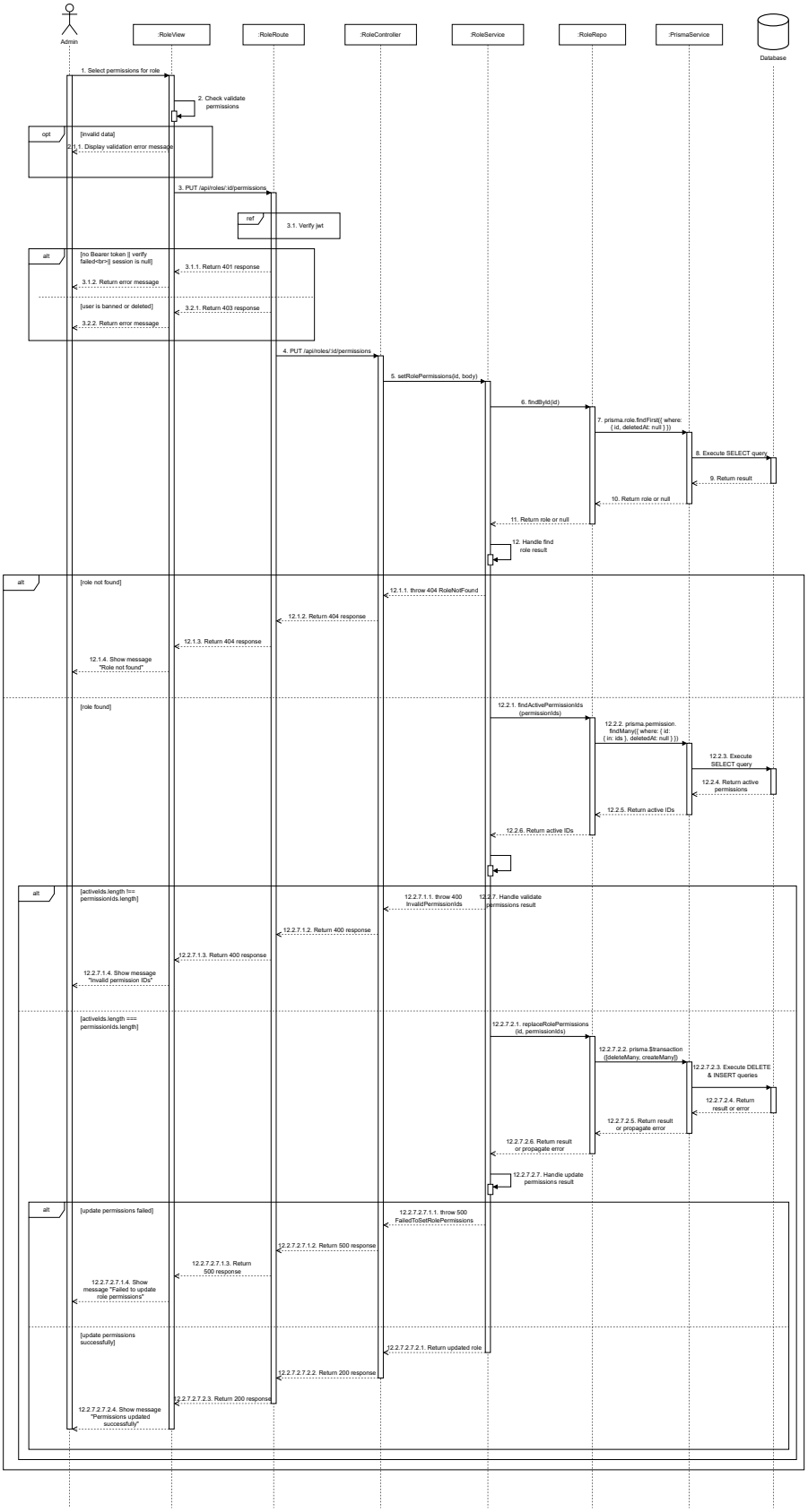


Figure 19: Sequence diagram of UC-06.06 Update Role Permissions

3.4 Permission Management

Permission Management exposes the route-level permission catalogue so that an administrator can inspect the permissions available for role assignment.

a. Class Diagram

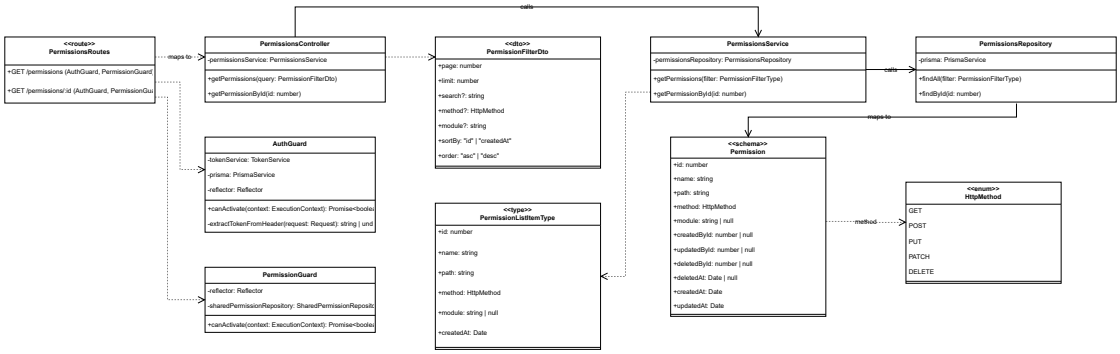


Figure 20: Class diagram of Permission Management

b. Sequence Diagram(s)

3.4.1 UC-15.01 – View Permission List

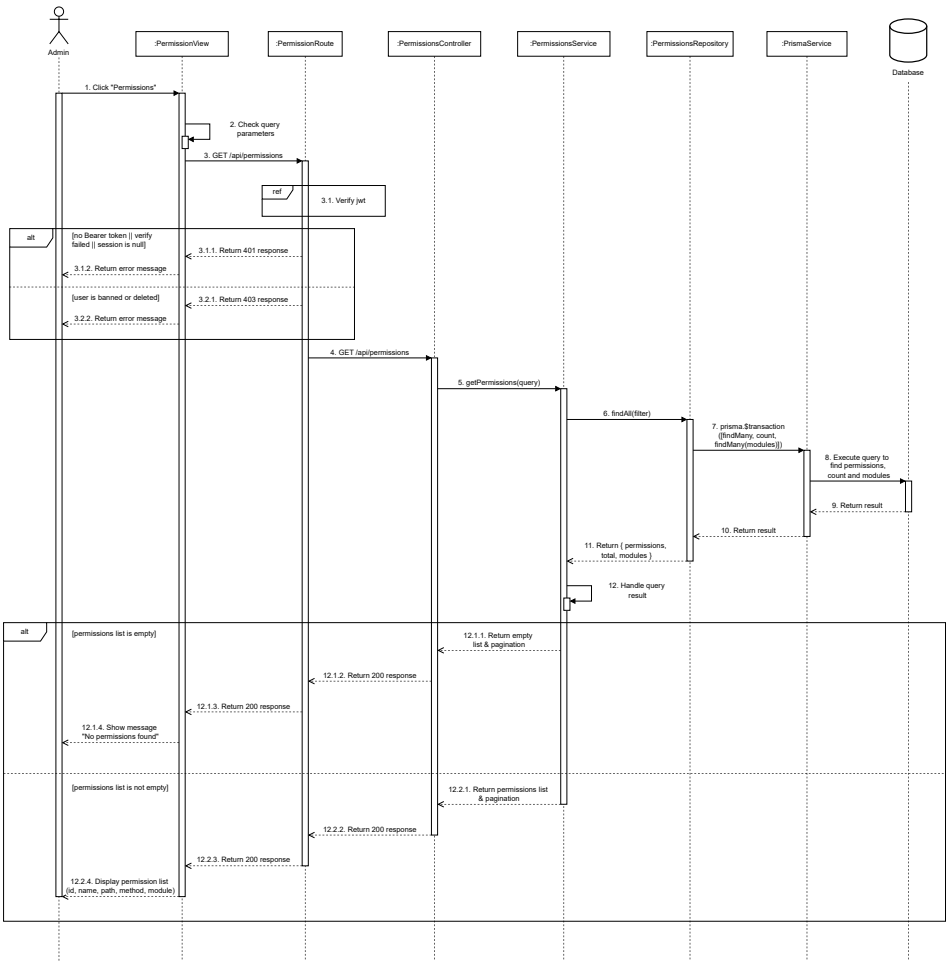


Figure 21: Sequence diagram of UC-15.01 View Permission List

3.4.2 UC-15.02 – View Permission Detail

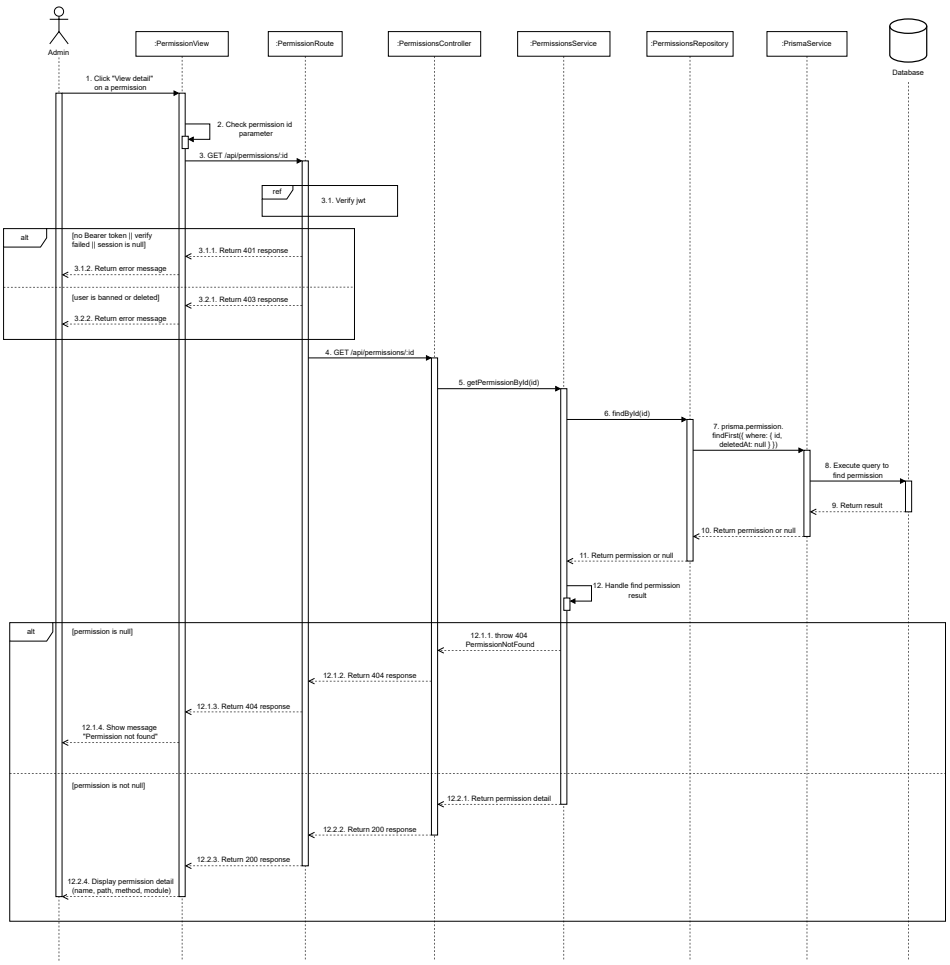


Figure 22: Sequence diagram of UC-15.02 View Permission Detail

3.5 Session Management

Session Management gives an authenticated user visibility over the devices that currently hold a refresh-token session and allows a session to be revoked remotely.

a. Class Diagram

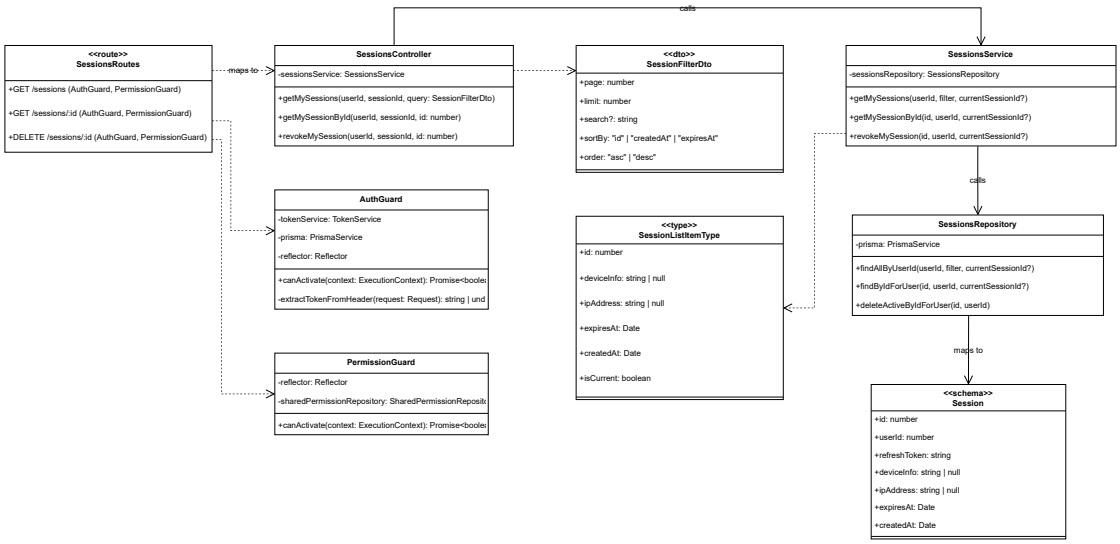


Figure 23: Class diagram of Session Management

b. Sequence Diagram(s)

3.5.1 UC-16.01 – View Session List

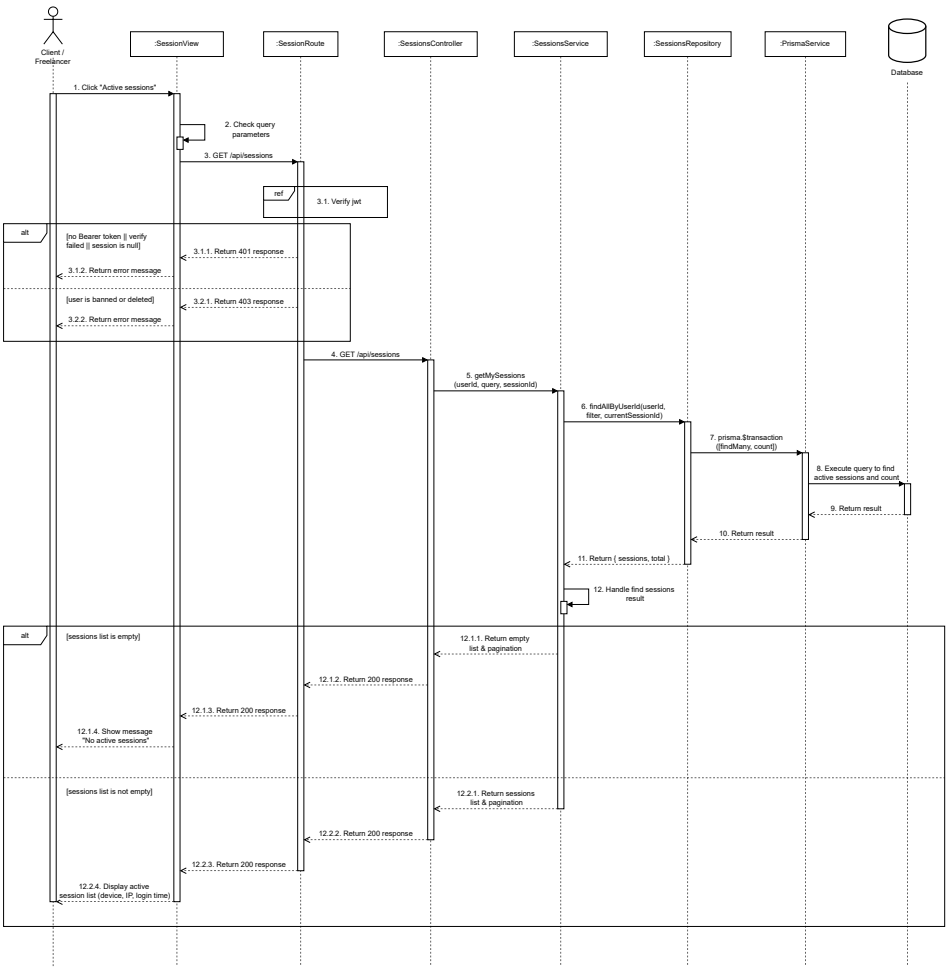


Figure 24: Sequence diagram of UC-16.01 View Session List

3.5.2 UC-16.02 – View Session Detail

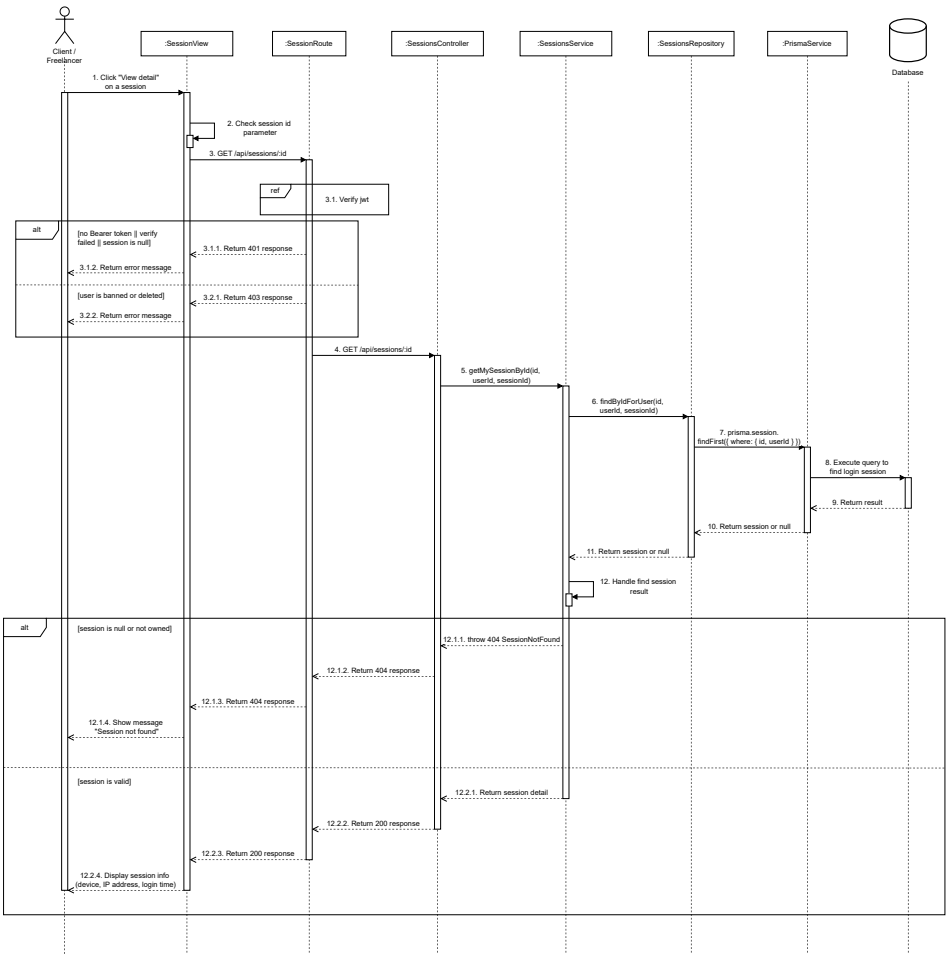


Figure 25: Sequence diagram of UC-16.02 View Session Detail

3.5.3 UC-16.03 – Revoke Session

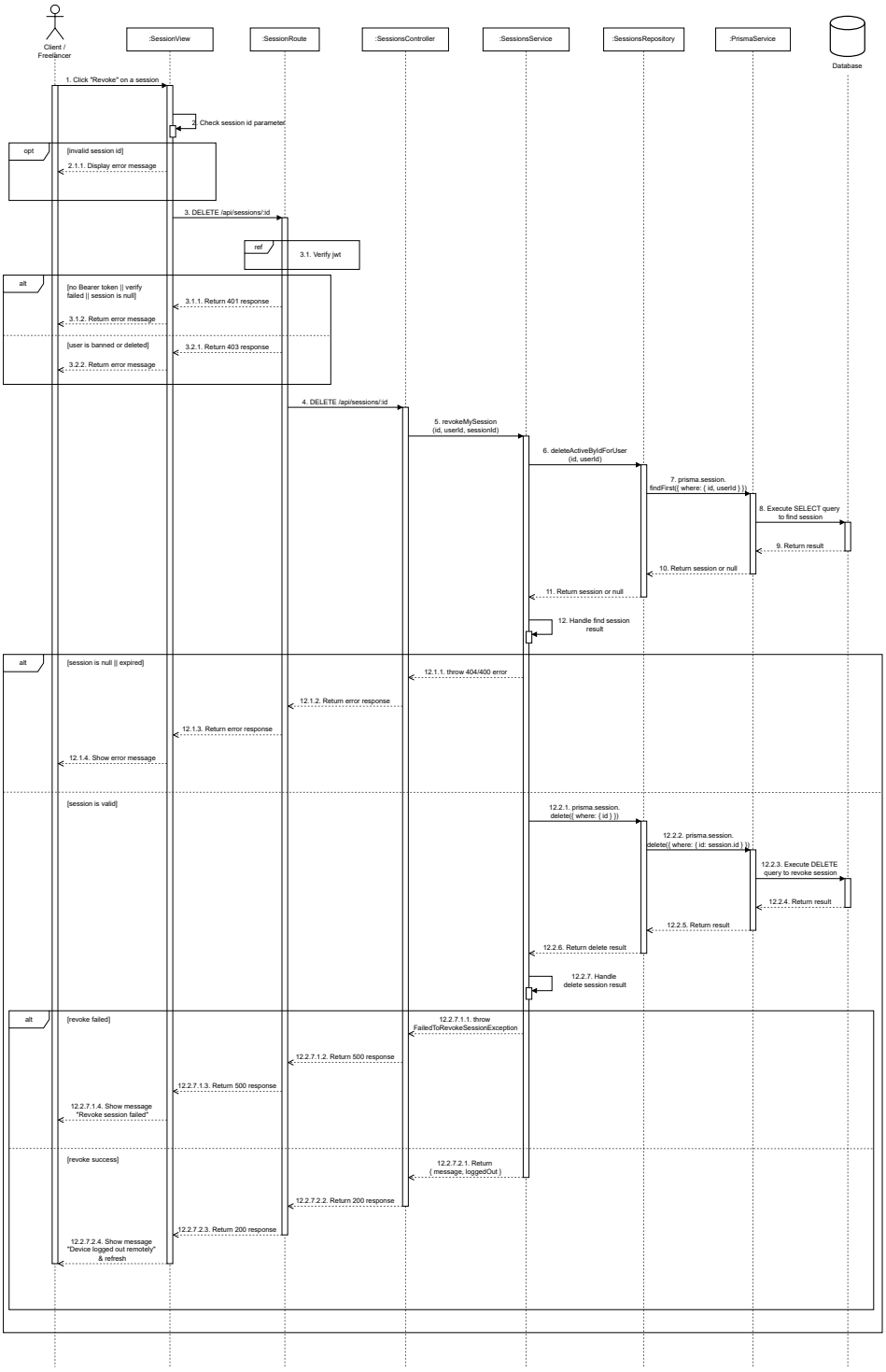


Figure 26: Sequence diagram of UC-16.03 Revoke Session

3.6 Forum Category Management

Forum Category Management covers the administrator-side category catalogue and the public category views used by the forum. Deletion is a soft delete, so a category can be restored

later.

a. Class Diagram

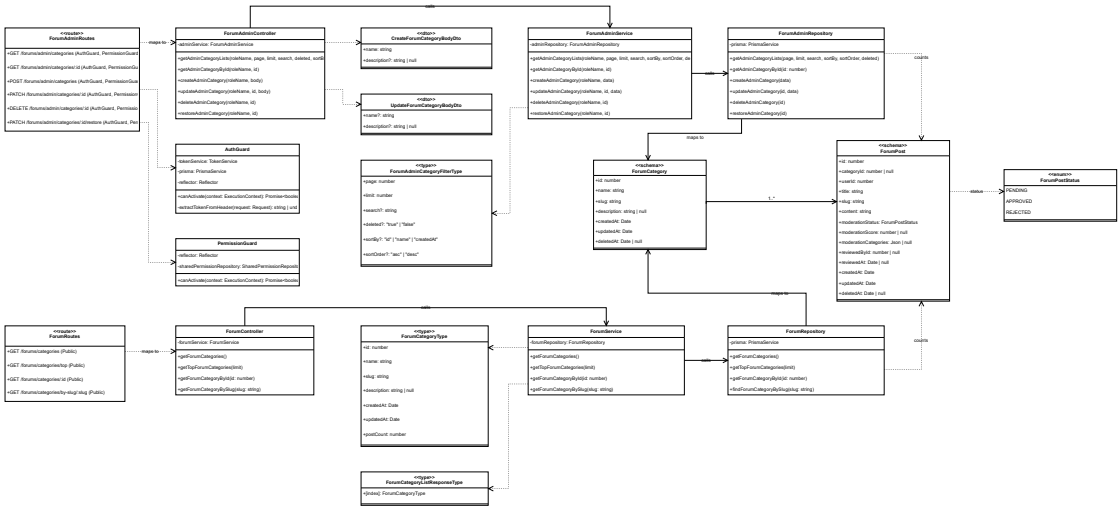


Figure 27: Class diagram of Forum Category Management

b. Sequence Diagram(s)

3.6.1 UC-17.01 – Create Forum Category

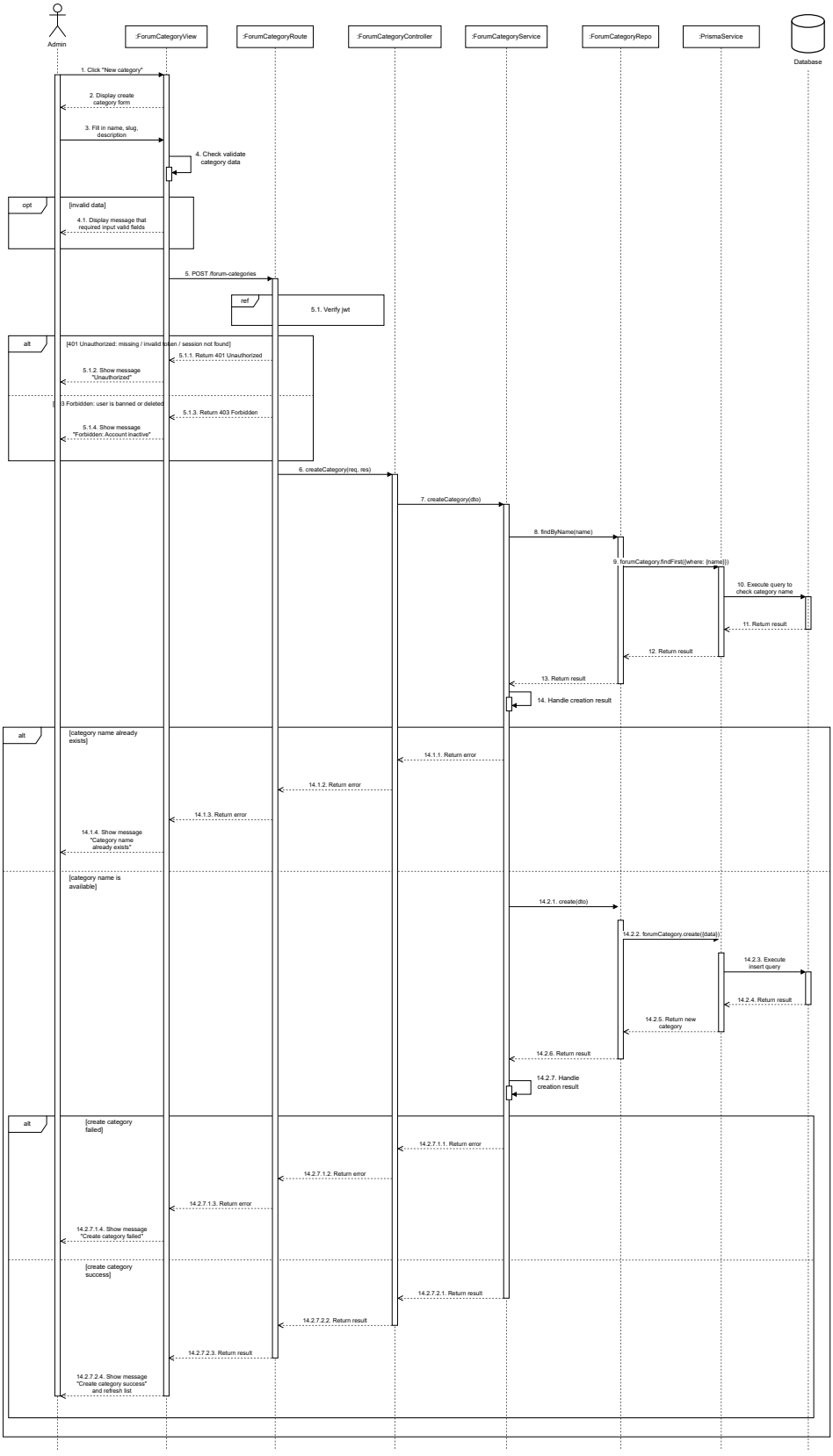


Figure 28: Sequence diagram of UC-17.01 Create Forum Category

3.6.2 UC-17.02 – Update Forum Category

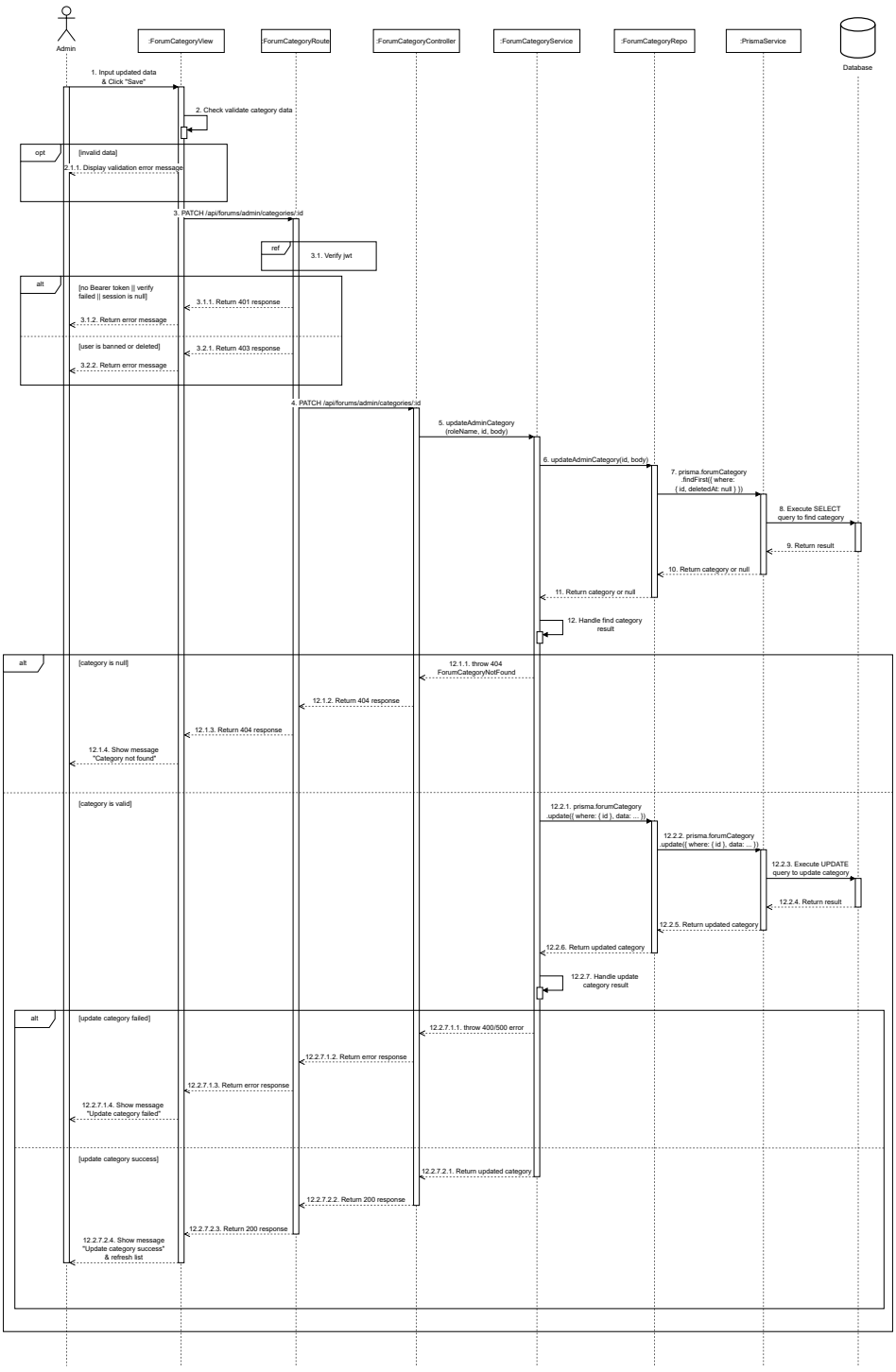


Figure 29: Sequence diagram of UC-17.02 Update Forum Category

3.6.3 UC-17.03 – Delete Forum Category

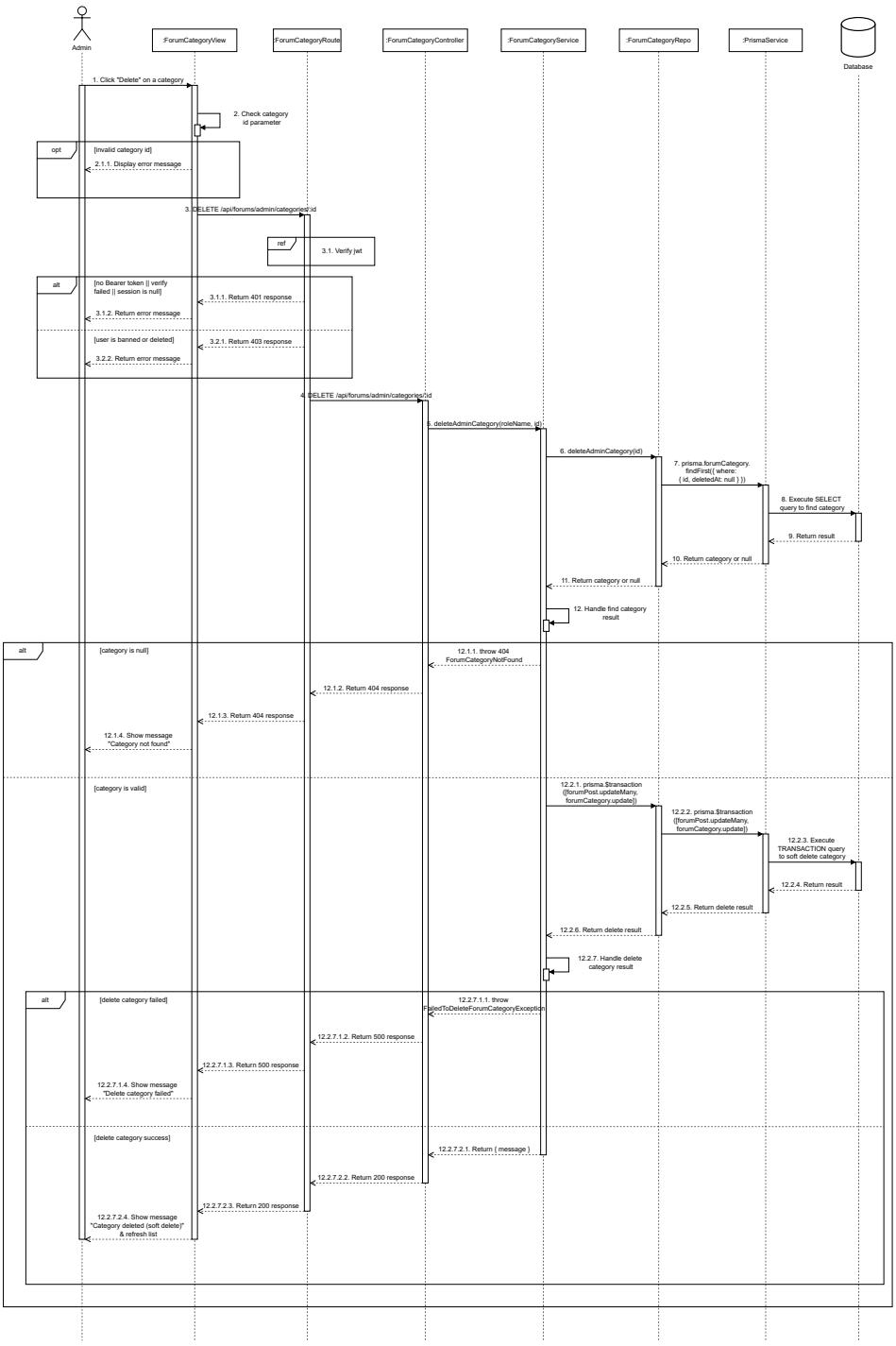


Figure 30: Sequence diagram of UC-17.03 Delete Forum Category

3.6.4 UC-17.06 – Restore Forum Category

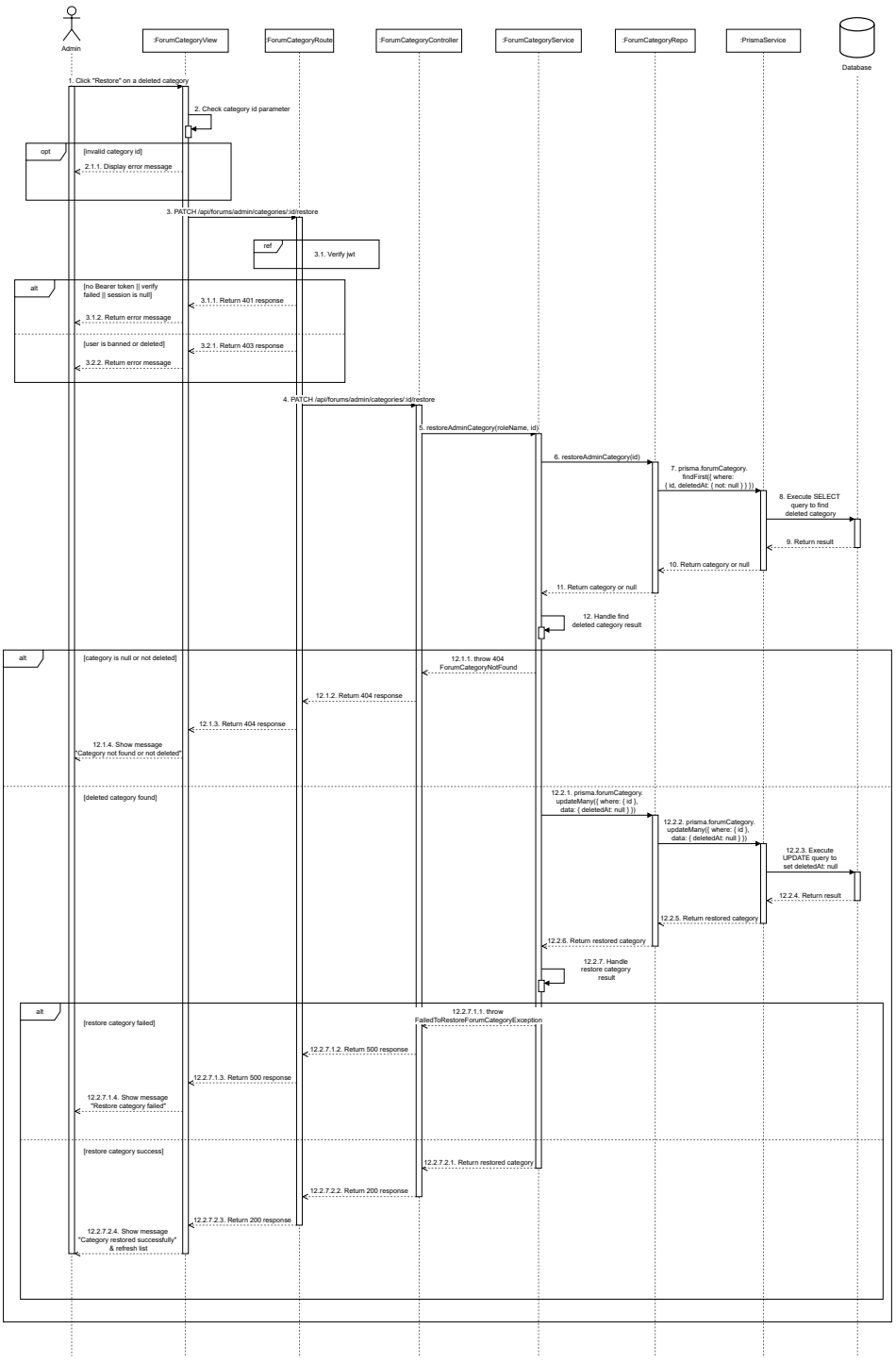


Figure 31: Sequence diagram of UC-17.06 Restore Forum Category

3.6.5 UC-14.01 – View Forum Category List

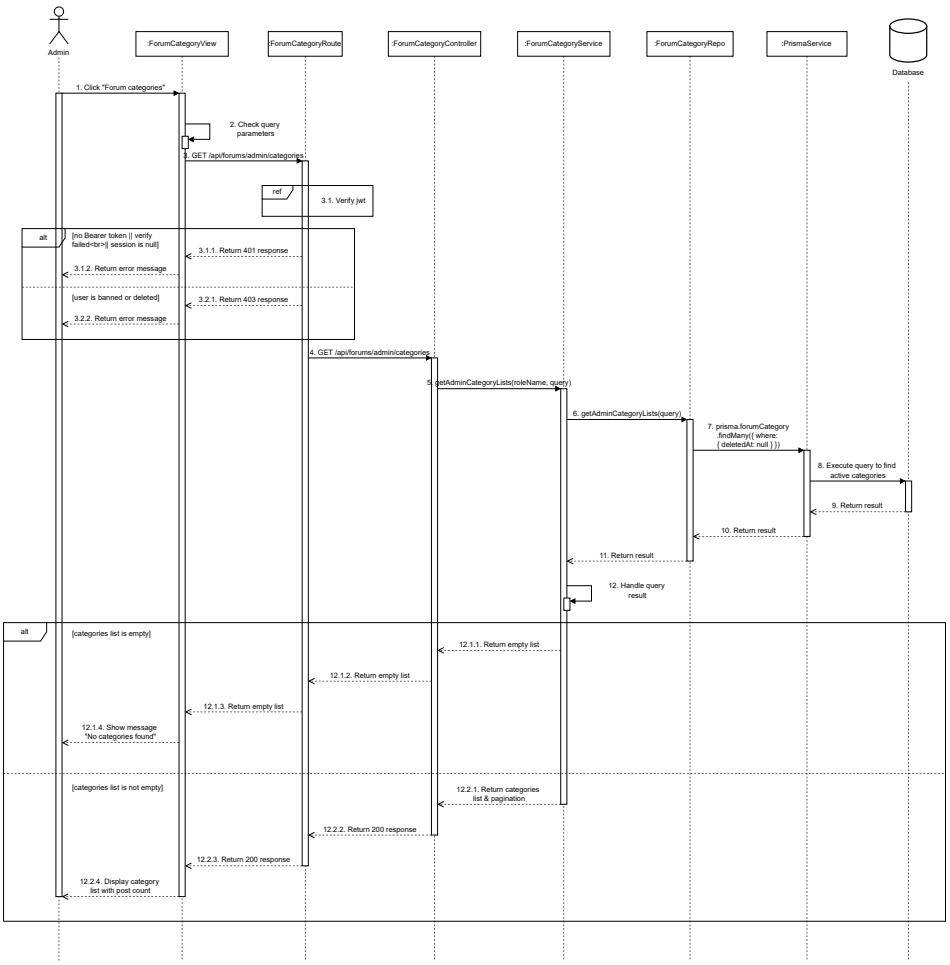


Figure 32: Sequence diagram of UC-14.01 View Forum Category List

3.6.6 UC-14.02 – View Forum Category Detail

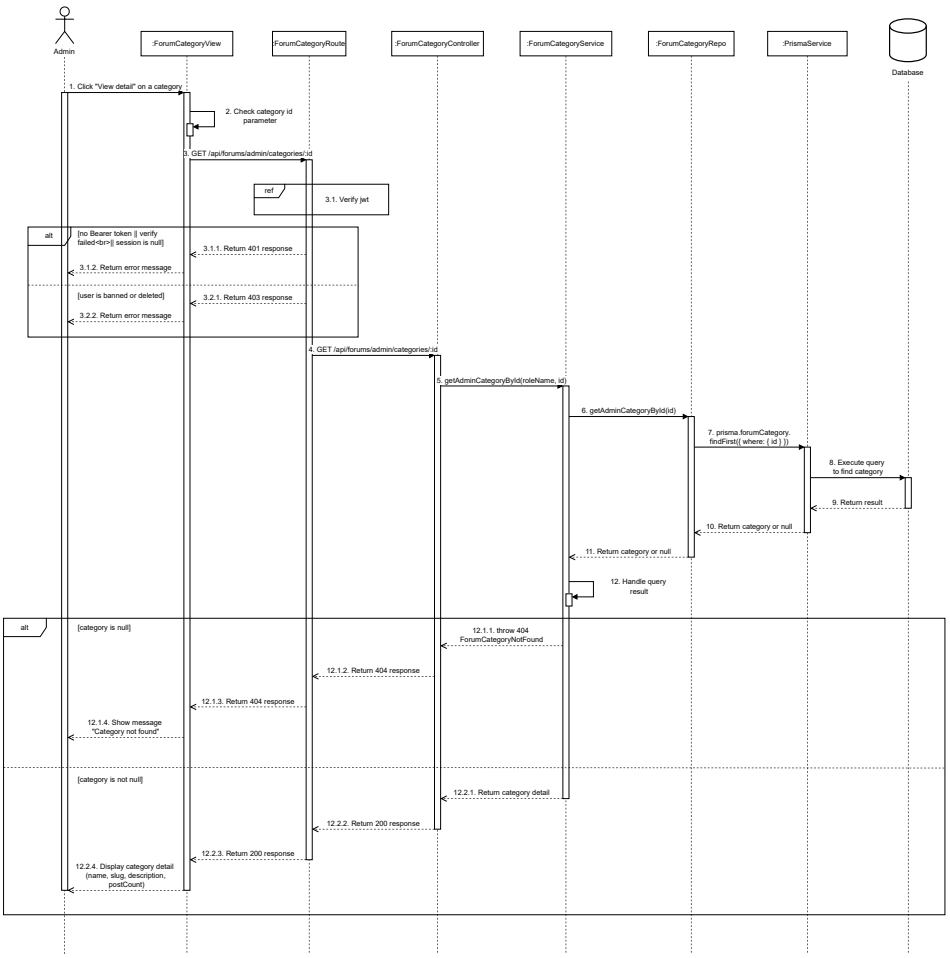


Figure 33: Sequence diagram of UC-14.02 View Forum Category Detail

3.7 User Management

User Management provides the administrator with a searchable user catalogue, a consolidated account detail view, and dedicated editors for the Client and Freelancer profiles attached to an account.

a. Class Diagram

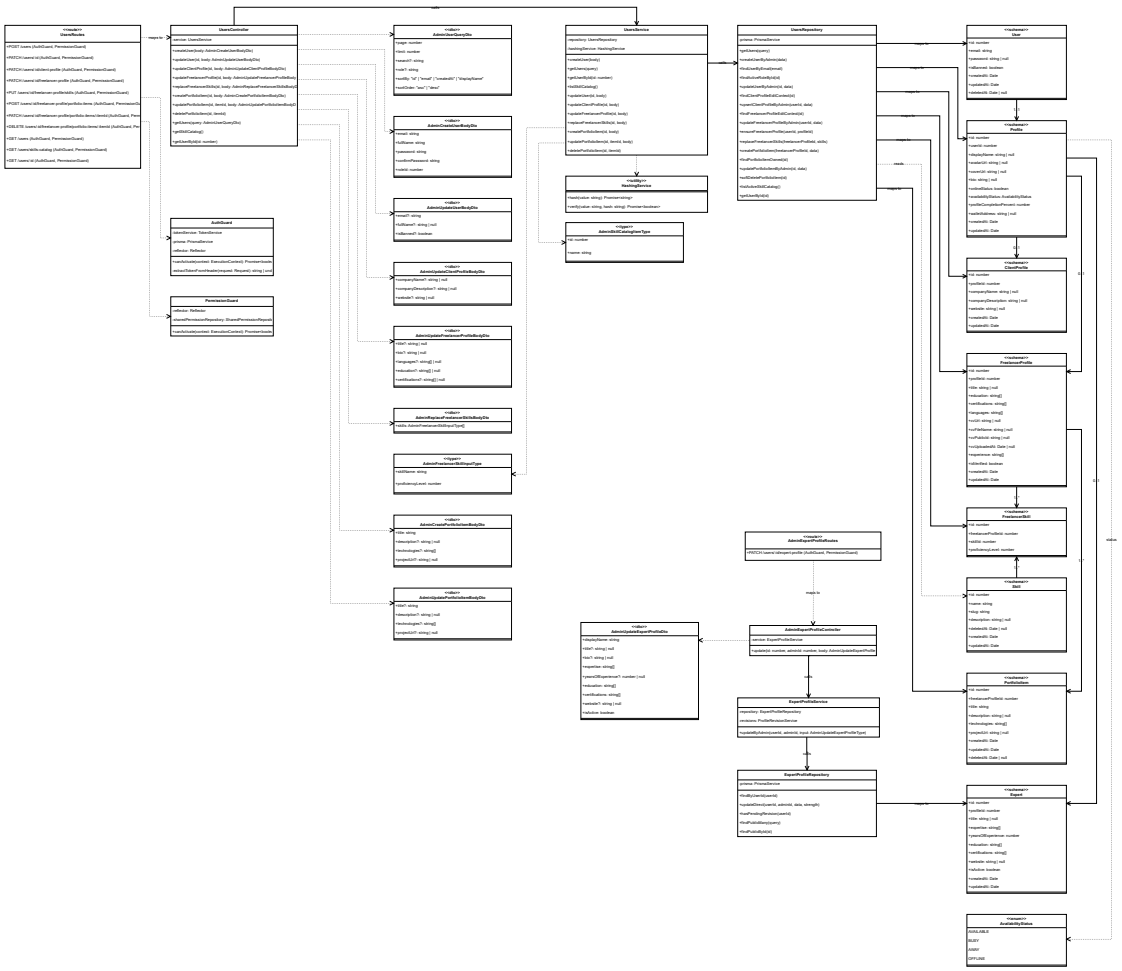


Figure 34: Class diagram of User Management

b. Sequence Diagram(s)

3.7.1 UC-28.01 – View User List

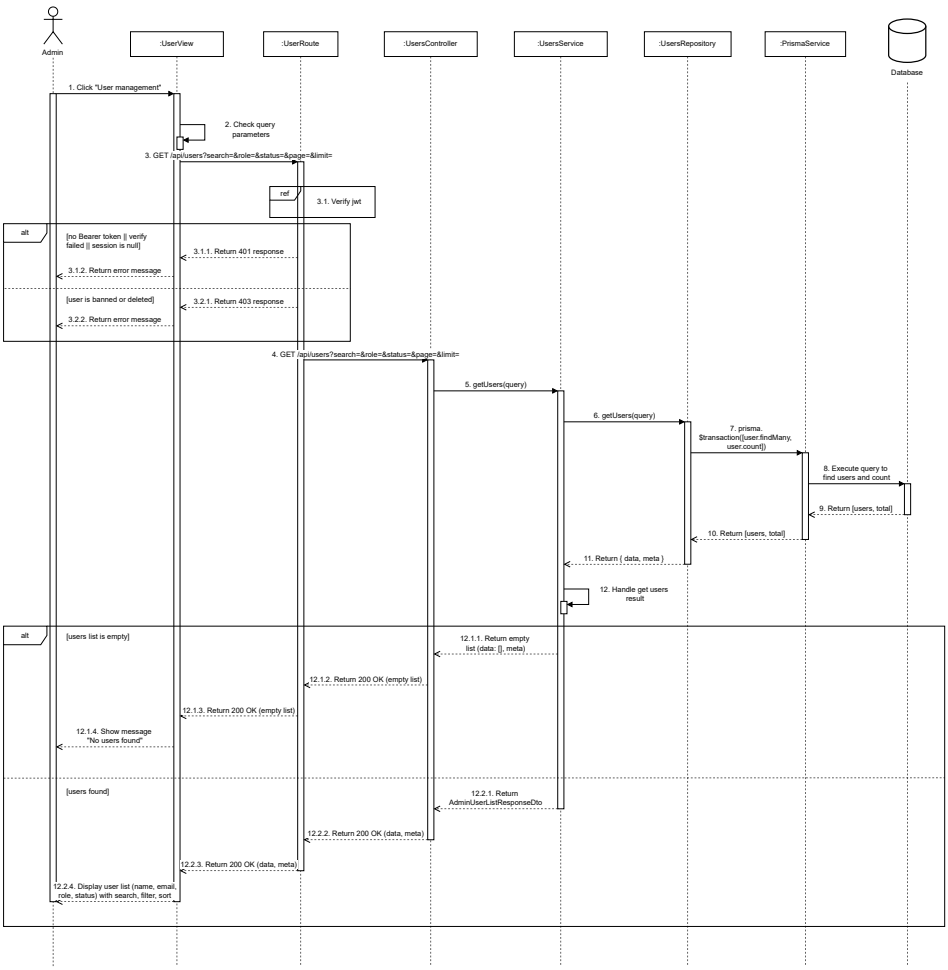


Figure 35: Sequence diagram of UC-28.01 View User List

3.7.2 UC-28.02 – View User Detail

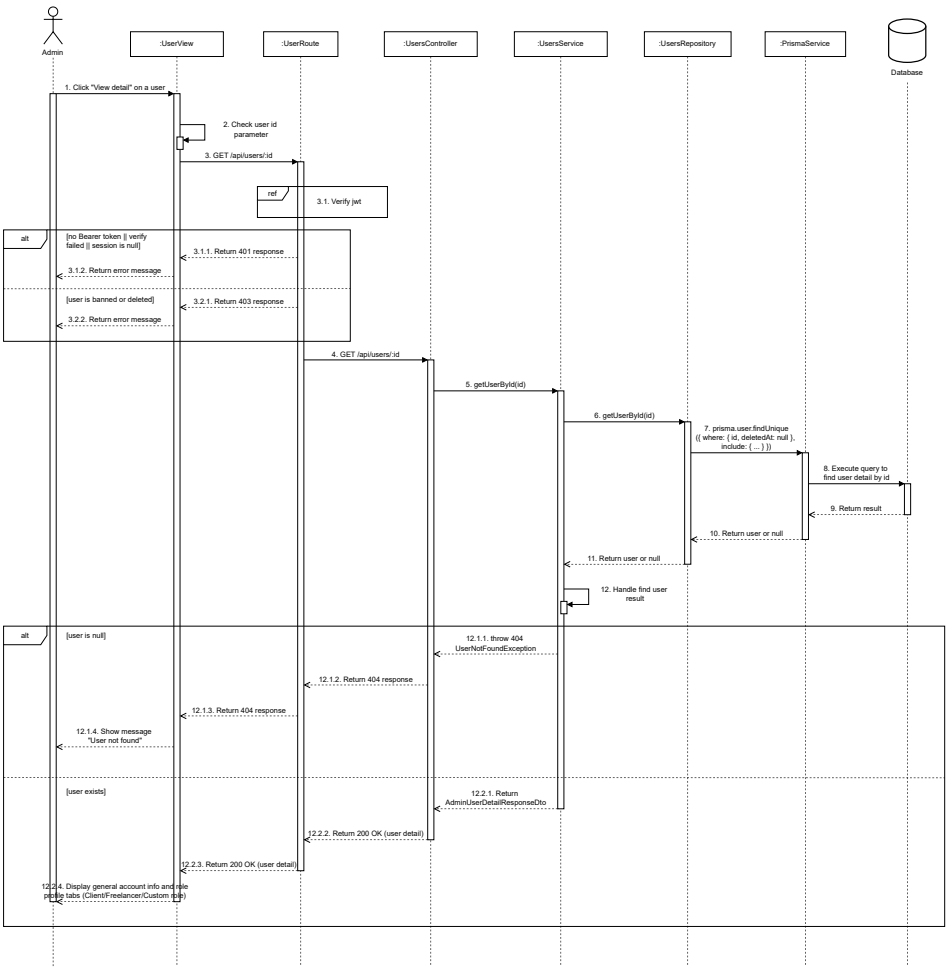


Figure 36: Sequence diagram of UC-28.02 View User Detail

3.7.3 UC-28.03 – Create User

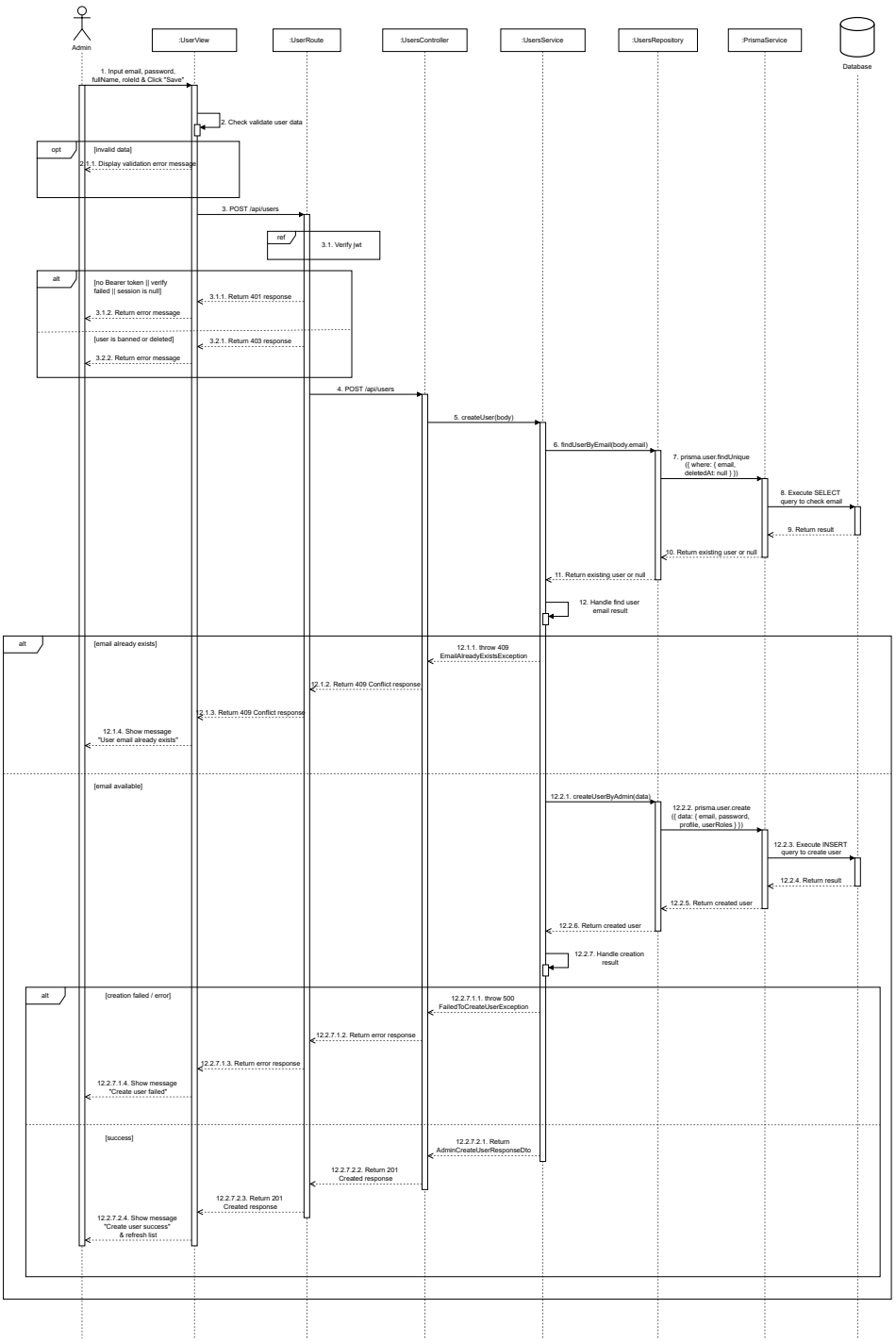


Figure 37: Sequence diagram of UC-28.03 Create User

3.7.4 UC-28.04 – Update User

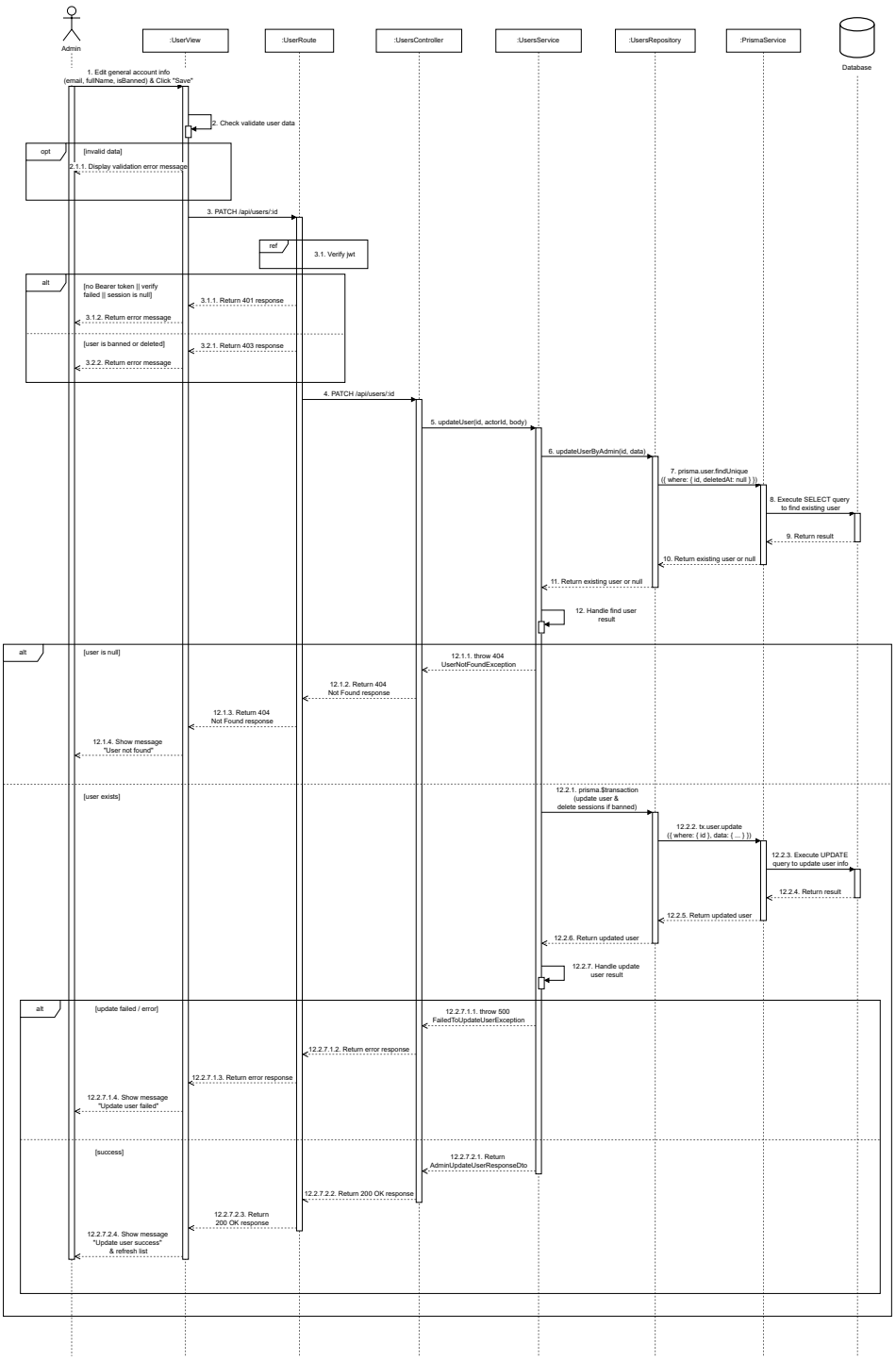


Figure 38: Sequence diagram of UC-28.04 Update User

3.7.5 UC-28.05 – Update Client Profile

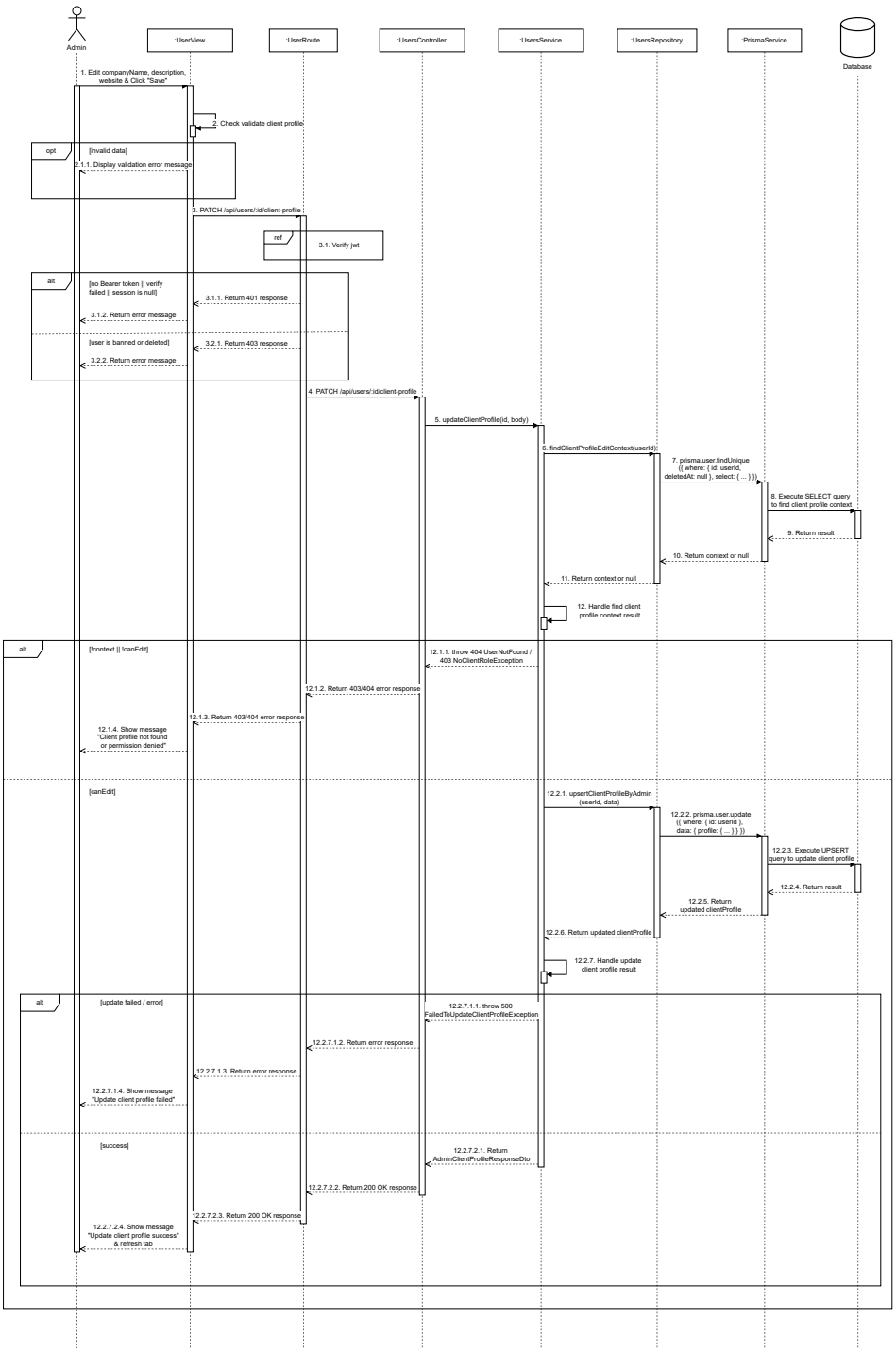


Figure 39: Sequence diagram of UC-28.05 Update Client Profile

3.7.6 UC-28.06 – Update Freelancer Basic Info

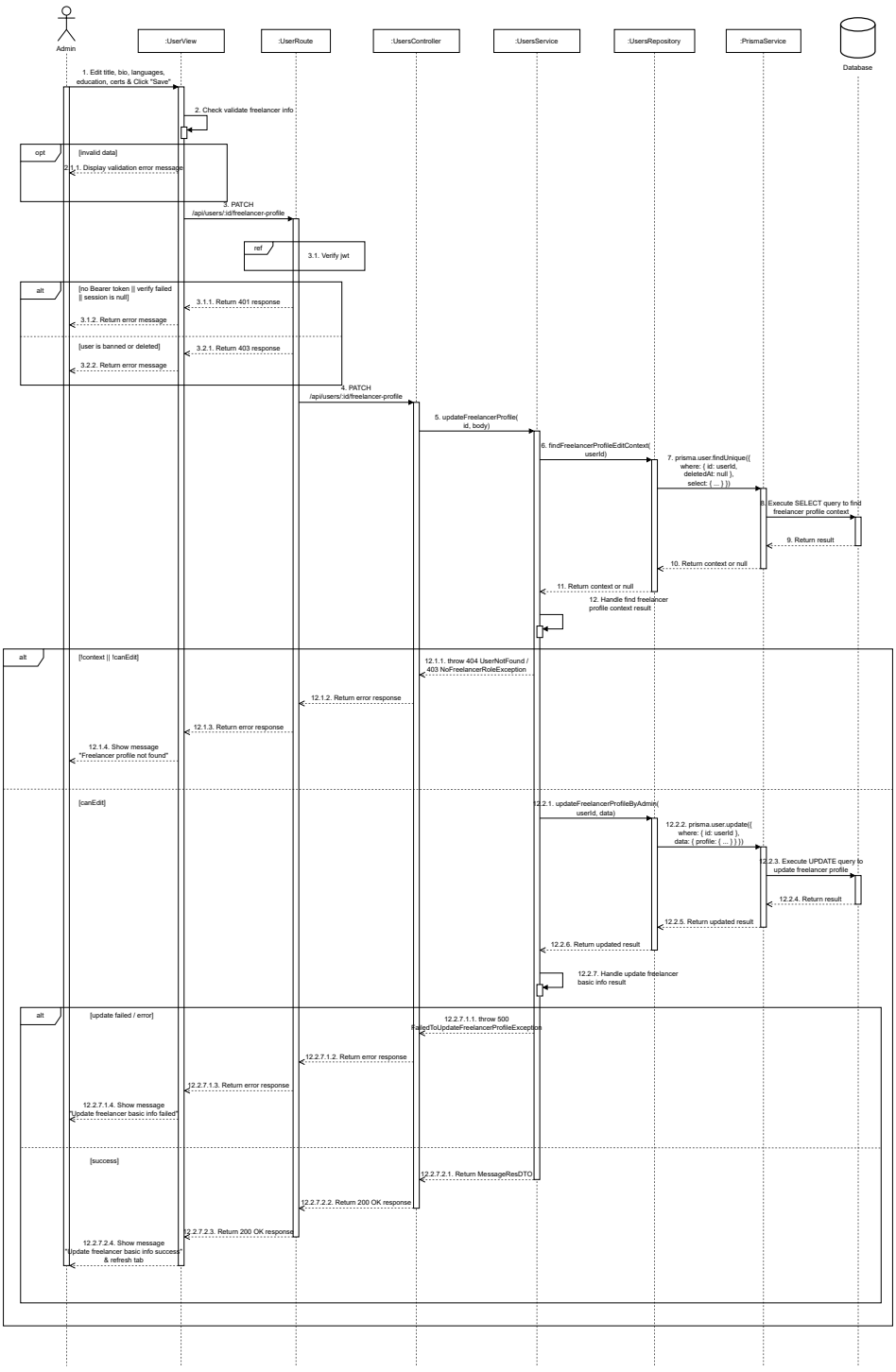


Figure 40: Sequence diagram of UC-28.06 Update Freelancer Basic Info

3.7.7 UC-28.07 – Update Freelancer Skills

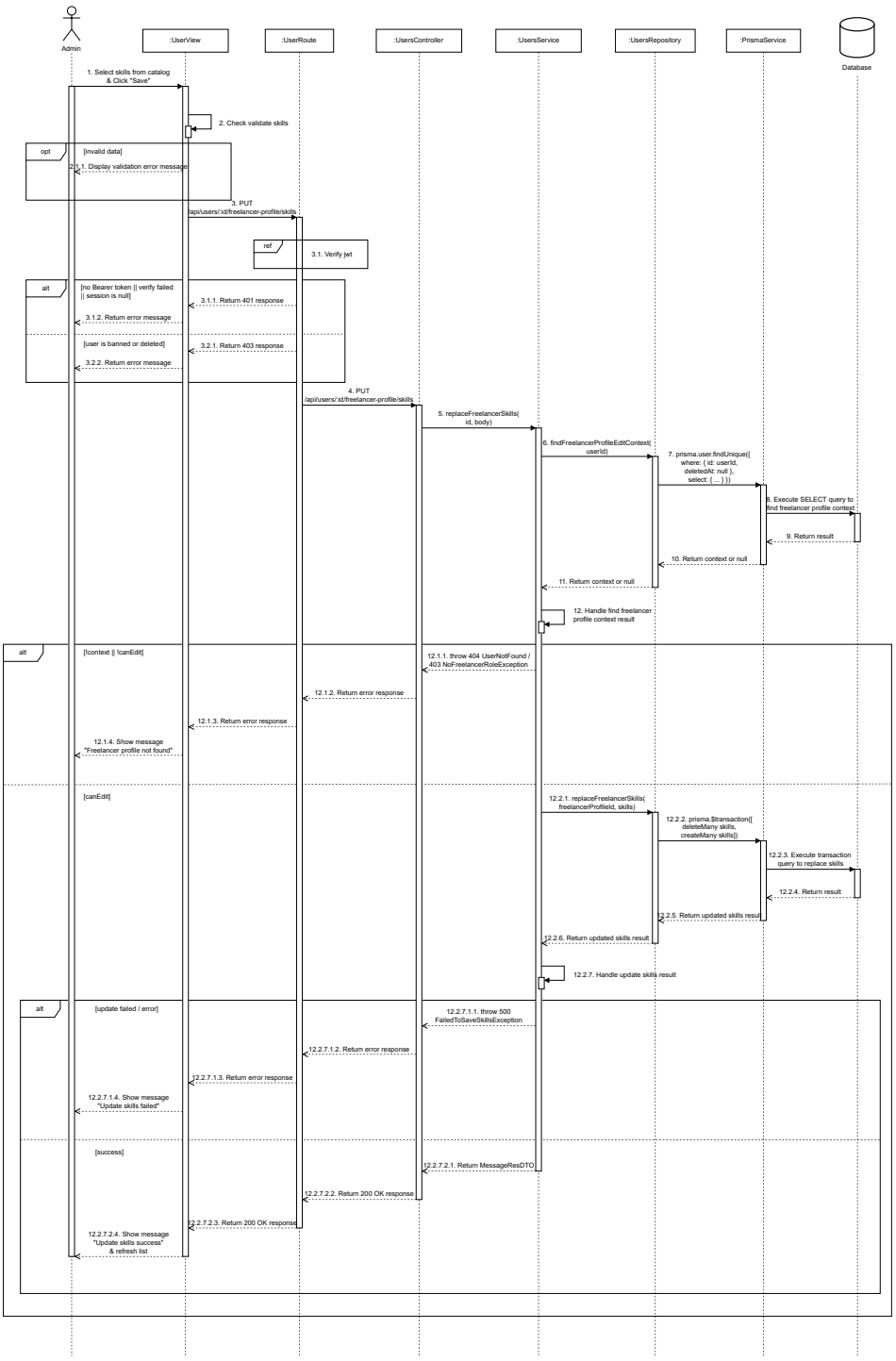


Figure 41: Sequence diagram of UC-28.07 Update Freelancer Skills

3.7.8 UC-28.08 – Update Freelancer Portfolio

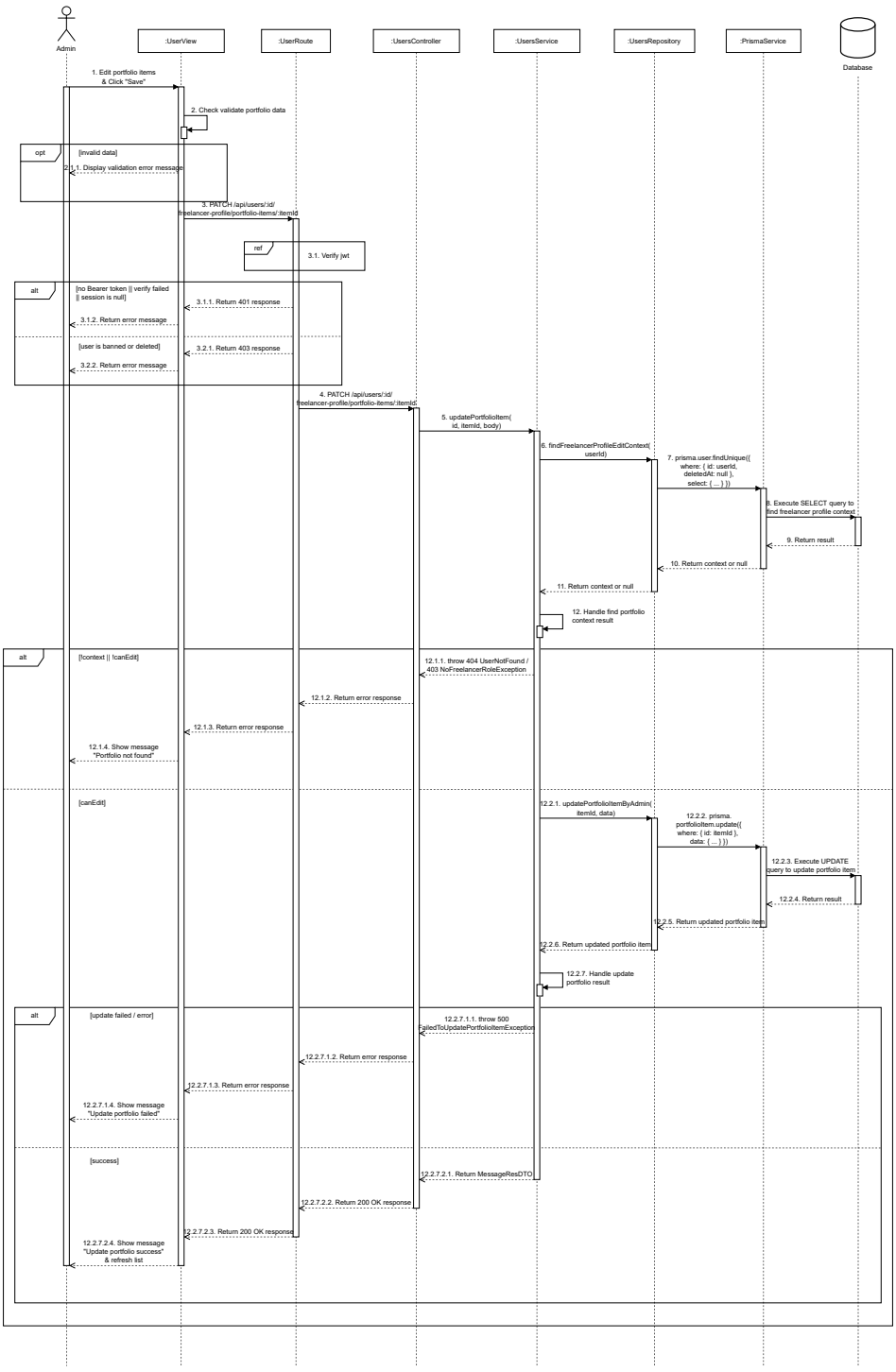


Figure 42: Sequence diagram of UC-28.08 Update Freelancer Portfolio

3.8 Banner Management

Banner Management maintains the advertisement banners shown in configured page positions. The module supports image upload, lifecycle dates, activation state, and hard deletion.

a. Class Diagram

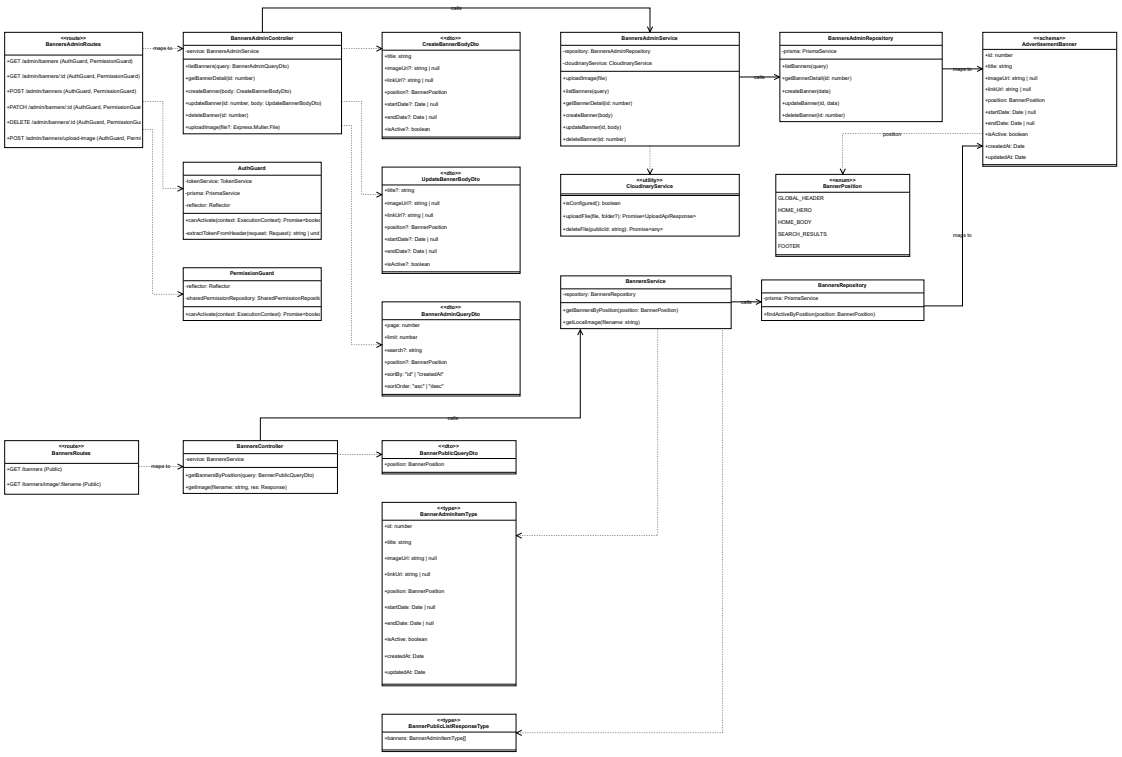


Figure 43: Class diagram of Banner Management

b. Sequence Diagram(s)

3.8.1 UC-37.01 – View Banner List

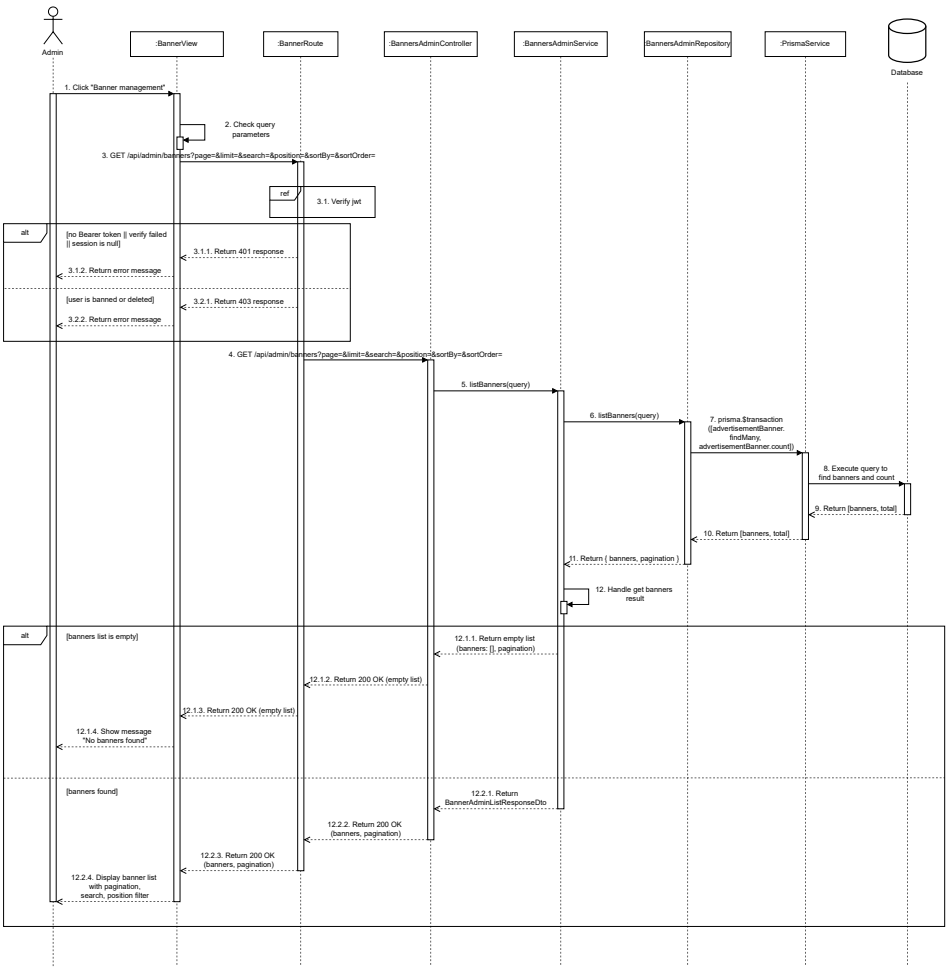


Figure 44: Sequence diagram of UC-37.01 View Banner List

3.8.2 UC-37.02 – View Banner Detail

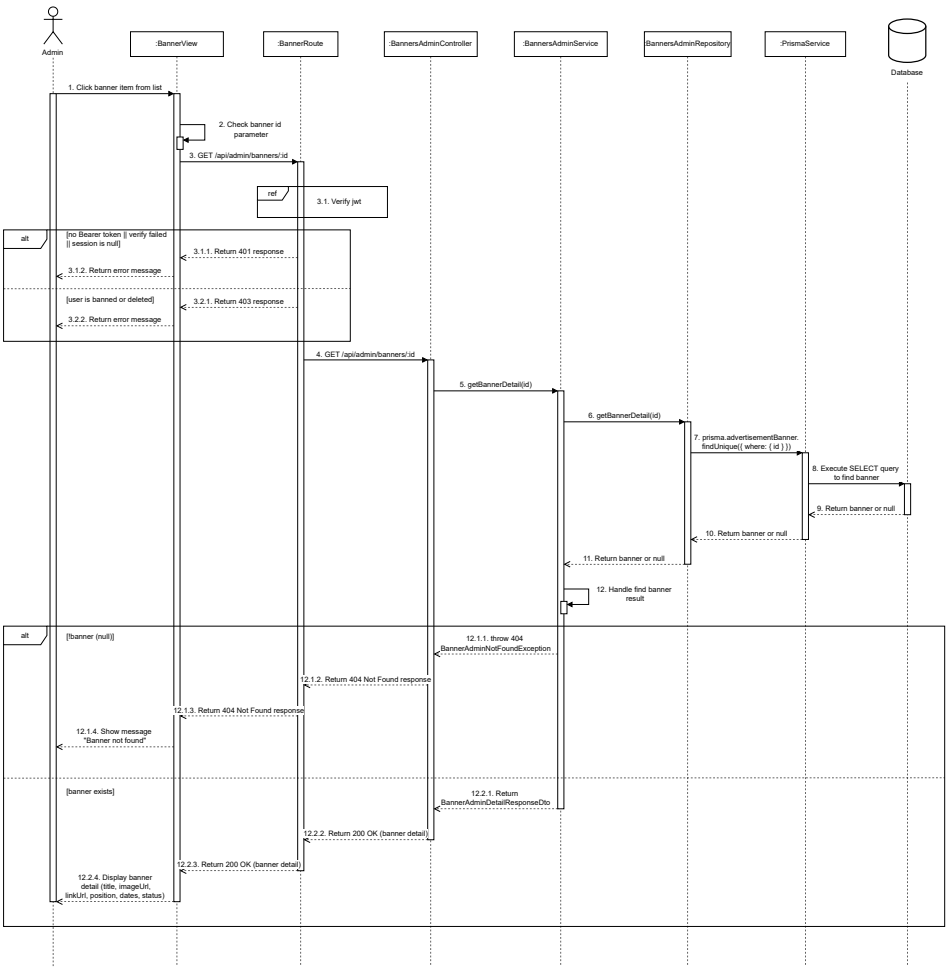


Figure 45: Sequence diagram of UC-37.02 View Banner Detail

3.8.3 UC-37.03 – Create Banner

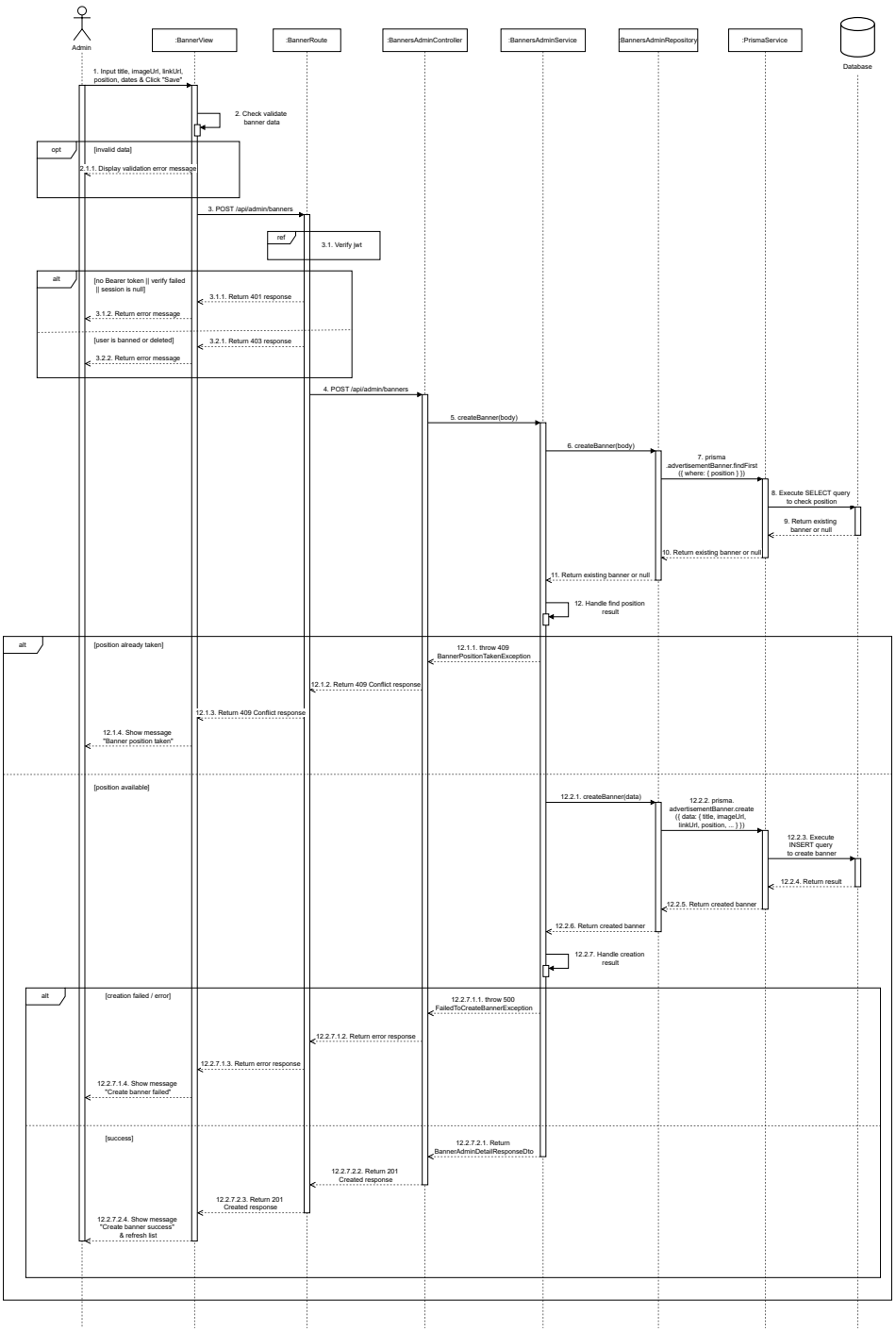


Figure 46: Sequence diagram of UC-37.03 Create Banner

3.8.4 UC-37.04 – Update Banner

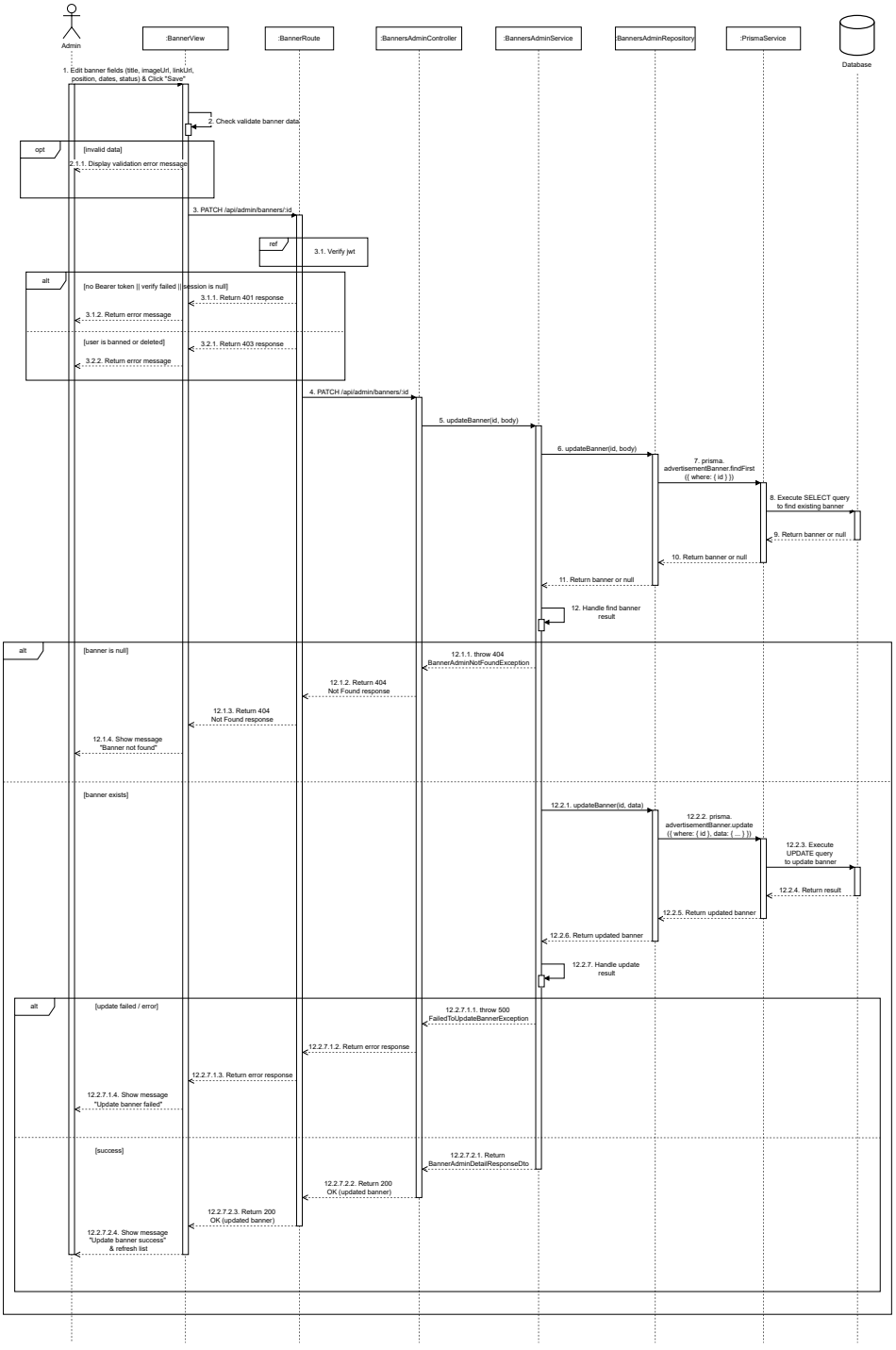


Figure 47: Sequence diagram of UC-37.04 Update Banner

3.8.5 UC-37.05 – Delete Banner

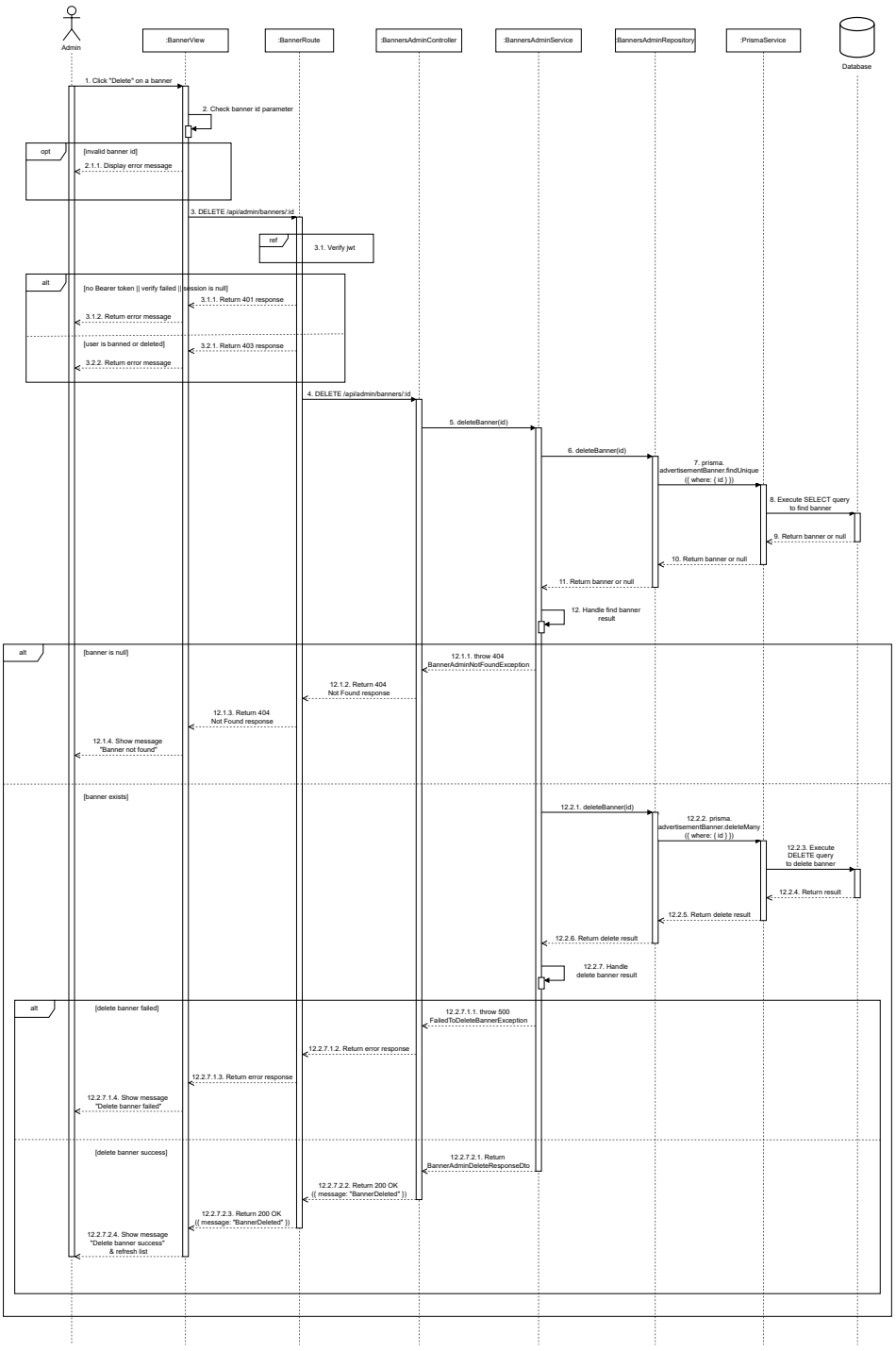


Figure 48: Sequence diagram of UC-37.05 Delete Banner

3.9 Skill Management

Skill Management owns the master skill catalogue used by freelancer profiles and job postings. Skills are soft-deleted so that existing references remain valid, and a deleted skill can be restored.

a. Class Diagram

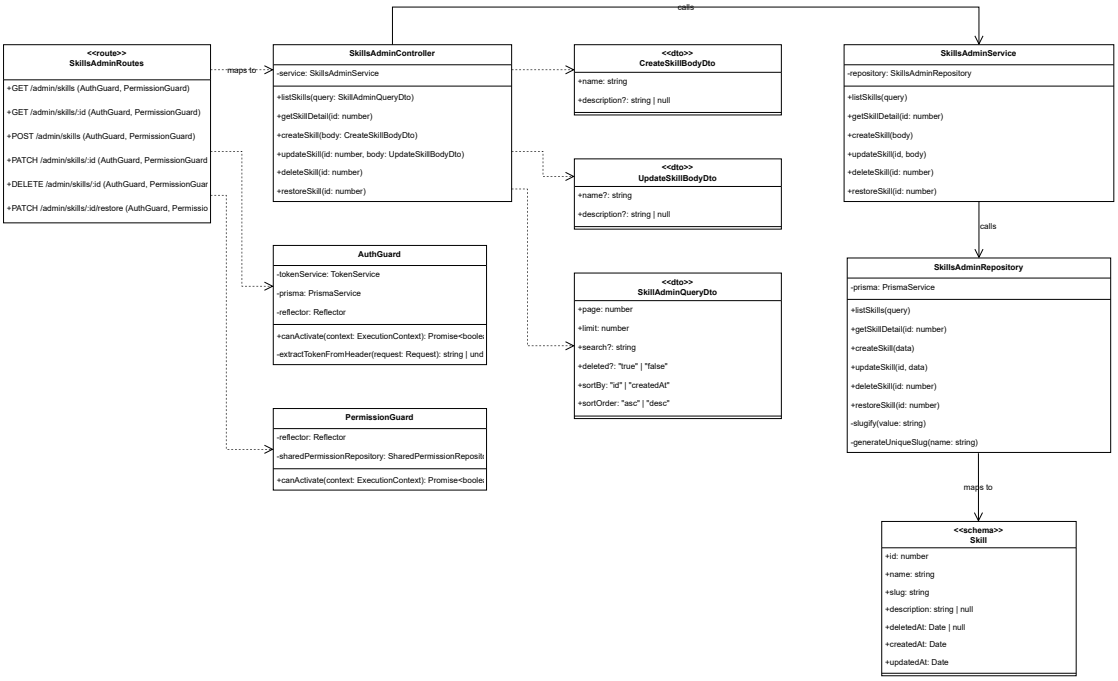


Figure 49: Class diagram of Skill Management

b. Sequence Diagram(s)

3.9.1 UC-38.01 – View Skill List

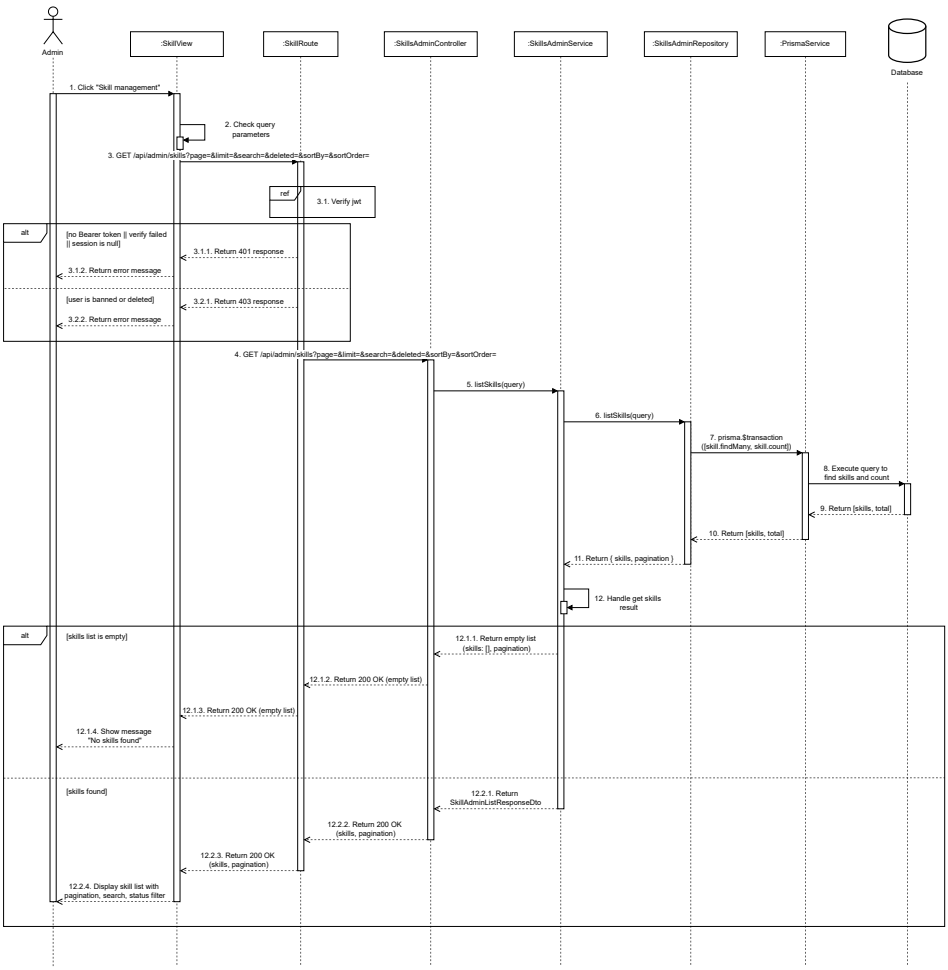


Figure 50: Sequence diagram of UC-38.01 View Skill List

3.9.2 UC-38.02 – View Skill Detail

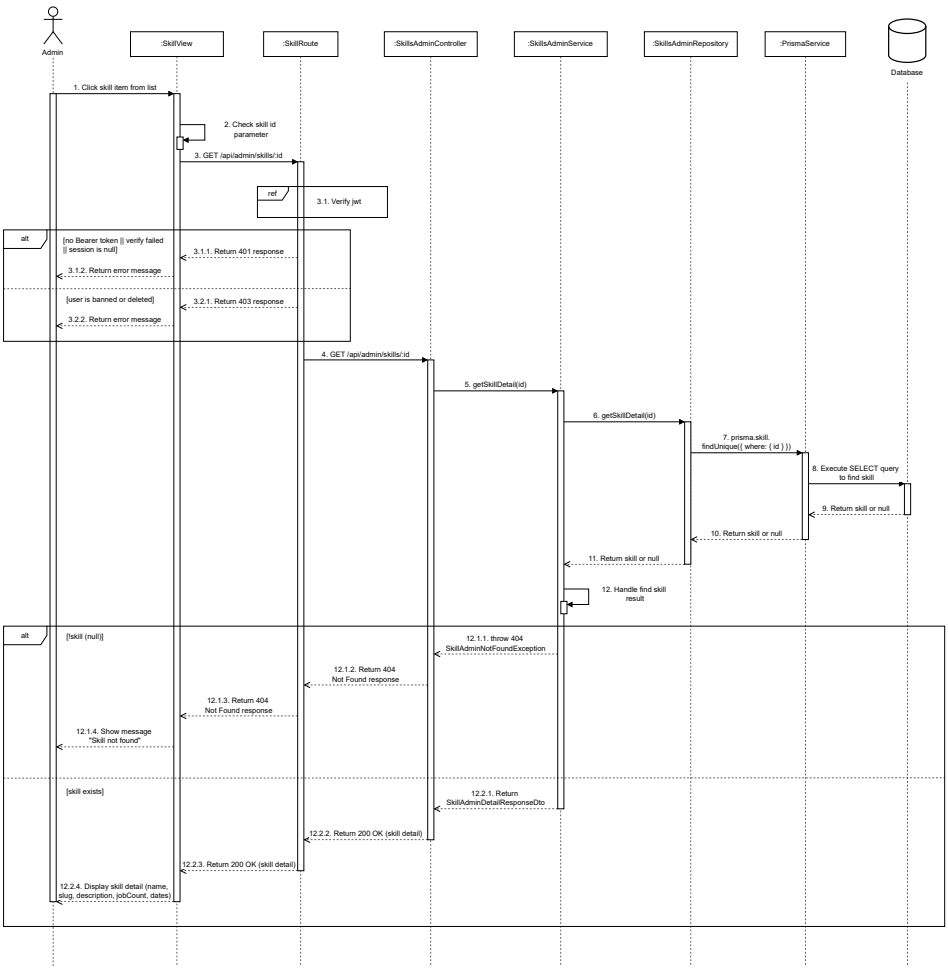


Figure 51: Sequence diagram of UC-38.02 View Skill Detail

3.9.3 UC-38.03 – Create Skill

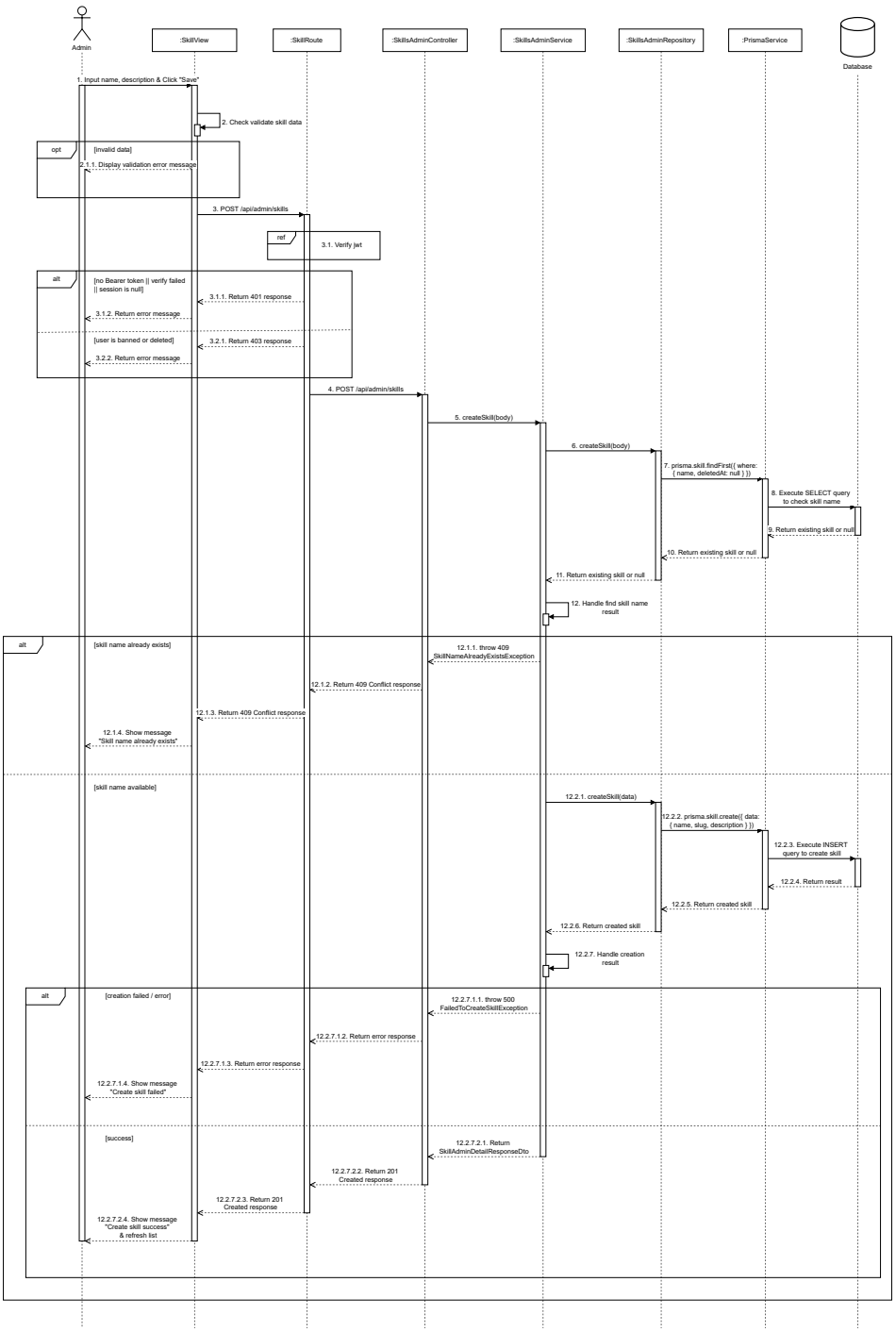


Figure 52: Sequence diagram of UC-38.03 Create Skill

3.9.4 UC-38.04 – Update Skill

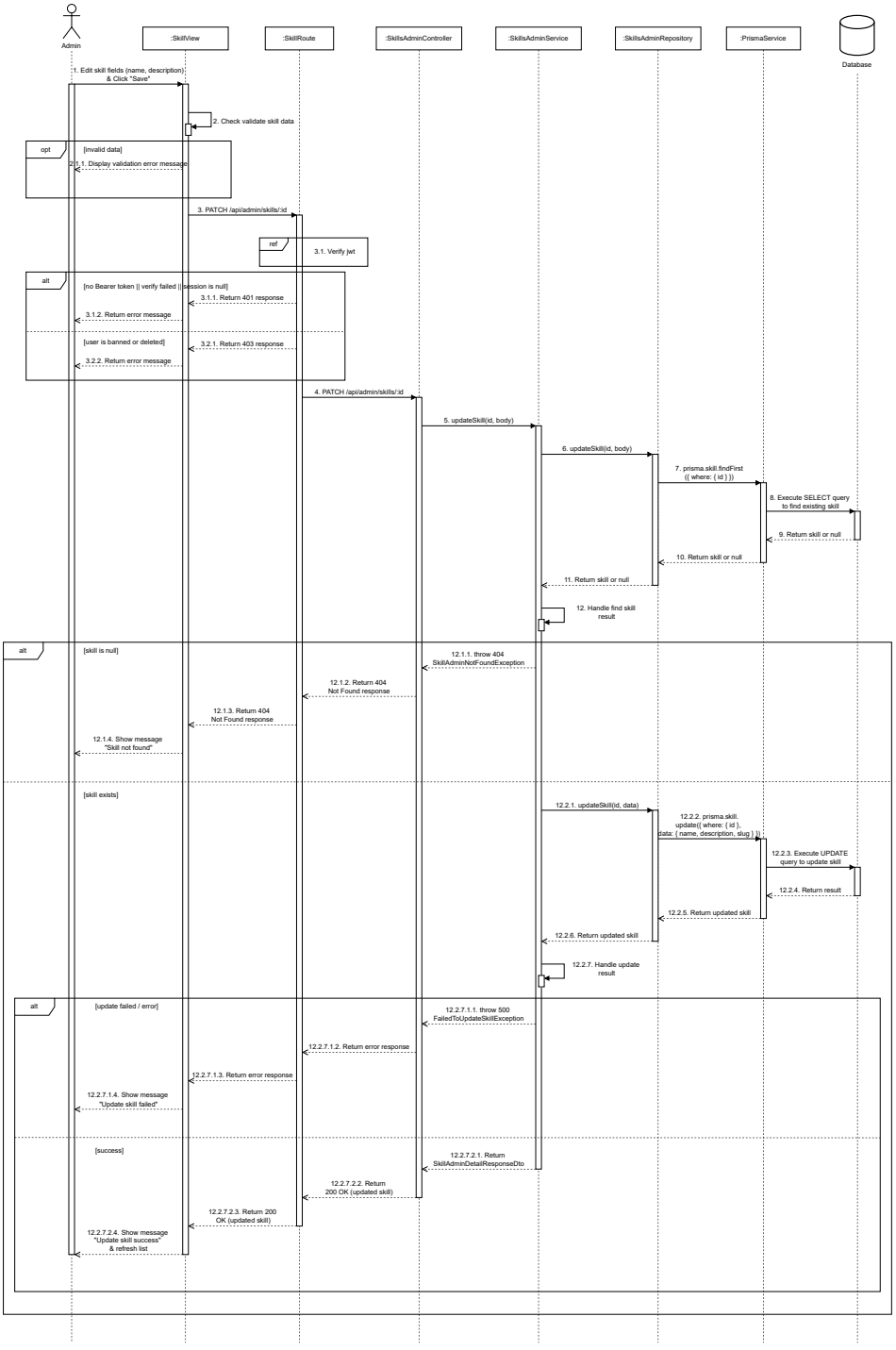


Figure 53: Sequence diagram of UC-38.04 Update Skill

3.9.5 UC-38.05 – Delete Skill

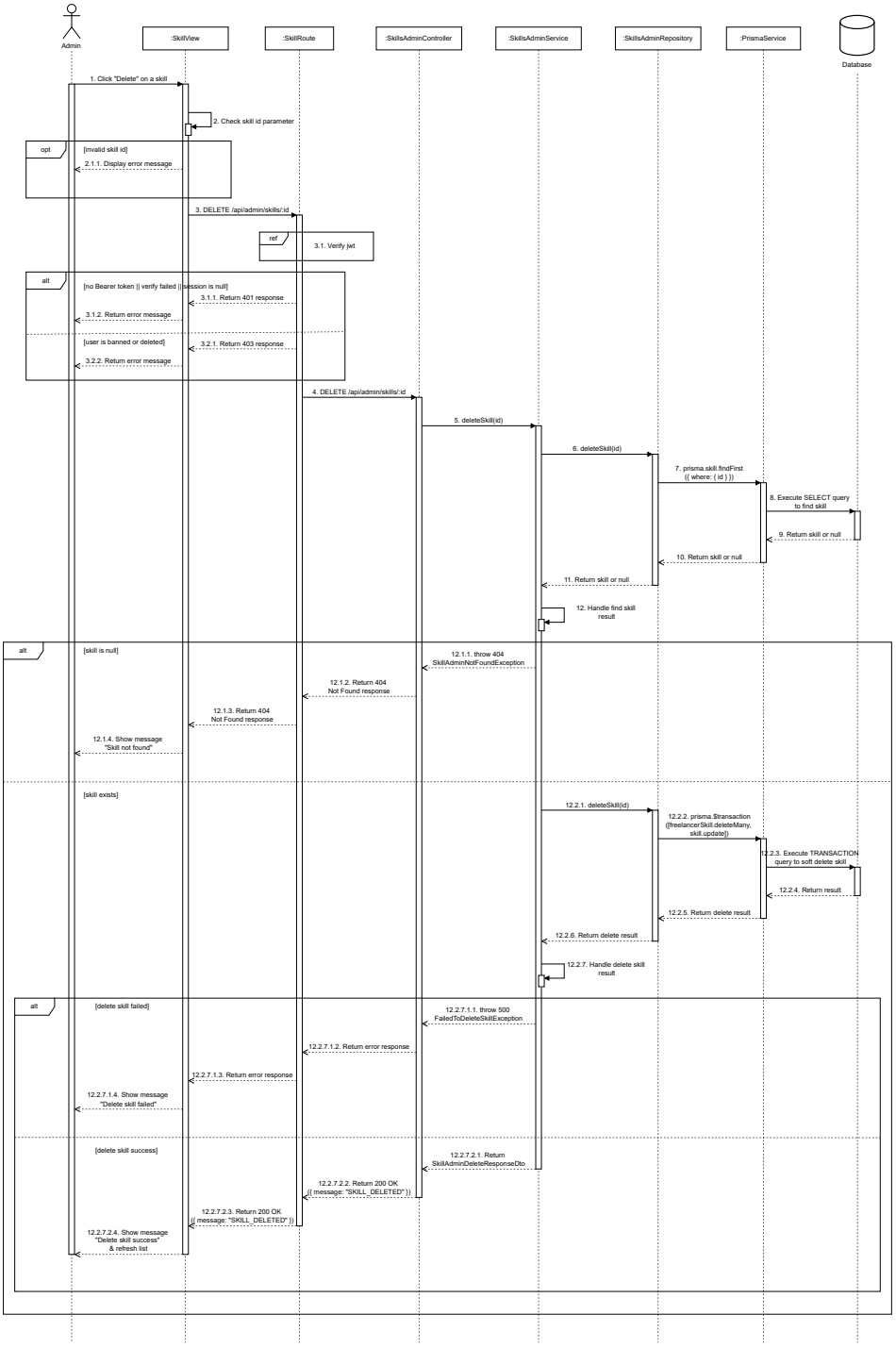


Figure 54: Sequence diagram of UC-38.05 Delete Skill

3.9.6 UC-38.06 – Restore Skill

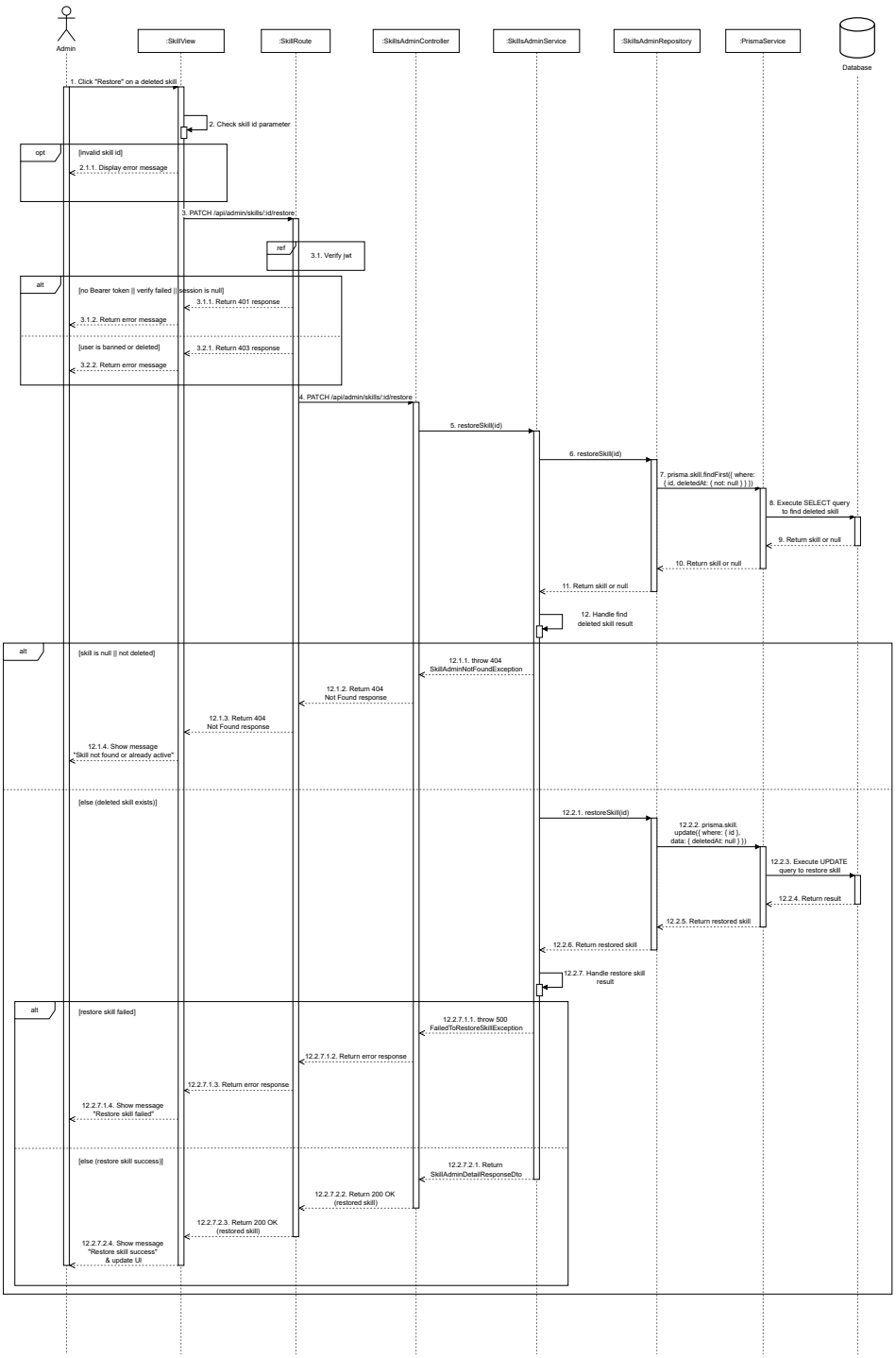


Figure 55: Sequence diagram of UC-38.06 Restore Skill

3.10 Expert Profile Management

The administrator-side Expert Profile editor updates an expert account directly. Any pending self-service revision is superseded, and the account owner is notified of the administra-

tive change.

a. Class Diagram

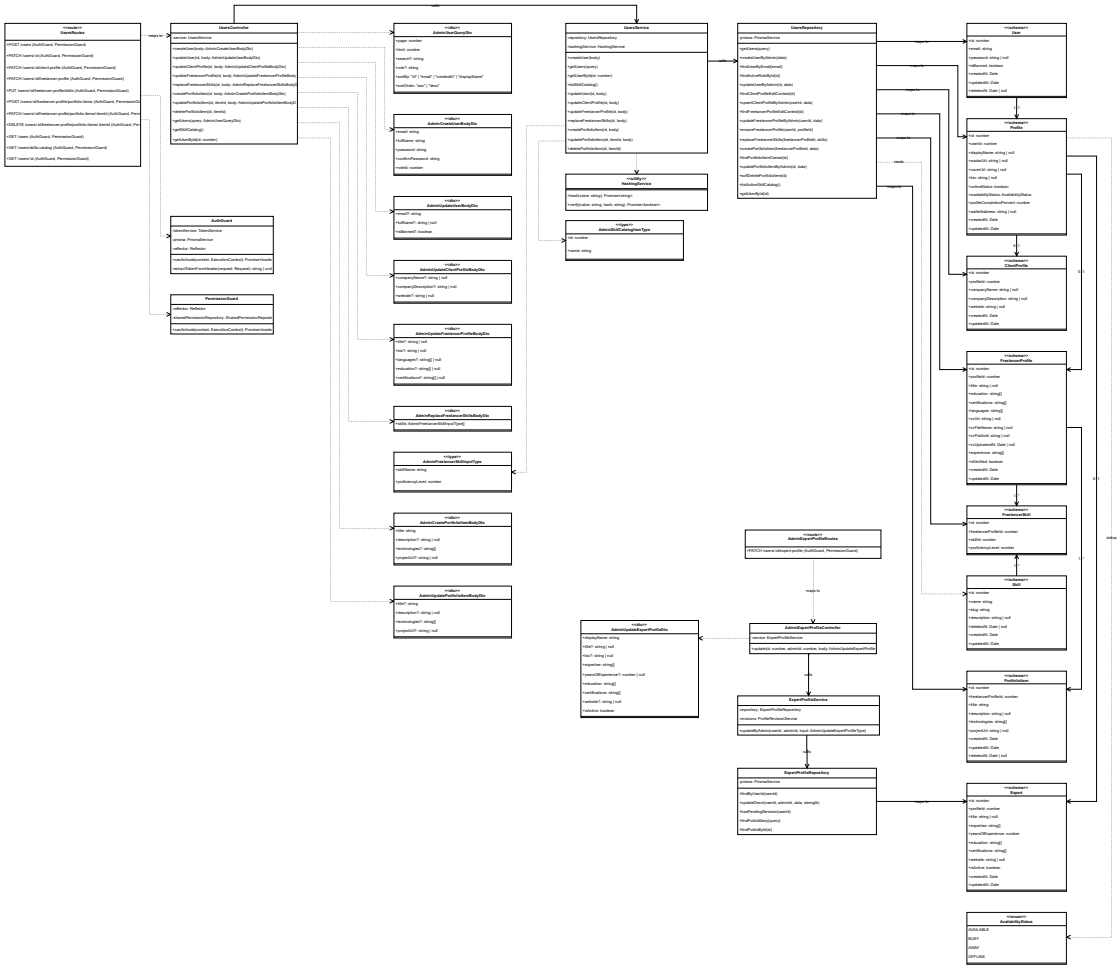


Figure 56: Class diagram of Expert Profile Management

b. Sequence Diagram(s)

3.10.1 UC-28.09 – Update Expert Profile

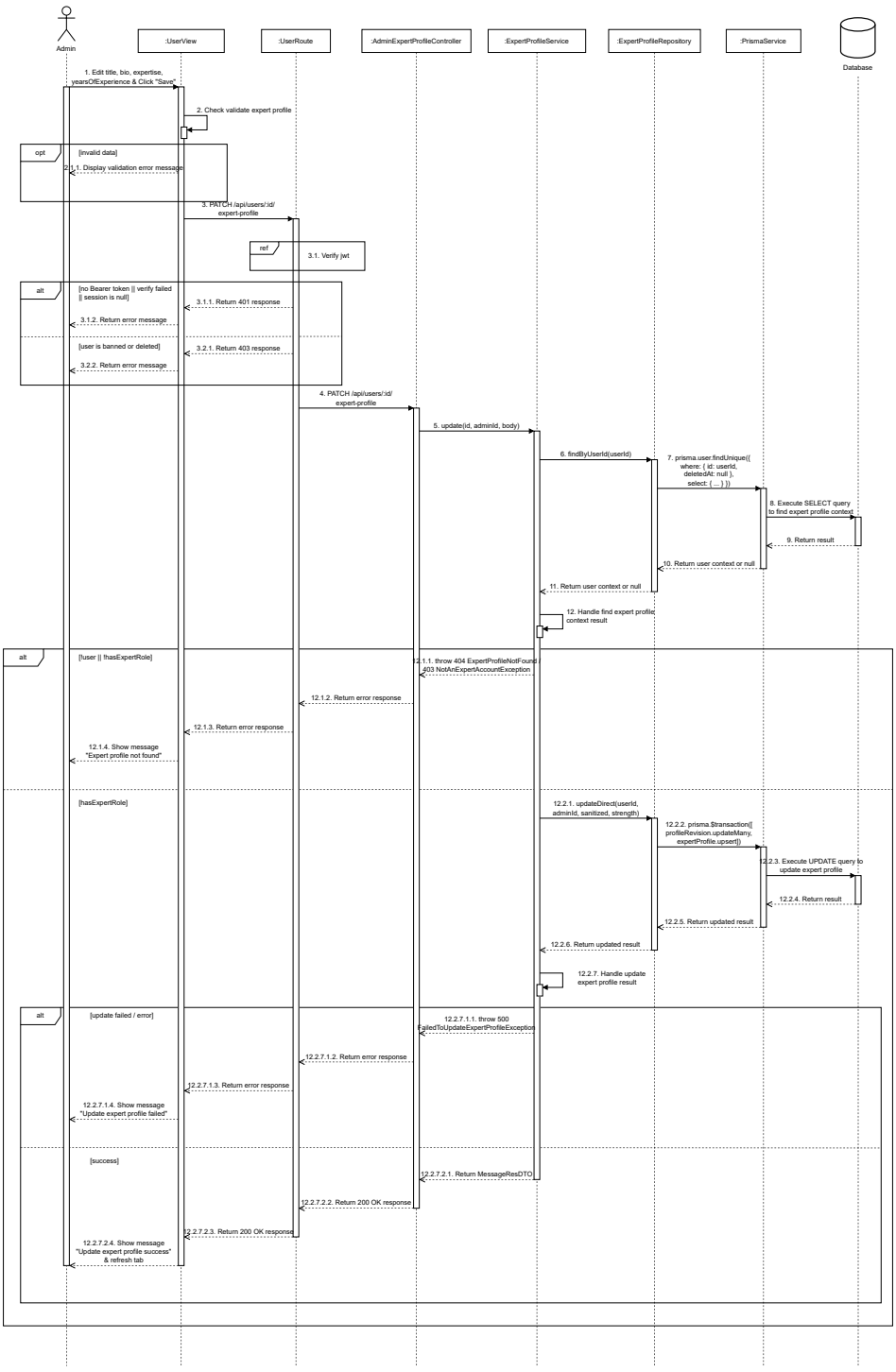


Figure 57: Sequence diagram of UC-28.09 Update Expert Profile

3.11 Job Category Management

Job Category Management maintains the taxonomy used by job postings. A category can be created, updated, soft-deleted, and restored; deletion is blocked while active jobs still

reference the category.

a. Class Diagram

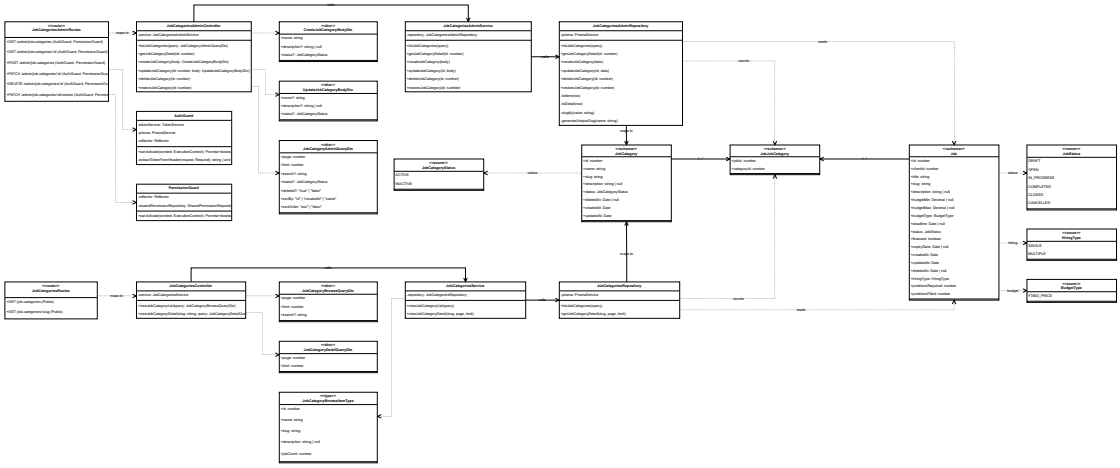


Figure 58: Class diagram of Job Category Management

b. Sequence Diagram(s)

3.11.1 UC-46.01 – View Job Category List

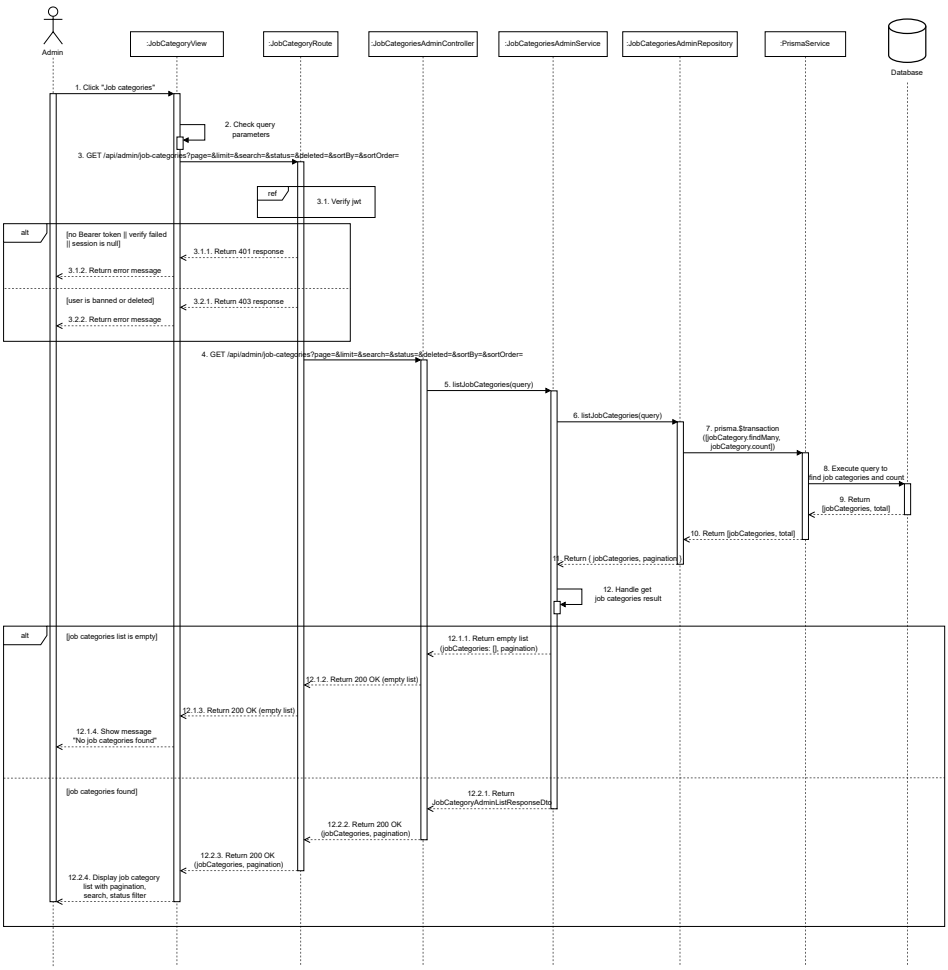


Figure 59: Sequence diagram of UC-46.01 View Job Category List

3.11.2 UC-46.02 – View Job Category Detail

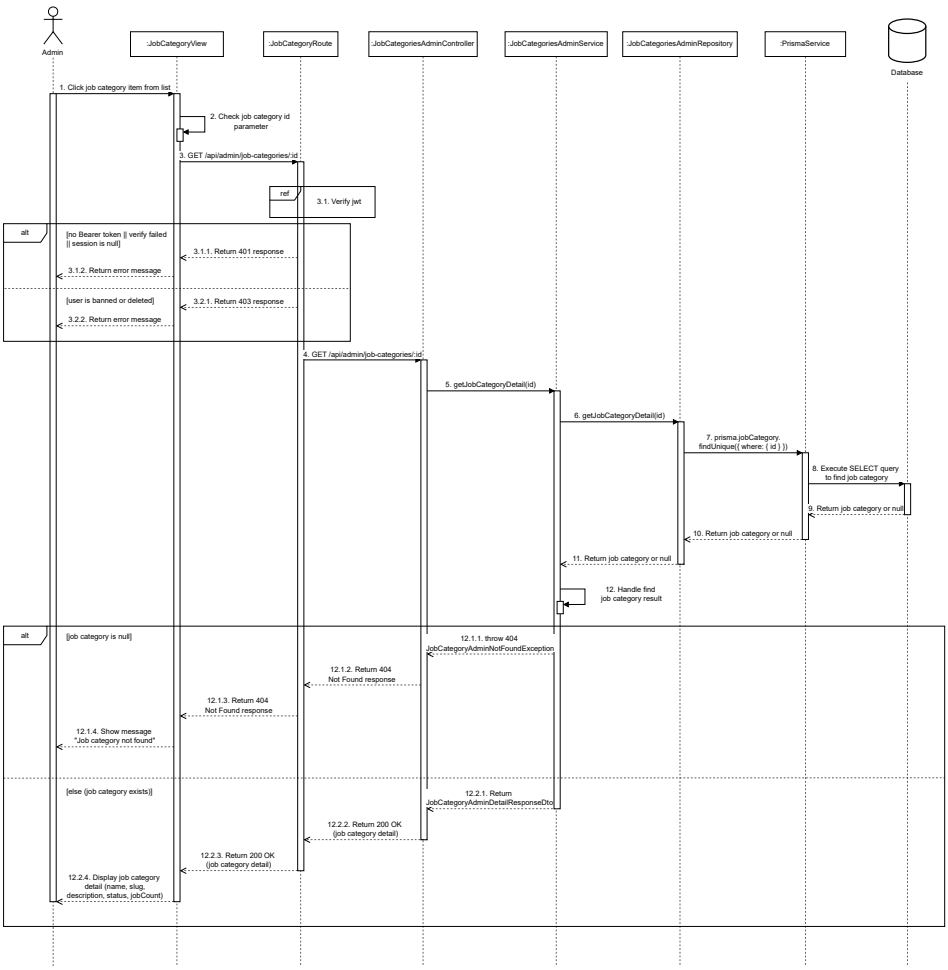


Figure 60: Sequence diagram of UC-46.02 View Job Category Detail

3.11.3 UC-46.03 – Create Job Category

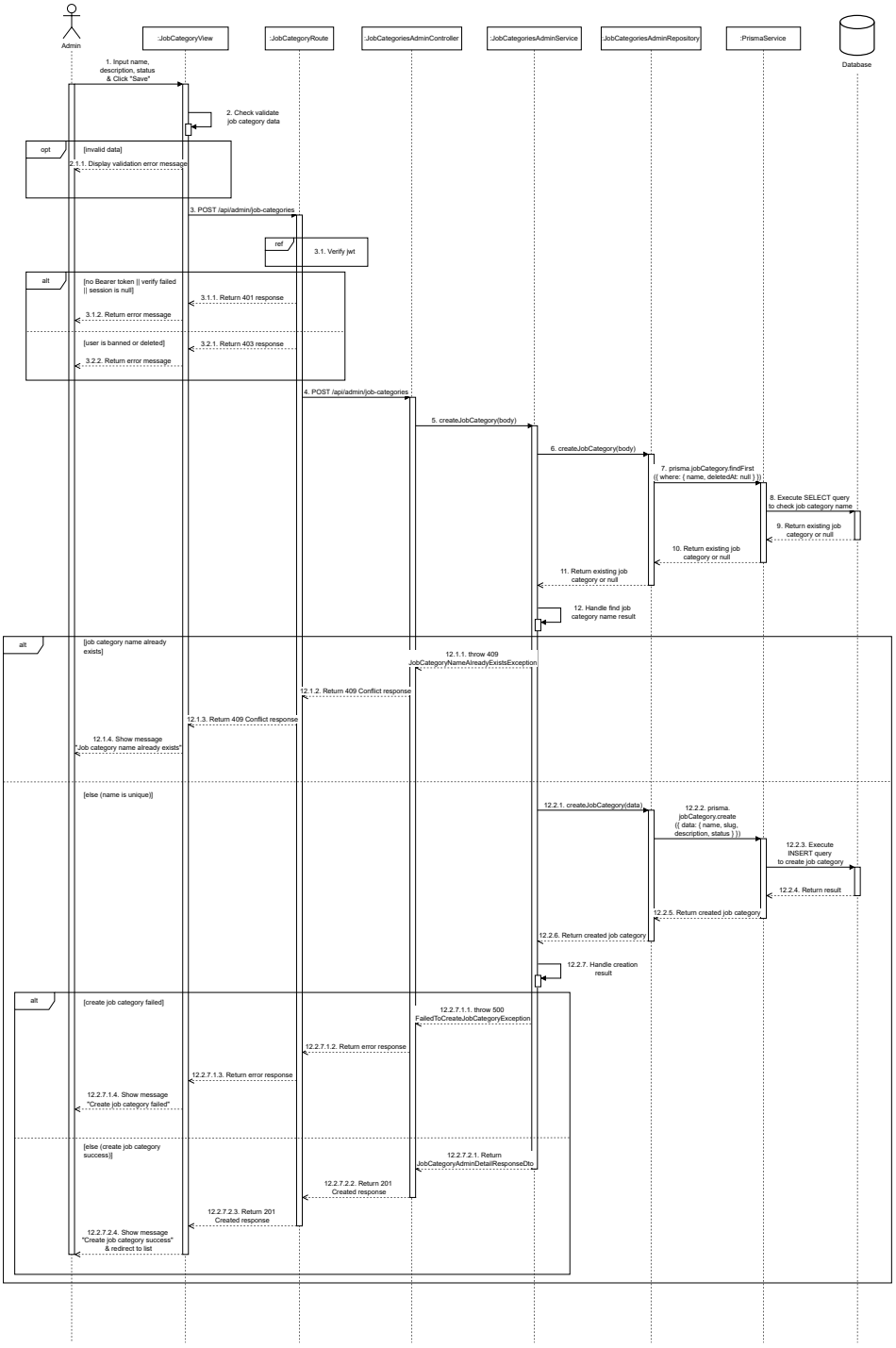


Figure 61: Sequence diagram of UC-46.03 Create Job Category

3.11.4 UC-46.04 – Update Job Category

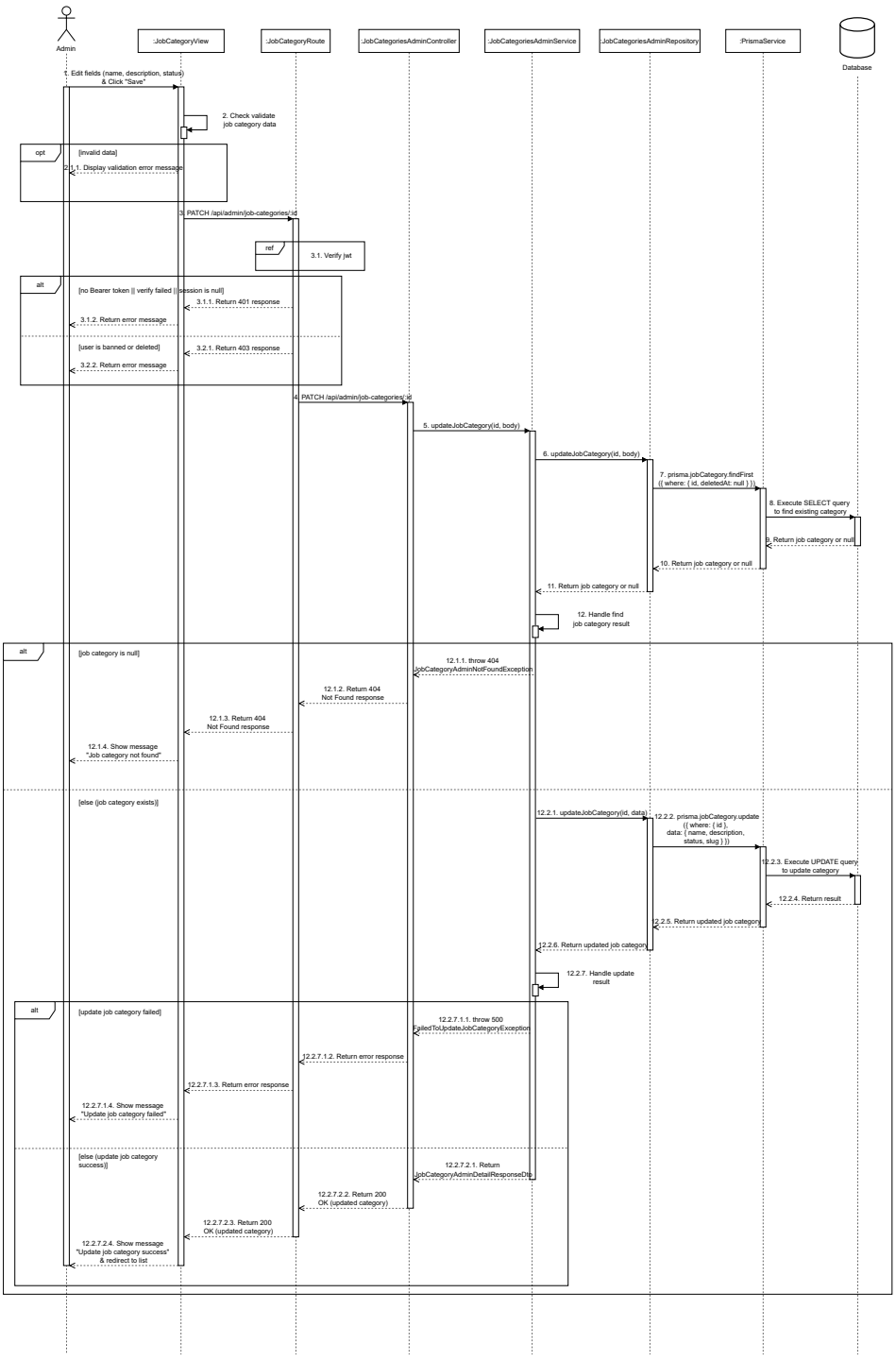


Figure 62: Sequence diagram of UC-46.04 Update Job Category

3.11.5 UC-46.05 – Delete Job Category

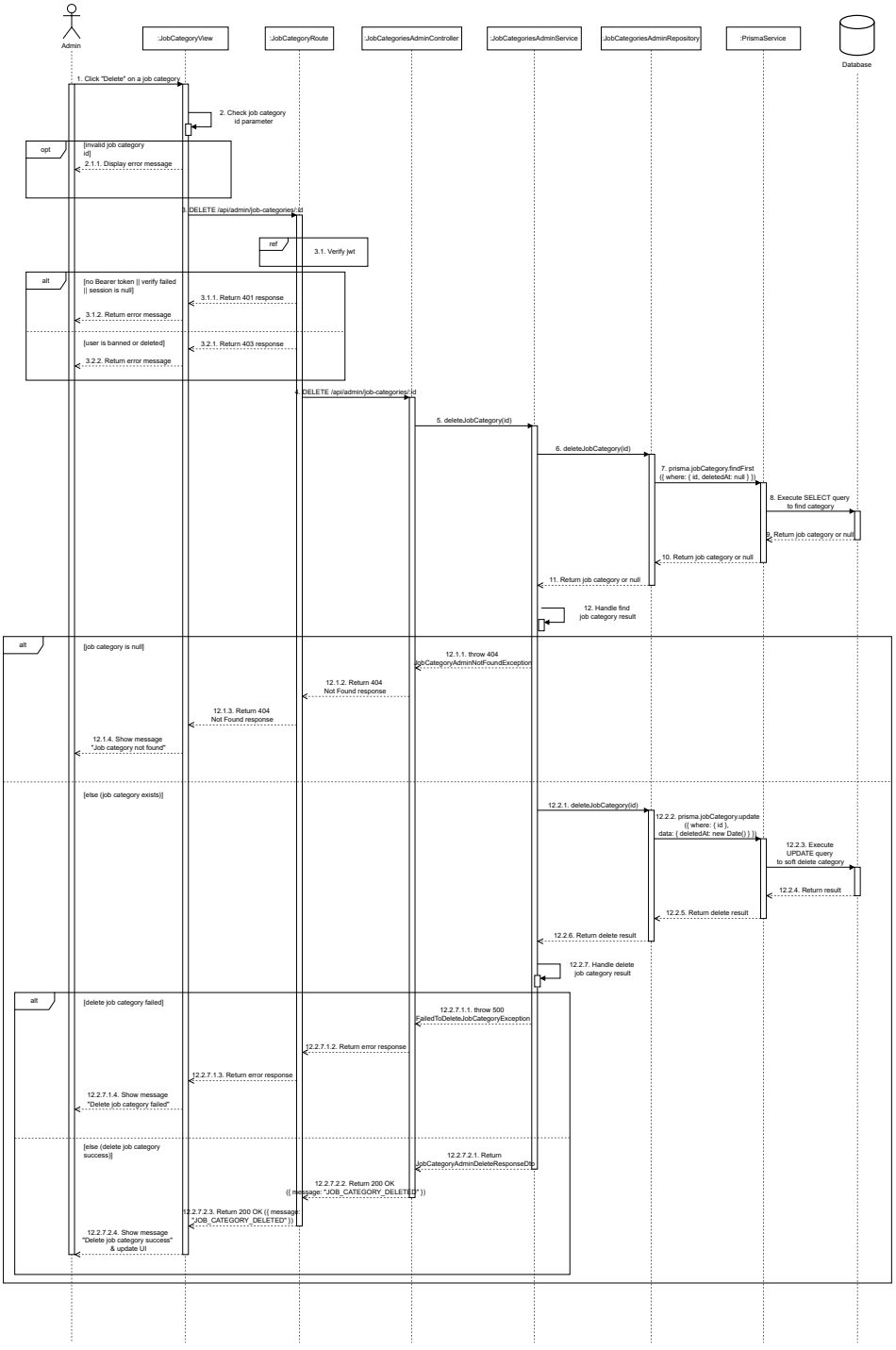


Figure 63: Sequence diagram of UC-46.05 Delete Job Category

4 Class Specification

4.1 AuthGuard Class

4.1.1 ClassMethod

No	Method	Description
01	`AuthGuard( reflector: Reflector, tokenService: Token Service, sessionRepo: Session Repository)`	A method to initialize the authentication guard with dependency services. – `reflector`: `Reflector` – metadata reflector to check if a route is marked public. – `tokenService`: `TokenService` – token decoding and verification utility. – `sessionRepo`: `SessionRepository` – repository to query session active status. – *Returns:* `void` – constructor.
02	`canActivate(context: ExecutionContext)`	A method to inspect the execution context, extract the Bearer token, verify its validity and expiration, and verify that the session has not been revoked. – `context`: `ExecutionContext` – execution context containing HTTP request/response. – *Returns:* `Promise<boolean>` – `true` if authentication succeeds, throws `UnauthorizedException (401)` otherwise.
03	`extractTokenFromHeader( request: Request)`	A helper method to extract the Bearer token from the incoming HTTP Authorization header. – `request`: `Request` – current Express request object. – *Returns:* `string | null` – extracted raw token string, or `null` if not found.

4.1.2 ClassAttribute

No	Attribute	Type
01	`reflector`	`Reflector`
02	`tokenService`	`TokenService`
03	`sessionRepo`	`SessionRepository`

4.2 AuthController Class

4.2.1 ClassMethod

No	Method	Description
01	`AuthController(auth Service: AuthService)`	A method to initialize the controller with the authentication service. – `authService`: `AuthService` – service that owns the authentication use cases. – *Returns:* `void` – constructor, no return value.
02	`register(body: Register Dto)`	A method to create an email/password account after OTP verification. – `body`: `RegisterDto` – email, OTP code, password, full name, and initial role. – *Returns:* `Promise<RegisterResponseType>` – the created account without password fields. – Route: `POST /auth/register`; public endpoint.
03	`sendOTP(body: SendOTP BodyDTO)`	A method to generate and email an OTP for registration or password reset. – `body`: `SendOTPBodyDTO` – email and verification-code type. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `POST /auth/otp`; public endpoint.
04	`login( body: LoginBodyDTO, userAgent: string, ip Address: string)`	A method to authenticate an email/password account and create a session. – `body`: `LoginBodyDTO` – email and password. – `userAgent`: `string` – device information used as session metadata. – `ipAddress`: `string` – client IP stored with the session. – *Returns:* `Promise<LoginResponseType>` – access and refresh tokens. – Route: `POST /auth/login`; public endpoint.

No	Method	Description
05	`refreshToken( body: RefreshTokenBodyDTO, userAgent: string, ipAddress: string)`	A method to rotate a valid refresh token into a new token pair. – `body.refreshToken`: `string` – the current refresh token. – `userAgent`: `string` – device information for the replacement session. – `ipAddress`: `string` – client IP for the replacement session. – *Returns:* `Promise<LoginResponseType>` – the new access and refresh tokens. – Route: `POST /auth/refresh-token`; public endpoint.
06	`logout(body: LogoutBodyDTO)`	A method to terminate the session identified by a refresh token. – `body.refreshToken`: `string` – refresh token of the session to remove. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `POST /auth/logout`; public endpoint.
07	`forgotPassword(body: ForgotPasswordBodyDTO)`	A method to reset a password after validating a password-reset OTP. – `body`: `ForgotPasswordBodyDTO` – email, OTP code, and new password. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `POST /auth/forgot-password`; public endpoint.
08	`getGoogleLink( userAgent: string, ip: string)`	A method to build the Google OAuth consent URL. – `userAgent`: `string` – encoded in the OAuth state. – `ip`: `string` – encoded in the OAuth state. – *Returns:* `GetAuthorizationUrlResType` – the authorization URL. – Route: `GET /auth/google-link`; public endpoint.

No	Method	Description
09	`googleCallback( code: string, state: string, res: Response)`	A method to exchange the Google authorization code and redirect the browser. – `code`: `string` – authorization code returned by Google. – `state`: `string` – base64url payload with device and IP information. – `res`: `Response` – Express response used for the redirect. – *Returns:* `Promise<void>` – redirects to the web application with tokens or an error flag. – Route: `GET /auth/google/callback`; public endpoint.
10	`getMe(userId: number)`	A method to return the authenticated account and its roles. – `userId`: `number` – current user injected by `@User Active('userId')`. – *Returns:* `Promise<GetMeResType>` – account, profile summary, and roles. – Route: `GET /auth/me`; authenticated endpoint.
11	`switchRole( userId: number, session Id: number, body: SwitchRoleBodyDto)`	A method to make an already assigned role the primary role. – `userId`: `number` – current user. – `sessionId`: `number` – current session identifier. – `body.role`: role name to activate. – *Returns:* `Promise<SwitchRoleResponseType>` – a new access token for that role. – Route: `POST /auth/switch-role`; authenticated endpoint.
12	`joinRole( userId: number, session Id: number, body: JoinRoleBodyDto)`	A method to assign an additional Client or Freelancer role. – `userId`: `number` – current user. – `sessionId`: `number` – current session identifier. – `body.role`: role name to join. – *Returns:* `Promise<JoinRoleResponseType>` – a new access token for the joined role. – Route: `POST /auth/join-role`; authenticated endpoint.

4.2.2 ClassAttribute

No	Attribute	Type
01	`authService`	`AuthService` – private readonly authentication service.
02	`logger`	`Logger` – private readonly NestJS logger for OAuth callback errors.

4.3 AuthService Class

4.3.1 ClassMethod

No	Method	Description
01	`AuthService( hashingService: HashingService, emailService: EmailService, sharedRoleRepository: SharedRoleRepository, authRepository: AuthRepository, tokenService: TokenService)`	A method to initialize the service and its Google OAuth client. – `hashingService`: `HashingService` – password hashing and verification. – `emailService`: `EmailService` – OTP delivery. – `sharedRoleRepository`: `SharedRoleRepository` – resolves system role identifiers. – `authRepository`: `AuthRepository` – account, OTP, session, and OAuth persistence. – `tokenService`: `TokenService` – JWT signing and verification. – *Returns:* `void` – constructor, no return value.
02	`register(body: RegisterBodyType)`	A method to validate registration data and create an account. – `body`: `RegisterBodyType` – email, OTP, password, full name, and role. – *Returns:* `Promise<UserType>` – the created account without sensitive fields. – Throws `EmailAlreadyExistsException`, `UserBannedException`, `InvalidVerificationCodeException`, or `OTPExpiredException`.

No	Method	Description
03	`sendOTP(body: SendOTPBodyType)`	A method to issue an OTP while enforcing the attempt window. – `body`: `SendOTPBodyType` – email and verification-code type. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `EmailAlreadyExistsException`, `EmailNotFoundException`, or `TooManyAttemptsException`.
04	`login(body: LoginBodyType and session metadata)`	A method to verify credentials, ban state, and the primary role before issuing tokens. – `body`: `LoginBodyType` – email, password, user agent, and IP address. – *Returns:* `Promise<{ accessToken: string; refreshToken: string }>` – the token pair. – Throws `IncorrectEmailException`, `IncorrectPasswordException`, `UserBannedException`, or `RoleNotFoundException`.
05	`generateAccessAndRefreshTokens(payload)`	A private method to sign a refresh token, persist a session, and sign the access token. – `payload`: object – user id, role id, role name, user agent, and IP address. – *Returns:* `Promise<{ accessToken: string; refreshToken: string }>` – the new token pair. – The access token carries the persisted session id.
06	`refreshToken(payload)`	A method to validate and rotate a refresh token. – `payload`: object – refresh token, user agent, and IP address. – *Returns:* `Promise<{ accessToken: string; refreshToken: string }>` – the replacement token pair. – Throws `RefreshTokenRevokedException` or `UserBannedException`.
07	`logout(refreshToken: string)`	A method to remove the session that owns a refresh token. – `refreshToken`: `string` – token of the session to terminate. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `RefreshTokenRevokedException` when the session is absent.

No	Method	Description
08	`forgotPassword(payload: ForgotPasswordBodyType)`	A method to validate a reset OTP, hash the new password, and remove the OTP. – `payload`: `ForgotPasswordBodyType` – email, OTP, and new password. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `EmailNotFoundException`, `UserBanned Exception`, or OTP validation errors.
09	`validateVerificationCode( uniqueValue)`	A method to load an OTP and reject missing or expired codes. – `uniqueValue`: object – email, code, and verification type. – *Returns:* `Promise<EmailVerificationType>` – the matching OTP record. – Throws `InvalidVerificationCodeException` or `OT PExpiredException`.
10	`getMe(userId: number)`	A method to return the current account, profile summary, and roles. – `userId`: `number` – current account identifier. – *Returns:* `Promise<GetMeResType>` – the authenticated account view. – Throws `NotFoundException` when absent or `User BannedException` when banned.
11	`switchRole(userId, sessionId, body)`	A method to change the primary role and issue a new access token. – `userId`: `number` – current account. – `sessionId`: `number` – current session. – `body`: `SwitchRoleBodyType` – target role. – *Returns:* `Promise<{ accessToken: string }>` – the role-scoped access token. – Throws `RoleNotFoundException`.

No	Method	Description
12	`joinRole(userId, sessionId, body)`	A method to assign an additional role and issue a new access token. – `userId`: `number` – current account. – `sessionId`: `number` – current session. – `body`: `JoinRoleBodyType` – role to join. – *Returns:* `Promise<{ accessToken: string }>` – the role-scoped access token. – Throws `RoleNotFoundException` or `RoleAlreadyAssignedException`.
13	`getAuthorizationUrl(payload)`	A method to build the Google OAuth consent URL with a base64url state. – `payload`: object – user agent and IP address. – *Returns:* `GetAuthorizationUrlResType` – the authorization URL.
14	`googleCallback(payload)`	A method to exchange a Google code, create or load the account, and issue tokens. – `payload`: object – Google authorization code and state. – *Returns:* `Promise<{ accessToken: string; refreshToken: string }>` – the token pair. – Creates a Client account and an OAuth account on first login; rejects banned accounts.

4.3.2 ClassAttribute

No	Attribute	Type
01	`hashingService`	`HashingService` – private readonly password utility.
02	`emailService`	`EmailService` – private readonly OTP delivery utility.
03	`sharedRoleRepository`	`SharedRoleRepository` – private readonly system-role lookup.
04	`authRepository`	`AuthRepository` – private readonly authentication persistence.
05	`tokenService`	`TokenService` – private readonly JWT utility.
06	`oauth2Client`	`OAuth2Client` – public Google OAuth client.

No	Attribute	Type
07	`logger`	`Logger` – private readonly NestJS logger.

4.4 AuthRepository Class

4.4.1 ClassMethod

No	Method	Description
01	`AuthRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findUserByEmail(email: string)`	A method to load a non-deleted account by email. – `email`: `string` – unique account email. – *Returns:* `Promise<UserType | null>` – the account without password timestamps, or `null`.
03	`findUserByEmailInclude Roles(email: string)`	A method to load an account with its role assignments. – `email`: `string` – unique account email. – *Returns:* `Promise<UserType and roles | null>` – the account and ordered roles, or `null`.
04	`findVerificationCode( uniqueValue)`	A method to find an OTP by email, code, and type. – `uniqueValue`: object – email, code, and verification type. – *Returns:* `Promise<EmailVerificationType | null>` – the matching OTP, or `null`.
05	`findVerificationCodeBy EmailAndType(email, type)`	A method to load the current OTP record for an email and type. – `email`: `string` – account email. – `type`: `VerificationCodeType` – registration or password-reset code. – *Returns:* `Promise<EmailVerificationType | null>` – the OTP record, or `null`.

No	Method	Description
06	`createUserAndRegister( deleteOtp, user)`	A method to delete the consumed OTP and create the account in one transaction. – `deleteOtp`: object – OTP code, type, and email. – `user`: object – email, password, full name, and initial role. – *Returns:* `Promise<UserType>` – the created account.
07	`createUserAndRegister Google(user)`	A method to create a Google account with a Client role and profile. – `user`: object – email, full name, and role identifier. – *Returns:* `Promise<UserType and roles>` – the created account with roles.
08	`upsertVerificationCode( data)`	A method to create or replace an OTP record. – `data`: object – email, type, code, expiry, and attempt count. – *Returns:* `Promise<EmailVerificationType>` – the persisted OTP.
09	`findUserForLogin(email: string)`	A method to load an account and ordered roles for authentication. – `email`: `string` – account email. – *Returns:* `Promise<UserType and roles | null>` – the account, or `null`.
10	`createSession(data)`	A method to persist a refresh-token session. – `data`: object – user id, refresh token, device, IP address, and expiry. – *Returns:* `Promise<Session>` – the created session with its identifier.
11	`deleteSessionByRefresh Token(data)`	A method to delete a session by refresh token and user id. – `data`: object – refresh token and user id. – *Returns:* `Promise<Session>` – the deleted session.
12	`findUniqueRefreshToken IncludeUserRole(data)`	A method to load a session, its account, and the account roles by refresh token. – `data.refreshToken`: `string` – refresh token to resolve. – *Returns:* `Promise<Session and user roles | null>` – the session graph, or `null`.

No	Method	Description
13	`update(where, data)`	A method to update a non-deleted account. – `where`: object – account identifier. – `data`: `Partial<UserType>` – fields to change. – *Returns:* `Promise<UserType | null>` – the updated account.
14	`deleteVerifycationCode( uniqueValue)`	A method to delete an OTP after a successful password reset. – `uniqueValue`: object – email and verification type. – *Returns:* `Promise<EmailVerificationType>` – the deleted OTP.
15	`findUserById(userId: number)`	A method to load an account with profile and ordered roles. – `userId`: `number` – account identifier. – *Returns:* `Promise<UserType and profile and roles | null>` – the account view, or `null`.
16	`switchPrimaryRole(userId, roleName)`	A method to make an assigned role primary in one transaction. – `userId`: `number` – account identifier. – `roleName`: `string` – role to activate. – *Returns:* `Promise<RoleType | null>` – the activated role, or `null` when not assigned.
17	`joinRole(userId, role Name)`	A method to assign a Client or Freelancer role and create its profile row. – `userId`: `number` – account identifier. – `roleName`: `JoinRoleBodyType['role']` – role to join. – *Returns:* `Promise<{ status: 'joined' | ' already-assigned'; role: RoleType } | null>` – the assignment result, or `null` when the role is absent.
18	`createOauthAccount(data)`	A method to link a Google identity to an account. – `data`: object – user id, provider, and provider user id. – *Returns:* `Promise<OauthAccount>` – the created OAuth link.

4.4.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.5 RolesController Class

4.5.1 ClassMethod

No	Method	Description
01	`RolesController(rolesService: RolesService)`	A method to initialize the controller with the role service. – `rolesService`: `RolesService` – service that owns role use cases. – *Returns:* `void` – constructor, no return value.
02	`getRoles()`	A method to return all active roles. – *Returns:* `Promise<RoleListResponseType>` – the role list. – Route: `GET /roles`.
03	`getRoleById(id: number)`	A method to return one role with its permissions. – `id`: `number` – role identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<RoleDetailResponseType>` – the role detail. – Route: `GET /roles/:id`.
04	`createRole(body: CreateRoleBodyDto)`	A method to create a custom role. – `body`: `CreateRoleBodyDto` – role name and optional description. – *Returns:* `Promise<CreateRoleResponseType>` – the created role. – Route: `POST /roles`.
05	`updateRole(id: number, body: UpdateRoleBodyDto)`	A method to update a role's name or description. – `id`: `number` – role identifier. – `body`: `UpdateRoleBodyDto` – optional name and description. – *Returns:* `Promise<UpdateRoleResponseType>` – the updated role. – Route: `PATCH /roles/:id`.

No	Method	Description
06	`setRolePermissions( id: number, body: SetRolePermissions BodyDto)`	A method to replace every permission assigned to a role. – `id: number` – role identifier. – `body.permissionIds: number[]` – the complete replacement set. – *Returns:* `Promise<SetRolePermissionsResponse Type>` – the updated role detail. – Route: `PUT /roles/:id/permissions`.
07	`deleteRole(id: number)`	A method to soft-delete an unused custom role. – `id: number` – role identifier. – *Returns:* `Promise<DeleteRoleResponseType>` – a success message. – Route: `DELETE /roles/:id`.

4.5.2 ClassAttribute

No	Attribute	Type
01	`rolesService`	`RolesService` – private readonly role service.

4.6 RolesService Class

4.6.1 ClassMethod

No	Method	Description
01	`RolesService(roles Repository: Roles Repository)`	A method to initialize the service with the role repository. – `rolesRepository: RolesRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`isSystemRoleName(name: string)`	A private method to detect the four built-in role names. – `name: string` – role name to test. – *Returns:* `boolean` – true for Admin, Client, Freelancer, or Expert.

No	Method	Description
03	`getRoles()`	A method to load the active role list. – *Returns:* `Promise<RoleListResponseType>` – the role list. – Failure is mapped to `FailedToLoadRolesException`.
04	`getRoleById(id: number)`	A method to load a role and its permissions. – `id: number` – role identifier. – *Returns:* `Promise<RoleDetailResponseType>` – the role detail. – Prisma errors are mapped to `FailedToLoadRoleDetailException`.
05	`createRole(body: CreateRoleBodyType)`	A method to validate the name and create a custom role. – `body: CreateRoleBodyType` – role name and optional description. – *Returns:* `Promise<CreateRoleResponseType>` – the created role. – Throws `CannotModifySystemRoleException` or `RoleAlreadyExistsException`.
06	`updateRole(id: number, body: UpdateRoleBodyType)`	A method to validate system-role and duplicate-name rules before updating. – `id: number` – role identifier. – `body: UpdateRoleBodyType` – optional name and description. – *Returns:* `Promise<UpdateRoleResponseType>` – the updated role. – Throws `CannotModifySystemRoleException` or `RoleAlreadyExistsException`.
07	`deleteRole(id: number)`	A method to soft-delete a custom role that is not assigned to any user. – `id: number` – role identifier. – *Returns:* `Promise<DeleteRoleResponseType>` – a success message. – Throws `CannotModifySystemRoleException` or `RoleInUseException`.

No	Method	Description
08	`setRolePermissions(id, body)`	A method to validate permission ids and replace the role permission set. – `id`: `number` – role identifier. – `body`: `SetRolePermissionsBodyType` – permission ids to assign. – *Returns:* `Promise<SetRolePermissionsResponseType>` – the updated role detail. – Throws `InvalidPermissionIdsException`.

4.6.2 ClassAttribute

No	Attribute	Type
01	`rolesRepository`	`RolesRepository` – private readonly persistence dependency.
02	`logger`	`Logger` – private readonly NestJS logger.

4.7 RolesRepository Class

4.7.1 ClassMethod

No	Method	Description
01	`RolesRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findAll()`	A method to load all non-deleted roles ordered by id. – *Returns:* `Promise<RoleListItemType[]>` – the role list.
03	`findById(id: number)`	A method to load a role and its active permissions. – `id`: `number` – role identifier. – *Returns:* `Promise<RoleDetailResponseType>` – the role detail. – Throws `RoleNotFoundException`.

No	Method	Description
04	`findActiveByName(name, excludeId?)`	A method to find an active role by case-insensitive name. – `name: string` – role name. – `excludeId: number | undefined` – optional role to exclude. – *Returns:* `Promise<RoleListItemType | null>` – the matching role, or `null`.
05	`createRole(body: CreateRoleBodyType)`	A method to insert a role. – `body: CreateRoleBodyType` – name and optional description. – *Returns:* `Promise<CreateRoleResponseType>` – the created role.
06	`updateRole(id: number, body: UpdateRoleBodyType)`	A method to update the provided role fields. – `id: number` – role identifier. – `body: UpdateRoleBodyType` – optional name and description. – *Returns:* `Promise<UpdateRoleResponseType>` – the updated role.
07	`softDeleteRole(id: number)`	A method to set `deletedAt` on a role. – `id: number` – role identifier. – *Returns:* `Promise<void>` – no value.
08	`countAssignedUsers(roleId: number)`	A method to count user-role assignments. – `roleId: number` – role identifier. – *Returns:* `Promise<number>` – the assignment count.
09	`findActivePermissionIds(ids: number[])`	A method to filter permission ids down to active permissions. – `ids: number[]` – requested permission ids. – *Returns:* `Promise<number[]>` – the active ids.
10	`replaceRolePermissions(roleId, permissionIds)`	A method to delete the old role permissions and insert the new set in a transaction. – `roleId: number` – role identifier. – `permissionIds: number[]` – replacement permission ids. – *Returns:* `Promise<void>` – no value.

4.7.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.8 PermissionsController Class

4.8.1 ClassMethod

No	Method	Description
01	`PermissionsController( permissionsService: PermissionsService)`	A method to initialize the controller with the permission service. – `permissionsService`: `PermissionsService` – service that owns permission queries. – *Returns:* `void` – constructor, no return value.
02	`getPermissions(query: PermissionFilterDto)`	A method to return a filtered page of route permissions. – `query`: `PermissionFilterDto` – page, limit, search, method, module, and sort options. – *Returns:* `Promise<PermissionListResponseType>` – permissions, pagination, and module names. – Route: `GET /permissions`.
03	`getPermissionById(id: number)`	A method to return one permission. – `id`: `number` – permission identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<PermissionDetailResponseType>` – the permission detail. – Route: `GET /permissions/:id`.

4.8.2 ClassAttribute

No	Attribute	Type
01	`permissionsService`	`PermissionsService` – private readonly permission service.

4.9 PermissionsService Class

4.9.1 ClassMethod

No	Method	Description
01	`PermissionsService( permissionsRepository: PermissionsRepository)`	A method to initialize the service with the permission repository. – `permissionsRepository: PermissionsRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`getPermissions(filter: PermissionFilterType)`	A method to load a permission page and compute pagination metadata. – `filter: PermissionFilterType` – page, limit, search, method, module, and sort options. – *Returns:* `Promise<PermissionListResponseType>` – permissions, pagination, and module names. – Failure is mapped to `FailedToLoadPermissions Exception`.
03	`getPermissionById(id: number)`	A method to load one permission. – `id: number` – permission identifier. – *Returns:* `Promise<PermissionDetailResponse Type>` – the permission detail. – Prisma errors are mapped to `FailedToLoad PermissionDetailException`.

4.9.2 ClassAttribute

No	Attribute	Type
01	`permissionsRepository`	`PermissionsRepository` – private readonly persistence dependency.
02	`logger`	`Logger` – private readonly NestJS logger.

4.10 PermissionsRepository Class

4.10.1 ClassMethod

No	Method	Description
01	`PermissionsRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findAll(filter: PermissionFilterType)`	A method to query permissions, the matching total, and the distinct module list. – `filter`: `PermissionFilterType` – page, limit, search, method, module, and sort options. – *Returns:* `Promise<{ permissions: PermissionListItemType[]; total: number; modules: string[] }>` – the page and supporting data. – Search matches name, path, or module case-insensitively.
03	`findById(id: number)`	A method to load one active permission. – `id`: `number` – permission identifier. – *Returns:* `Promise<PermissionDetailResponseType>` – the permission detail. – Throws `PermissionNotFoundException`.

4.10.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.11 SessionsController Class

4.11.1 ClassMethod

No	Method	Description
01	`SessionsController( sessionsService: SessionsService)`	A method to initialize the controller with the session service. – `sessionsService`: `SessionsService` – service that owns session queries. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getMySessions( userId: number, sessionId: number, query: SessionFilterDto)`	A method to return the current user's sessions. – `userId: number` – current user. – `sessionId: number` – current session, used to mark the active row. – `query: SessionFilterDto` – page, limit, search, and sort options. – *Returns:* `Promise<SessionListResponseType>` – sessions and pagination. – Route: `GET /sessions`; authenticated endpoint.
03	`getMySessionById( userId: number, sessionId: number, id: number)`	A method to return one owned session. – `userId: number` – current user. – `sessionId: number` – current session, used to mark the active row. – `id: number` – session identifier. – *Returns:* `Promise<SessionDetailResponseType>` – the session detail. – Route: `GET /sessions/:id`; authenticated endpoint.
04	`revokeMySession( userId: number, sessionId: number, id: number)`	A method to revoke one owned session. – `userId: number` – current user. – `sessionId: number` – current session, used to compute `loggedOut`. – `id: number` – session identifier. – *Returns:* `Promise<RevokeSessionResponseType>` – a success message and logout flag. – Route: `DELETE /sessions/:id`; authenticated endpoint.

4.11.2 ClassAttribute

No	Attribute	Type
01	`sessionsService`	`SessionsService` – private readonly session service.

4.12 SessionsService Class

4.12.1 ClassMethod

No	Method	Description
01	`SessionsService(sessionsRepository: SessionsRepository)`	A method to initialize the service with the session repository. – `sessionsRepository`: `SessionsRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`getMySessions(userId, filter, currentSessionId?)`	A method to load a page of the user's sessions. – `userId`: `number` – current user. – `filter`: `SessionFilterType` – page, limit, search, and sort options. – `currentSessionId`: `number | undefined` – session to mark as current. – *Returns:* `Promise<SessionListResponseType>` – sessions and pagination. – Failure is mapped to `FailedToLoadSessionsException`.
03	`getMySessionById(id, userId, currentSessionId?)`	A method to load one session owned by the user. – `id`: `number` – session identifier. – `userId`: `number` – current user. – `currentSessionId`: `number | undefined` – session to mark as current. – *Returns:* `Promise<SessionDetailResponseType>` – the session detail. – Prisma errors are mapped to `FailedToLoadSessionDetailException`.
04	`revokeMySession(id, userId, currentSessionId?)`	A method to delete an active session owned by the user. – `id`: `number` – session identifier. – `userId`: `number` – current user. – `currentSessionId`: `number | undefined` – current session used to compute `loggedOut`. – *Returns:* `Promise<RevokeSessionResponseType>` – a success message and logout flag. – Prisma errors are mapped to `FailedToRevokeSessionException`.

4.12.2 ClassAttribute

No	Attribute	Type
01	`sessionsRepository`	`SessionsRepository` – private readonly persistence dependency.
02	`logger`	`Logger` – private readonly NestJS logger.

4.13 SessionsRepository Class

4.13.1 ClassMethod

No	Method	Description
01	`SessionsRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findAllByUserId(userId, filter, currentSessionId? )`	A method to query the user's sessions and mark the current one. – `userId`: `number` – owner identifier. – `filter`: `SessionFilterType` – page, limit, search, and sort options. – `currentSessionId`: `number | undefined` – session to mark as current. – *Returns:* `Promise<{ sessions: SessionListItem Type[]; total: number }>` – the page and total. – Search matches device information or IP address.
03	`findByIdForUser(id, user Id, currentSessionId?)`	A method to load one session owned by the user. – `id`: `number` – session identifier. – `userId`: `number` – owner identifier. – `currentSessionId`: `number | undefined` – session to mark as current. – *Returns:* `Promise<SessionDetailResponseType>` – the session detail. – Throws `SessionNotFoundException`.

No	Method	Description
04	`deleteActiveByIdForUser(id, userId)`	A method to delete an unexpired session owned by the user. – `id`: `number` – session identifier. – `userId`: `number` – owner identifier. – *Returns:* `Promise<void>` – no value. – Throws `SessionNotFoundException` or `SessionAlreadyExpiredException`.

4.13.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.14 Forum Category Administration (Shared Classes)

UC-14.01–14.02 and UC-17.01–17.06 use the shared `ForumAdminController`, `ForumAdminServi` and `ForumAdminRepository`. Their complete specifications, including the category list, detail, create, update, delete, and restore endpoints, are given once in the shared forum administration class specifications for Sprint 3 (see the `ForumAdminController` subsection).

4.15 UsersController Class

4.15.1 ClassMethod

No	Method	Description
01	`UsersController(service: UsersService)`	A method to initialize the controller with the user administration service. – `service`: `UsersService` – service that owns admin user use cases. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`createUser(body: AdminCreateUserBodyDto)`	A method to create an account with an initial role. – `body: AdminCreateUserBodyDto` – email, password, full name, and role. – *Returns:* `Promise<AdminCreateUserResponseType>` – the created account. – Route: `POST /users`.
03	`updateUser(id: number, actorId: number, body: AdminUpdateUserBodyDto)`	A method to update email, display name, or ban state. – `id: number` – target user identifier. – `actorId: number` – current administrator, used for the self-ban check. – `body: AdminUpdateUserBodyDto` – optional email, full name, and ban flag. – *Returns:* `Promise<AdminUpdateUserResponseType>` – the updated account. – Route: `PATCH /users/:id`.
04	`updateClientProfile(id: number, body: AdminUpdateClientProfileBodyDto)`	A method to update the Client profile attached to an account. – `id: number` – target user identifier. – `body: AdminUpdateClientProfileBodyDto` – company name, description, and website. – *Returns:* `Promise<AdminClientProfileResponseType>` – the updated Client profile. – Route: `PATCH /users/:id/client-profile`.
05	`updateFreelancerProfile(id: number, body: AdminUpdateFreelancerProfileBodyDto)`	A method to update the Freelancer headline and biography. – `id: number` – target user identifier. – `body: AdminUpdateFreelancerProfileBodyDto` – title, bio, languages, education, and certifications. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `PATCH /users/:id/freelancer-profile`.
06	`replaceFreelancerSkills(id: number, body: AdminReplaceFreelancerSkillsBodyDto)`	A method to replace every skill on a Freelancer profile. – `id: number` – target user identifier. – `body.skills`: array of skill name and proficiency pairs. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `PUT /users/:id/freelancer-profile/skills`.

No	Method	Description
07	`createPortfolioItem( id: number, body: AdminCreate PortfolioItemBodyDto)`	A method to add a portfolio item to a Freelancer profile. – `id: number` – target user identifier. – `body: AdminCreatePortfolioItemBodyDto` – title, description, technologies, and project URL. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `POST /users/:id/freelancer-profile/ portfolio-items`.
08	`updatePortfolioItem( id: number, itemId: number, body: AdminUpdate PortfolioItemBodyDto)`	A method to update one portfolio item owned by the Freelancer profile. – `id: number` – target user identifier. – `itemId: number` – portfolio item identifier. – `body: AdminUpdatePortfolioItemBodyDto` – optional item fields. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `PATCH /users/:id/freelancer-profile/ portfolio-items/:itemId`.
09	`deletePortfolioItem( id: number, itemId: number)`	A method to soft-delete one portfolio item. – `id: number` – target user identifier. – `itemId: number` – portfolio item identifier. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `DELETE /users/:id/freelancer-profile/ portfolio-items/:itemId`.
10	`getUsers(query: Admin UserQueryDto)`	A method to return a filtered, paginated user list. – `query: AdminUserQueryDto` – page, limit, search, role, sort, and status filters. – *Returns:* `Promise<AdminUserListResponseType>` – users and pagination. – Route: `GET /users`.
11	`getSkillCatalog()`	A method to return the active skill catalogue for the profile editor. – *Returns:* `Promise<AdminSkillCatalogListType>` – active skills ordered by name. – Route: `GET /users/skills-catalog`.

No	Method	Description
12	`getUserById(id: number)`	A method to return a consolidated user detail. – `id: number` – target user identifier. – *Returns:* `Promise<AdminUserDetailResponseType>` – account, roles, and profile summaries. – Route: `GET /users/:id`.

4.15.2 ClassAttribute

No	Attribute	Type
01	`service`	`UsersService` – private readonly user administration service.

4.16 UsersService Class

4.16.1 ClassMethod

No	Method	Description
01	`UsersService( repository: Users Repository, hashingService: Hashing Service)`	A method to initialize the service with persistence and password hashing. – `repository: UsersRepository` – user persistence dependency. – `hashingService: HashingService` – password hashing utility. – *Returns:* `void` – constructor, no return value.
02	`createUser(body: Admin CreateUserBodyType)`	A method to validate the email and role, hash the password, and create the account. – `body: AdminCreateUserBodyType` – email, password, full name, and role id. – *Returns:* `Promise<AdminCreateUserResponseType>` – the created account. – Throws `EmailAlreadyExistsException`, `CreateUserRoleNotFoundException`, `CannotAssignAdminRoleException`, or `FailedToCreateUserException`.

No	Method	Description
03	`getUsers(query: AdminUserQueryType)`	A method to load a user page. – `query`: `AdminUserQueryType` – page, limit, search, role, sort, and status filters. – *Returns:* `Promise<AdminUserListResponseType>` – users and pagination. – Failure is mapped to `FailedToGetUsersException`.
04	`getUserById(id: number)`	A method to load a consolidated account detail. – `id`: `number` – target user identifier. – *Returns:* `Promise<AdminUserDetailResponseType>` – account, roles, and profiles. – Throws `UserNotFoundException` when absent or `FailedToGetUserException` on an unexpected read failure.
05	`listSkillCatalog()`	A method to return the active skill catalogue. – *Returns:* `Promise<AdminSkillCatalogListType>` – active skills ordered by name.
06	`updateUser(id, actorId, body)`	A method to update account fields while enforcing the self-ban and email rules. – `id`: `number` – target user identifier. – `actorId`: `number` – current administrator. – `body`: `AdminUpdateUserBodyType` – optional email, full name, and ban flag. – *Returns:* `Promise<AdminUpdateUserResponseType>` – the updated account. – Throws `CannotBanSelfException`, `EmailAlreadyExistsException`, `UserNotFoundException`, or `FailedToUpdateUserException`.
07	`updateClientProfile(userId, body)`	A method to validate the Client profile context and upsert the profile. – `userId`: `number` – target user identifier. – `body`: `AdminUpdateClientProfileBodyType` – company fields. – *Returns:* `Promise<AdminClientProfileResponseType>` – the updated Client profile. – Throws `UserNotFoundException`, `NoClientRoleForClientProfileException`, or `FailedToUpdateClientProfileException`.

No	Method	Description
08	`updateFreelancerProfile(userId, body)`	A method to validate the Freelancer context and upsert profile fields. – `userId`: `number` – target user identifier. – `body`: `AdminUpdateFreelancerProfileBodyType` – title, bio, languages, education, and certifications. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `NoFreelancerRoleForProfileException` or `FailedToUpdateFreelancerProfileException`.
09	`replaceFreelancerSkills(userId, body)`	A method to replace all Freelancer skills and create missing catalogue skills. – `userId`: `number` – target user identifier. – `body`: `AdminReplaceFreelancerSkillsBodyType` – skill name and proficiency pairs. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `NoFreelancerRoleForProfileException` or `FailedToSaveSkillsException`.
10	`createPortfolioItem(userId, body)`	A method to create a portfolio item on the Freelancer profile. – `userId`: `number` – target user identifier. – `body`: `AdminCreatePortfolioItemBodyType` – item fields. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `FailedToCreatePortfolioItemException`.
11	`updatePortfolioItem(userId, itemId, body)`	A method to update an owned portfolio item. – `userId`: `number` – target user identifier. – `itemId`: `number` – portfolio item identifier. – `body`: `AdminUpdatePortfolioItemBodyType` – optional item fields. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `PortfolioItemNotFoundException` or `FailedToUpdatePortfolioItemException`.

No	Method	Description
12	`deletePortfolioItem(userId, itemId)`	A method to soft-delete an owned portfolio item. – `userId`: `number` – target user identifier. – `itemId`: `number` – portfolio item identifier. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `PortfolioItemNotFoundException` or `FailedToDeletePortfolioItemException`.
13	`getFreelancerProfileContext(userId)`	A private method to load the profile and role context for a Freelancer edit. – `userId`: `number` – target user identifier. – *Returns:* `Promise<context>` – profile id, Freelancer profile id, and role flag. – Throws `UserNotFoundException` or `NoFreelancerRoleForProfileException`.
14	`ensureFreelancerProfileRow(userId, context)`	A private method to create the Profile and FreelancerProfile rows when missing. – `userId`: `number` – target user identifier. – `context`: object – profile id and role context. – *Returns:* `Promise<number | null>` – the Freelancer profile id. – Used by skill and portfolio editors before writing child rows.

4.16.2 ClassAttribute

No	Attribute	Type
01	`repository`	`UsersRepository` – private readonly persistence dependency.
02	`hashingService`	`HashingService` – private readonly password utility.

4.17 UsersRepository Class

4.17.1 ClassMethod

No	Method	Description
01	`UsersRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`getUsers(query: AdminUserQueryType)`	A method to query users with search, role, status, sorting, and pagination. – `query`: `AdminUserQueryType` – list filters. – *Returns:* `Promise<AdminUserListResponseType>` – users and pagination.
03	`createUserByAdmin(data)`	A method to create the account, profile, and primary role in one write. – `data`: object – email, hashed password, full name, role id, and role name. – *Returns:* `Promise<AdminCreateUserResponseType>` – the created account. – Creates an empty Expert profile when the role is Expert.
04	`findUserByEmail(email)`	A method to find a non-deleted account by email. – `email`: `string` – account email. – *Returns:* `Promise<{ id: number } | null>` – the account id, or `null`.
05	`findActiveRoleById(id)`	A method to load an active role by id. – `id`: `number` – role identifier. – *Returns:* `Promise<{ id: number; name: string } | null>` – the role, or `null`.
06	`updateUserByAdmin(id, data)`	A method to update account fields, profile name, and sessions on ban. – `id`: `number` – target account. – `data`: object – optional email, full name, and ban flag. – *Returns:* `Promise<AdminUpdateUserResponseType | null>` – the updated account, or `null` when absent. – Setting `isBanned` to true deletes every session in the same transaction.
07	`findClientProfileEditContext(id)`	A method to load the profile context required by the Client editor. – `id`: `number` – target account. – *Returns:* `Promise<context | null>` – profile id, Client profile id, and role flag.

No	Method	Description
08	`upsertClientProfileBy Admin(userId, data)`	A method to create or update the Client profile. – `userId`: `number` – target account. – `data`: object – company name, description, and website. – *Returns:* `Promise<ClientProfile | null>` – the persisted Client profile.
09	`findFreelancerProfileEdit Context(id)`	A method to load the profile context required by the Freelancer editor. – `id`: `number` – target account. – *Returns:* `Promise<context | null>` – profile id, Freelancer profile id, and role flag.
10	`updateFreelancerProfileBy Admin(userId, data)`	A method to update Profile biography and Freelancer profile fields. – `userId`: `number` – target account. – `data`: object – title, bio, languages, education, and certifications. – *Returns:* `Promise<void>` – no value.
11	`ensureFreelancerProfile( userId, profileId)`	A method to create the missing Profile or FreelancerProfile row. – `userId`: `number` – target account. – `profileId`: `number | null` – existing profile id, or `null`. – *Returns:* `Promise<number | null>` – the Freelancer profile id.
12	`replaceFreelancerSkills( freelancerProfileId, skills)`	A method to replace all skills and create missing catalogue skills in a transaction. – `freelancerProfileId`: `number` – Freelancer profile identifier. – `skills`: array of skill name and proficiency pairs. – *Returns:* `Promise<void>` – no value. – Free-text names are matched case-insensitively; missing skills receive a generated slug.
13	`createPortfolioItem( freelancerProfileId, data)`	A method to insert a portfolio item. – `freelancerProfileId`: `number` – Freelancer profile identifier. – `data`: object – title, description, technologies, and project URL. – *Returns:* `Promise<PortfolioItem>` – the created item.

No	Method	Description
14	`findPortfolioItemOwned(id)`	A method to load a portfolio item with its owner and delete state. – `id`: `number` – portfolio item identifier. – *Returns:* `Promise<{ id: number; freelancerProfileId: number; deletedAt: Date | null } | null>` – the ownership record, or `null`.
15	`updatePortfolioItemByAdmin(id, data)`	A method to update only the provided portfolio item fields. – `id`: `number` – portfolio item identifier. – `data`: object – optional item fields. – *Returns:* `Promise<PortfolioItem>` – the updated item.
16	`softDeletePortfolioItem(id)`	A method to set the portfolio item delete timestamp. – `id`: `number` – portfolio item identifier. – *Returns:* `Promise<PortfolioItem>` – the soft-deleted item.
17	`listActiveSkillCatalog()`	A method to return active skills ordered by name. – *Returns:* `Promise<AdminSkillCatalogListType>` – the catalogue.
18	`getUserById(id)`	A method to load the full account graph for the detail page. – `id`: `number` – target account. – *Returns:* `Promise<AdminUserDetailResponseType | null>` – the detail view, or `null`.

4.17.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.18 BannersAdminController Class

4.18.1 ClassMethod

No	Method	Description
01	`BannersAdminController(service: BannersAdminService)`	A method to initialize the controller with the banner administration service. – `service`: `BannersAdminService` – service that owns banner use cases. – *Returns:* `void` – constructor, no return value.
02	`listBanners(query: BannerAdminQueryDto)`	A method to return a filtered, paginated banner list. – `query`: `BannerAdminQueryDto` – page, limit, search, position, active, and sort options. – *Returns:* `Promise<BannerAdminListResponseType>` – banners and pagination. – Route: `GET /admin/banners`.
03	`createBanner(body: CreateBannerBodyDto)`	A method to create an advertisement banner. – `body`: `CreateBannerBodyDto` – title, image URL, link, position, active state, and dates. – *Returns:* `Promise<BannerAdminDetailResponseType>` – the created banner. – Route: `POST /admin/banners`.
04	`updateBanner(id: number, body: UpdateBannerBodyDto)`	A method to update a banner. – `id`: `number` – banner identifier. – `body`: `UpdateBannerBodyDto` – optional banner fields. – *Returns:* `Promise<BannerAdminDetailResponseType>` – the updated banner. – Route: `PATCH /admin/banners/:id`.
05	`deleteBanner(id: number)`	A method to permanently delete a banner. – `id`: `number` – banner identifier. – *Returns:* `Promise<BannerAdminDeleteResponseType>` – a success message. – Route: `DELETE /admin/banners/:id`.
06	`uploadImage(file?: Express.Multer.File)`	A method to upload a banner image and return its URL. – `file`: `Express.Multer.File | undefined` – multipart image field named `file`. – *Returns:* `Promise<{ imageUrl: string; publicId: string }>` – the stored image reference. – Route: `POST /admin/banners/upload-image`; multipart field `file`; maximum 10 MB.

No	Method	Description
07	`getBannerDetail(id: number)`	A method to return one banner. – `id: number` – banner identifier. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the banner detail. – Route: `GET /admin/banners/:id`.

4.18.2 ClassAttribute

No	Attribute	Type
01	`service`	`BannersAdminService` – private readonly banner administration service.

4.19 BannersAdminService Class

4.19.1 ClassMethod

No	Method	Description
01	`BannersAdminService( repository: BannersAdmin Repository, cloudinaryService: CloudinaryService)`	A method to initialize the service with persistence and image storage. – `repository: BannersAdminRepository` – banner persistence dependency. – `cloudinaryService: CloudinaryService` – remote image storage. – *Returns:* `void` – constructor, no return value.
02	`uploadImage(file?: Express.Multer.File)`	A method to validate an image and store it on Cloudinary or the local upload directory. – `file: Express.Multer.File | undefined` – multipart image. – *Returns:* `Promise<{ imageUrl: string; public Id: string }>` – the stored image reference. – Throws `BannerImageRequiredException`, `Banner ImageInvalidTypeException`, `BannerImageTooLarge Exception`, or `FailedToUploadBannerImage Exception`.

No	Method	Description
03	`listBanners(query: BannerAdminQueryType)`	A method to load a banner page. – `query: BannerAdminQueryType` – list filters. – *Returns:* `Promise<BannerAdminListResponseType>` – banners and pagination. – Failure is mapped to `FailedToLoadBannerList Exception`.
04	`getBannerDetail(id: number)`	A method to load one banner. – `id: number` – banner identifier. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the banner detail. – Throws `BannerAdminNotFoundException` or `Failed ToLoadBannerDetailException`.
05	`createBanner(body: BannerCreateBodyType)`	A method to create a banner after validating its position. – `body: BannerCreateBodyType` – banner fields. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the created banner. – Throws `BannerPositionTakenException` or `Failed ToCreateBannerException`.
06	`updateBanner(id: number, body: BannerUpdateBody Type)`	A method to update a banner after validating its position. – `id: number` – banner identifier. – `body: BannerUpdateBodyType` – optional banner fields. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the updated banner. – Throws `BannerAdminNotFoundException`, `Banner PositionTakenException`, or `FailedToUpdateBanner Exception`.
07	`deleteBanner(id: number)`	A method to permanently delete a banner. – `id: number` – banner identifier. – *Returns:* `Promise<BannerAdminDeleteResponse Type>` – a success message. – Throws `BannerAdminNotFoundException` or `Failed ToDeleteBannerException`.

4.19.2 ClassAttribute

No	Attribute	Type
01	`repository`	`BannersAdminRepository` – private readonly persistence dependency.
02	`cloudinaryService`	`CloudinaryService` – private readonly image storage dependency.

4.20 BannersAdminRepository Class

4.20.1 ClassMethod

No	Method	Description
01	`BannersAdminRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`listBanners(query: BannerAdminQueryType)`	A method to query banners with search, position, active-state, and sorting filters. – `query`: `BannerAdminQueryType` – list filters. – *Returns:* `Promise<BannerAdminListResponseType>` – banners and pagination.
03	`getBannerDetail(id: number)`	A method to load one banner. – `id`: `number` – banner identifier. – *Returns:* `Promise<BannerAdminDetailResponse Type | null>` – the banner, or `null`.
04	`createBanner(data: BannerCreateBodyType)`	A method to validate the position constraint and insert a banner. – `data`: `BannerCreateBodyType` – banner fields. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the created banner. – Throws `BannerPositionTakenException`.

No	Method	Description
05	`updateBanner(id, data)`	A method to validate the position constraint and update a banner. – `id`: `number` – banner identifier. – `data`: `BannerUpdateBodyType` – optional banner fields. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the updated banner. – Throws `BannerAdminNotFoundException` or `Banner PositionTakenException`.
06	`deleteBanner(id: number)`	A method to delete a banner and report whether a row was removed. – `id`: `number` – banner identifier. – *Returns:* `Promise<BannerAdminDeleteResponse Type>` – a success message. – Throws `BannerAdminNotFoundException`.

4.20.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.21 SkillsAdminController Class

4.21.1 ClassMethod

No	Method	Description
01	`SkillsAdminController( service: SkillsAdmin Service)`	A method to initialize the controller with the skill administration service. – `service`: `SkillsAdminService` – service that owns skill use cases. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`listSkills(query: SkillAdminQueryDto)`	A method to return a filtered, paginated skill list. – `query`: `SkillAdminQueryDto` – page, limit, search, active, and sort options. – *Returns:* `Promise<SkillAdminListResponseType>` – skills and pagination. – Route: `GET /admin/skills`.
03	`createSkill(body: CreateSkillBodyDto)`	A method to create a skill. – `body`: `CreateSkillBodyDto` – name and optional description. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the created skill. – Route: `POST /admin/skills`.
04	`updateSkill(id: number, body: UpdateSkillBodyDto)`	A method to update a skill. – `id`: `number` – skill identifier. – `body`: `UpdateSkillBodyDto` – optional name, description, and active state. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the updated skill. – Route: `PATCH /admin/skills/:id`.
05	`restoreSkill(id: number)`	A method to restore a soft-deleted skill. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the restored skill. – Route: `PATCH /admin/skills/:id/restore`.
06	`deleteSkill(id: number)`	A method to soft-delete a skill. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDeleteResponseType>` – a success message. – Route: `DELETE /admin/skills/:id`.
07	`getSkillDetail(id: number)`	A method to return one skill. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the skill detail. – Route: `GET /admin/skills/:id`.

4.21.2 ClassAttribute

No	Attribute	Type
01	`service`	`SkillsAdminService` – private readonly skill administration service.

4.22 SkillsAdminService Class

4.22.1 ClassMethod

No	Method	Description
01	`SkillsAdminService( repository: SkillsAdmin Repository)`	A method to initialize the service with the skill repository. – `repository`: `SkillsAdminRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`listSkills(query: Skill AdminQueryType)`	A method to load a skill page. – `query`: `SkillAdminQueryType` – list filters. – *Returns:* `Promise<SkillAdminListResponseType>` – skills and pagination. – Failure is mapped to `FailedToLoadSkillList Exception`.
03	`getSkillDetail(id: number)`	A method to load one skill. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponse Type>` – the skill detail. – Throws `SkillAdminNotFoundException` or `Failed ToLoadSkillDetailException`.
04	`createSkill(body: Admin CreateSkillBodyType)`	A method to create a skill and handle a concurrent duplicate. – `body`: `AdminCreateSkillBodyType` – name and optional description. – *Returns:* `Promise<SkillAdminDetailResponse Type>` – the created skill. – Throws `SkillNameAlreadyExistsException` or `FailedToCreateSkillException`.

No	Method	Description
05	`updateSkill(id, body)`	A method to update a skill and handle a concurrent duplicate. – `id: number` – skill identifier. – `body: AdminUpdateSkillBodyType` – optional name, description, and active state. – *Returns:* `Promise<SkillAdminDetailResponse Type>` – the updated skill. – Throws `SkillAdminNotFoundException`, `SkillName AlreadyExistsException`, or `FailedToUpdateSkill Exception`.
06	`deleteSkill(id: number)`	A method to soft-delete a skill. – `id: number` – skill identifier. – *Returns:* `Promise<SkillAdminDeleteResponse Type>` – a success message. – Throws `SkillAdminNotFoundException` or `Failed ToDeleteSkillException`.
07	`restoreSkill(id: number)`	A method to restore a soft-deleted skill. – `id: number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponse Type>` – the restored skill. – Throws `SkillAdminNotFoundException`, `SkillName AlreadyExistsException`, or `FailedToRestoreSkill Exception`.

4.22.2 ClassAttribute

No	Attribute	Type
01	`repository`	`SkillsAdminRepository` – private readonly persistence dependency.

4.23 SkillsAdminRepository Class

4.23.1 ClassMethod

No	Method	Description
01	`SkillsAdminRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`listSkills(query: SkillAdminQueryType)`	A method to query skills with search, active-state, sorting, and pagination. – `query`: `SkillAdminQueryType` – list filters. – *Returns:* `Promise<SkillAdminListResponseType>` – skills and pagination. – Search matches name, slug, or description.
03	`getSkillDetail(id: number)`	A method to load one skill. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponseType | null>` – the skill, or `null`.
04	`createSkill(data: AdminCreateSkillBodyType)`	A method to generate a unique slug and insert a skill. – `data`: `AdminCreateSkillBodyType` – name and optional description. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the created skill. – Throws `SkillNameAlreadyExistsException`.
05	`updateSkill(id, data)`	A method to update a skill and regenerate its slug when the name changes. – `id`: `number` – skill identifier. – `data`: `AdminUpdateSkillBodyType` – optional name, description, and active state. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the updated skill. – Throws `SkillAdminNotFoundException` or `SkillNameAlreadyExistsException`.
06	`deleteSkill(id: number)`	A method to set the skill delete timestamp. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDeleteResponseType>` – a success message. – Throws `SkillAdminNotFoundException`.

No	Method	Description
07	`restoreSkill(id: number)`	A method to clear the skill delete timestamp after checking name conflicts. – `id: number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponse Type>` – the restored skill. – Throws `SkillAdminNotFoundException` or `Skill NameAlreadyExistsException`.
08	`generateUniqueSlug(name: string)`	A private method to derive a slug and append a numeric suffix on collision. – `name: string` – skill name. – *Returns:* `Promise<string>` – the unique slug.

4.23.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.24 AdminExpertProfileController Class

4.24.1 ClassMethod

No	Method	Description
01	`AdminExpertProfile Controller(service: ExpertProfileService)`	A method to initialize the controller with the expert profile service. – `service: ExpertProfileService` – service that owns expert profile use cases. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`update( userId: number, adminId: number, body: AdminUpdateExpert ProfileDto)`	A method for an administrator to update an expert profile directly. – `userId: number` – expert account identifier. – `adminId: number` – current administrator, recorded on superseded revisions. – `body: AdminUpdateExpertProfileDto` – display name, bio, title, expertise, experience, education, certifications, website, and active state. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `PATCH /users/:id/expert-profile`.

4.24.2 ClassAttribute

No	Attribute	Type
01	`service`	`ExpertProfileService` – private readonly expert profile service.

4.25 ExpertProfileService Class

4.25.1 ClassMethod

No	Method	Description
01	`ExpertProfileService( repository: Expert ProfileRepository, revisions: Profile RevisionService)`	A method to initialize the service with persistence and revision handling. – `repository: ExpertProfileRepository` – expert profile persistence. – `revisions: ProfileRevisionService` – self-service revision workflow. – *Returns:* `void` – constructor, no return value.
02	`cleanList(values: string[])`	A private method to trim, de-duplicate, and preserve the first casing of a string list. – `values: string[]` – raw list values. – *Returns:* `string[]` – the cleaned list.

No	Method	Description
03	`sanitize(input)`	A private method to clean list fields and normalize the website value. – `input`: `UpdateExpertProfileType` – profile input. – *Returns:* `UpdateExpertProfileType` – the sanitized input.
04	`context(userId: number)`	A private method to load an expert account and verify its Expert role. – `userId`: `number` – target account. – *Returns:* `Promise<user context>` – account, profile, and roles. – Throws `ExpertProfileNotFoundException` or `NotAnExpertAccountException`.
05	`getMine(userId: number)`	A method to return the authenticated expert's profile. – `userId`: `number` – current expert account. – *Returns:* `Promise<ExpertProfileType>` – profile and expert fields. – Used by the self-service profile page, not by the admin endpoint.
06	`submitUpdate(userId, input)`	A method to submit a self-service expert profile revision. – `userId`: `number` – current expert account. – `input`: `UpdateExpertProfileType` – requested profile changes. – *Returns:* `Promise<revision result>` – the pending revision. – Throws `ExpertProfilePendingRevisionException` when a revision is already pending.
07	`updateByAdmin(userId, adminId, input)`	A method to update an expert profile directly and supersede pending revisions. – `userId`: `number` – target expert account. – `adminId`: `number` – current administrator. – `input`: `AdminUpdateExpertProfileType` – administrative profile changes. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `ExpertProfileNotFoundException` or `NotAnExpertAccountException`.

4.25.2 ClassAttribute

No	Attribute	Type
01	`repository`	`ExpertProfileRepository` – private readonly persistence dependency.
02	`revisions`	`ProfileRevisionService` – private readonly revision workflow dependency.

4.26 ExpertProfileRepository Class

4.26.1 ClassMethod

No	Method	Description
01	`ExpertProfileRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findByUserId(userId: number)`	A method to load an expert account with its profile and roles. – `userId`: `number` – target account. – *Returns:* `Promise<user context | null>` – the account graph, or `null`.
03	`updateDirect(userId, adminId, input, profile CompletionPercent)`	A method to supersede pending revisions, update the profile, and upsert the expert row in one transaction. – `userId`: `number` – target expert account. – `adminId`: `number` – reviewing administrator. – `input`: `AdminUpdateExpertProfileType` – sanitized profile changes. – `profileCompletionPercent`: `number` – recalculated completeness. – *Returns:* `Promise<void>` – no value. – Creates a system notification when a pending revision was superseded.

No	Method	Description
04	`hasPendingRevision(userId: number)`	A method to check for a pending expert revision. – `userId: number` – target account. – *Returns:* `Promise<{ id: number } | null>` – the pending revision, or `null`.
05	`findPublicMany(query: PublicExpertQueryType)`	A method to query active public expert profiles. – `query: PublicExpertQueryType` – expertise, search, and pagination filters. – *Returns:* `Promise<public expert page>` – public expert cards.
06	`findPublicById(id: number)`	A method to load one active public expert profile. – `id: number` – expert identifier. – *Returns:* `Promise<public expert | null>` – the public profile, or `null`.

4.26.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.27 JobCategoriesAdminController Class

4.27.1 ClassMethod

No	Method	Description
01	`JobCategoriesAdminController(service: JobCategoriesAdminService)`	A method to initialize the controller with the job category service. – `service: JobCategoriesAdminService` – service that owns job category use cases. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`listJobCategories(query: JobCategoryAdminQueryDto)`	A method to return a filtered, paginated job category list. – `query: JobCategoryAdminQueryDto` – page, limit, search, status, deleted, and sort options. – *Returns:* `Promise<JobCategoryAdminListResponseType>` – categories and pagination. – Route: `GET /admin/job-categories`.
03	`createJobCategory(body: CreateJobCategoryBodyDto)`	A method to create a job category. – `body: CreateJobCategoryBodyDto` – name, description, icon, and status. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the created category. – Route: `POST /admin/job-categories`.
04	`updateJobCategory(id: number, body: UpdateJobCategoryBodyDto)`	A method to update a job category. – `id: number` – category identifier. – `body: UpdateJobCategoryBodyDto` – optional category fields. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the updated category. – Route: `PATCH /admin/job-categories/:id`.
05	`restoreJobCategory(id: number)`	A method to restore a soft-deleted job category. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the restored category. – Route: `PATCH /admin/job-categories/:id/restore`.
06	`deleteJobCategory(id: number)`	A method to soft-delete a job category that has no active jobs. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDeleteResponseType>` – a success message. – Route: `DELETE /admin/job-categories/:id`.
07	`getJobCategoryDetail(id: number)`	A method to return one job category. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the category detail. – Route: `GET /admin/job-categories/:id`.

4.27.2 ClassAttribute

No	Attribute	Type
01	`service`	`JobCategoriesAdminService` – private readonly job category service.

4.28 JobCategoriesAdminService Class

4.28.1 ClassMethod

No	Method	Description
01	`JobCategoriesAdmin Service( repository: Job CategoriesAdmin Repository)`	A method to initialize the service with the job category repository. – `repository: JobCategoriesAdminRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`listJobCategories(query: JobCategoryAdminQuery Type)`	A method to load a job category page. – `query: JobCategoryAdminQueryType` – list filters. – *Returns:* `Promise<JobCategoryAdminListResponse Type>` – categories and pagination. – Failure is mapped to `FailedToLoadJobCategoryList Exception`.
03	`getJobCategoryDetail(id: number)`	A method to load one job category. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the category detail. – Throws `JobCategoryAdminNotFoundException` or `FailedToLoadJobCategoryDetailException`.
04	`createJobCategory(body: AdminCreateJobCategory BodyType)`	A method to create a category and handle a concurrent duplicate. – `body: AdminCreateJobCategoryBodyType` – category fields. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the created category. – Throws `JobCategoryNameAlreadyExistsException` or `FailedToCreateJobCategoryException`.

No	Method	Description
05	`updateJobCategory(id, body)`	A method to update a category and handle a concurrent duplicate. – `id: number` – category identifier. – `body: AdminUpdateJobCategoryBodyType` – optional category fields. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the updated category. – Throws `JobCategoryAdminNotFoundException`, `JobCategoryNameAlreadyExistsException`, or `FailedToUpdateJobCategoryException`.
06	`deleteJobCategory(id: number)`	A method to soft-delete a category after checking active job references. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDeleteResponseType>` – a success message. – Throws `JobCategoryAdminNotFoundException`, `JobCategoryHasJobsException`, or `FailedToDeleteJobCategoryException`.
07	`restoreJobCategory(id: number)`	A method to restore a soft-deleted category. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the restored category. – Throws `JobCategoryAdminNotFoundException`, `JobCategoryNameAlreadyExistsException`, or `FailedToRestoreJobCategoryException`.

4.28.2 ClassAttribute

No	Attribute	Type
01	`repository`	`JobCategoriesAdminRepository` – private readonly persistence dependency.

4.29 JobCategoriesAdminRepository Class

4.29.1 ClassMethod

No	Method	Description
01	`JobCategoriesAdmin Repository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`listJobCategories(query: JobCategoryAdminQuery Type)`	A method to query categories with search, status, deleted-state, sorting, and pagination. – `query`: `JobCategoryAdminQueryType` – list filters. – *Returns:* `Promise<JobCategoryAdminListResponse Type>` – categories and pagination. – Search matches name, slug, or description.
03	`getJobCategoryDetail(id: number)`	A method to load one category with its job count. – `id`: `number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType | null>` – the category, or `null`.
04	`createJobCategory(data)`	A method to generate a unique slug and insert a category. – `data`: `AdminCreateJobCategoryBodyType` – category fields. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the created category. – Throws `JobCategoryNameAlreadyExistsException`.
05	`updateJobCategory(id, data)`	A method to update a category and regenerate its slug when the name changes. – `id`: `number` – category identifier. – `data`: `AdminUpdateJobCategoryBodyType` – optional category fields. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the updated category. – Throws `JobCategoryAdminNotFoundException` or `JobCategoryNameAlreadyExistsException`.
06	`deleteJobCategory(id: number)`	A method to check active job references and set the delete timestamp. – `id`: `number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDelete ResponseType>` – a success message. – Throws `JobCategoryAdminNotFoundException` or `JobCategoryHasJobsException`.

No	Method	Description
07	`restoreJobCategory(id: number)`	A method to clear the delete timestamp after checking name conflicts. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the restored category. – Throws `JobCategoryAdminNotFoundException` or `JobCategoryNameAlreadyExistsException`.
08	`generateUniqueSlug(name: string)`	A private method to derive a slug and append a numeric suffix on collision. – `name: string` – category name. – *Returns:* `Promise<string>` – the unique slug.

4.29.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.